

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA CARTOGRÁFICA**

**VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA**

**BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS
UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM
TOOL CALLING**

**RIO DE JANEIRO
2026**

©2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmар ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Corrêa, Vinicius Henequim; Santana, Daniel.

Busca de Geoinformação em Catálogo de Dados Espaciais Utilizando Modelos de Linguagem de Grande Porte com Tool Calling / Vinicius Henequim Corrêa e Daniel Santana. – Rio de Janeiro, 2026.

113 f.

Orientador(es): Ivanildo Barbosa.

Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia Cartográfica, 2026.

1. Modelos de Linguagem de Grande Porte. 2. Tool Calling. 3. Catálogo de Dados Espaciais. 4. Geoinformação. 5. Ollama. 6. LangChain. 7. Avaliação de Modelos. i. Barbosa, Ivanildo (orient.) ii. Título

VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA

BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS
UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM
TOOL CALLING

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia Cartográfica.

Orientador(es): Ivanildo Barbosa, DSc

Rio de Janeiro

2026

RESUMO

O acervo cartográfico da Diretoria de Serviço Geográfico (DSG) é organizado por nomenclaturas técnicas — Índice de Nomenclatura (INOM), Mapa Índice (MI), escalas e tipos de produto do Sistema Cartográfico Nacional — cujo domínio é uma barreira para usuários não especialistas. O 1º Centro de Geoinformação (1º CGEO) desenvolveu um protótipo de busca em linguagem natural com Saída Estruturada e um único modelo local (Phi-4 14B), que demonstrou a viabilidade da abordagem, mas dependia de um dicionário de normalização fixo no código e não dispunha de um protocolo de avaliação. Este Projeto Final de Curso aprimorou a **camada de tradução linguística** dessa solução — a etapa em que a consulta livre do usuário é convertida em parâmetros estruturados de busca. O *backend* foi reimplementado em Python, com FastAPI e LangChain, e a Saída Estruturada foi substituída por *Tool Calling* nativo, sem dicionário de normalização e sem exemplos no *prompt*. Para medir a qualidade da tradução, construiu-se um *dataset* de 310 consultas em três camadas — casos de teste do protótipo, consultas redigidas pelos autores e consultas geradas programaticamente — com gabarito definido por um manual de anotação, leituras múltiplas explícitas para as consultas ambíguas e uma auditoria em três etapas (checagens automáticas, anotação independente às cegas e adjudicação registrada), que obteve concordância quase perfeita entre o gabarito e a anotação independente. Um *pipeline* reprodutível executou o *dataset* sobre quatro modelos abertos executáveis localmente via Ollama (Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B) e, como referência, sobre dois modelos de maior porte servidos em nuvem, medindo acurácia com intervalo de confiança, precisão, *recall* e F1 por campo, latência e significância das diferenças entre modelos. Três dos modelos locais — Gemma 4 E4B, Qwen 3 4B e Gemma 4 E2B — tiveram acurácias estatisticamente equivalentes, de 55,4%, 52,1% e 50,7%; o Mistral Nemo 12B ficou em 22,5%, por descrever a chamada da ferramenta em texto em vez de emití-la. Todos recusaram as consultas fora do domínio, e datas relativas e ordenação foram as categorias mais difíceis. Na estação de referência, com 4 GB de VRAM, o Gemma 4 E2B teve latência mediana de 0,7 s, contra 2,6 s do Qwen 3 4B, e é o modelo recomendado para uso operacional; os modelos em nuvem, nas mesmas consultas, acertaram 77,5% e 81,7%, o que indica a margem de melhoria disponível. O *dataset* auditado e o *pipeline* permitem repetir a comparação, com o mesmo critério, sempre que surgirem novos modelos.

Palavras-chave: Modelos de Linguagem de Grande Porte. *Tool Calling*. Catálogo de Dados Espaciais. Geoinformação. Ollama. LangChain. Avaliação de Modelos.

ABSTRACT

The cartographic collection of the Brazilian Army Geographic Service Directorate (DSG) is organized by technical nomenclatures — the Nomenclature Index (INOM), the Index Map (MI), and the scales and product types of the National Cartographic System — whose mastery is a barrier for non-specialist users. The 1st Geoinformation Center (1^o CGEO) developed a natural-language search prototype based on Structured Output and a single local model (Phi-4 14B), which demonstrated the feasibility of the approach but relied on a normalization dictionary hard-coded in the source and had no evaluation protocol. This Final Course Project improved the **natural-language translation layer** of that solution — the step in which the user’s free-form query is converted into structured search parameters. The backend was re-implemented in Python, with FastAPI and LangChain, and Structured Output was replaced by native Tool Calling, with no normalization dictionary and no worked examples in the prompt. To measure translation quality, a 310-query dataset was built in three layers — the prototype’s test cases, queries written by the authors, and programmatically generated queries — with ground truth defined by a written annotation manual, explicit multiple readings for ambiguous queries, and a three-stage audit (automatic checks, blind independent annotation, and recorded adjudication) that showed near-perfect agreement between the ground truth and the independent annotation. A reproducible pipeline ran the dataset on four open models that run locally through Ollama (Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B, and Mistral Nemo 12B) and, as a reference, on two larger cloud-served models, measuring accuracy with confidence intervals, per-field precision, recall and F1, latency, and the significance of differences between models. Three of the local models — Gemma 4 E4B, Qwen 3 4B, and Gemma 4 E2B — reached statistically equivalent accuracy, while Mistral Nemo 12B lagged far behind because it described the tool call in plain text instead of emitting it. All models correctly declined out-of-domain queries, and relative dates and sorting were the hardest categories. On the 4 GB reference workstation, Gemma 4 E2B had the lowest latency at the same accuracy and is the model recommended for operational use; the larger cloud-served models scored substantially higher on the same queries, which indicates the room for improvement available to local models. The audited dataset and the pipeline allow the comparison to be repeated, under the same criteria, whenever new models become available.

Keywords: Large Language Models. Tool Calling. Spatial Data Catalog. Geoinformation. Ollama. LangChain. Model Evaluation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo do mecanismo de <i>Tool Calling</i>	19
Figura 2 – Exemplo simplificado de definição de ferramenta para busca em catálogo de produtos cartográficos	20
Figura 3 – Estrutura Analítica do Projeto (EAP) adotada no aprimoramento . . .	40
Figura 4 – Arquitetura do protótipo de busca do 1º CGEO	41
Figura 5 – Arquitetura da solução aprimorada de busca	45
Figura 6 – Distribuição da latência da tradução nas rodadas completas, em escala logarítmica, por modelo	56
Figura 7 – Acurácia por consulta, por categoria e por modelo, nas rodadas completas	57
Figura 8 – F1-score por campo do <i>schema</i> de busca, por modelo, nas rodadas completas	59
Figura 9 – Distribuição da latência da tradução na estação de referência, em escala logarítmica	63
Figura 10 – Acurácia por consulta, por categoria, na estação de referência	63

LISTA DE TABELAS

Tabela 1	– Comparativo entre consulta técnica e consulta em linguagem natural	13
Tabela 2	– Modelos com suporte a <i>Tool Calling</i> disponíveis no Ollama (seleção)	23
Tabela 3	– Estrutura de cada entrada do <i>dataset</i> de avaliação	30
Tabela 4	– Composição do <i>dataset</i> de avaliação por origem e família	31
Tabela 5	– Consultas por categoria e por camada (métricas principais; uma consulta pode ter mais de uma categoria)	32
Tabela 6	– Frequência de cada campo na leitura preferencial do gabarito (métricas principais)	33
Tabela 7	– Resultados da auditoria do <i>dataset</i>	35
Tabela 8	– Ambiente de execução de cada rodada	54
Tabela 9	– Comparativo de desempenho entre os modelos avaliados	55
Tabela 10	– Latência da tradução, em milissegundos, por modelo	55
Tabela 11	– F1-score por categoria de consulta e por modelo	57
Tabela 12	– F1-score por campo do <i>schema</i> de busca, por modelo	58
Tabela 13	– Ambiente de execução de cada rodada — estação de referência (camadas P e N)	61
Tabela 14	– Comparativo de desempenho entre os modelos avaliados — estação de referência (camadas P e N)	61
Tabela 15	– Latência da tradução, em milissegundos, por modelo — estação de referência (camadas P e N)	61
Tabela 16	– Teste de McNemar exato entre pares de modelos — estação de referência (camadas P e N)	62
Tabela 17	– Mesmo modelo nas rodadas completas e na estação de referência (camadas P e N, primeira repetição)	64
Tabela 18	– Qwen 3 4B executado duas vezes na estação de referência, com os mesmos pesos, prompt, ferramenta e data de referência (camadas P e N)	64
Tabela 19	– Ambiente de execução de cada rodada — referência em nuvem (Groq)	65
Tabela 20	– Comparativo de desempenho entre os modelos avaliados — referência em nuvem (Groq)	65
Tabela 21	– F1-score por categoria de consulta e por modelo — referência em nuvem (Groq)	65
Tabela 22	– Teste de McNemar exato entre pares de modelos	66
Tabela 31	– Concordância por campo entre o gabarito e a anotação independente (leitura preferencial, após a adjudicação)	108

LISTA DE ABREVIATURAS E SIGLAS

1º CGEO	1º Centro de Geoinformação
ABNT	Associação Brasileira de Normas Técnicas
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
BDGEx	Banco de Dados Geográficos do Exército
DSG	Diretoria de Serviço Geográfico
ET-PCDG	Especificações Técnicas para Produtos de Conjuntos de Dados Geospaciais
F1	<i>F1-score</i>
GGUF	<i>GPT-Generated Unified Format</i>
GPU	<i>Graphics Processing Unit</i> (Unidade de Processamento Gráfico)
INDE	Infraestrutura Nacional de Dados Espaciais
INOM	Índice de Nomenclatura
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i> (Modelo de Linguagem de Grande Porte)
MI	Mapa Índice
NLP	<i>Natural Language Processing</i> (Processamento de Linguagem Natural)
OGC	<i>Open Geospatial Consortium</i>
PFC	Projeto Final de Curso
REST	<i>Representational State Transfer</i>
SCN	Sistema Cartográfico Nacional
SQL	<i>Structured Query Language</i>
VRAM	<i>Video Random Access Memory</i> (memória da GPU)
WCS	<i>Web Coverage Service</i>

WFS *Web Feature Service*

WMS *Web Map Service*

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Contextualização	12
1.2	Problema	13
1.3	Motivação	13
1.4	Lacuna Identificada	15
1.5	Objetivos	16
1.5.1	Objetivo Geral	16
1.5.2	Entregáveis do Projeto	16
1.6	Delimitação do Trabalho	16
1.7	Estrutura do Trabalho	17
2	REFERENCIAL TEÓRICO	18
2.1	Catálogos de Dados Espaciais	18
2.2	Tool Calling	19
2.2.1	Tool Calling <i>versus</i> Saída Estruturada	20
2.3	Modelos de Linguagem de Grande Porte e Execução Local	21
2.3.1	Vantagens da Execução Local	22
2.3.2	Ollama e Modelos com Suporte a Tool Calling	22
2.4	Orquestração com LangChain	23
2.5	Métricas de Avaliação	24
3	METODOLOGIA	26
3.1	Escopo do Projeto	26
3.1.1	Entregáveis	26
3.1.2	Exclusões	27
3.1.3	Requisitos que norteiam o escopo	27
3.2	Seleção de Modelos	28
3.2.1	Critérios de Seleção	28
3.2.2	Modelos Avaliados	28
3.2.3	Referência paralela em nuvem	29
3.3	Construção do <i>Dataset</i> de Avaliação	29
3.3.1	Estrutura de cada entrada	29
3.3.2	Categorias de Consultas	30
3.3.3	Camadas de Construção	30
3.3.4	Manual de Anotação e Leituras Múltiplas	32
3.4	Auditoria do <i>Dataset</i>	33

3.5	Protocolo de Avaliação	35
3.5.1	Visão Geral	35
3.5.2	Ambientes de Execução	36
3.5.3	Configuração dos Modelos	37
3.5.4	Métricas	37
3.5.5	Reprodutibilidade	39
4	APRIMORAMENTO DO PROTÓTIPO: ESTRUTURA ANALÍTICA DO PROJETO	40
4.1	Análise do Protótipo do 1º CGEO	40
4.1.1	Visão Geral	40
4.1.2	Arquitetura do Protótipo	41
4.1.3	Contribuições do Protótipo	42
4.1.4	Limitações Identificadas	42
4.2	Elicitação de Requisitos	43
4.2.1	Requisitos Funcionais	43
4.2.2	Requisitos Não Funcionais	44
4.3	Planejamento da Migração	44
4.3.1	Mapeamento Limitação → Solução	44
4.3.2	Componentes Reaproveitados, Reescritos e Novos	44
4.4	Desenvolvimento	45
4.4.1	Camada de API: FastAPI	46
4.4.2	Camada de Orquestração: LangChain + Ollama	47
4.4.3	Ferramentas de Busca	48
4.4.4	Camada de Persistência	49
4.4.5	Pipeline de Avaliação	50
4.4.6	Ferramentas de Apoio ao Desenvolvimento	51
4.5	Cronograma	52
5	RESULTADOS	54
5.1	Configuração Experimental	54
5.2	Comparativo Geral entre Modelos	55
5.3	Análise por Categoria de Consulta	57
5.4	Análise por Campo do <i>Schema</i>	58
5.5	Análise de Erros	59
5.5.1	Casos observacionais	60
5.6	Validação na Estação de Referência	61
5.7	Referência em Nuvem	64
5.8	Análise Estatística	66
5.9	Discussão	66

5.9.1	Recomendação de Modelo para Uso Operacional	67
6	CONCLUSÃO	69
6.1	Síntese do Trabalho	69
6.2	Retomada dos Objetivos	69
6.3	Principais Resultados	70
6.4	Limitações do Trabalho	71
6.5	Trabalhos Futuros	72
6.6	Considerações Finais	73
	REFERÊNCIAS	74
	APÊNDICE A – DATASET DE AVALIAÇÃO	77
A.1	Manual de anotação	77
A.2	Consultas do protótipo (P01–P22)	80
A.3	Consultas redigidas pelos autores (N01–N40)	86
A.4	Geração programática da camada G	93
A.5	Consultas simples geradas por <i>templates</i> (G-S)	94
A.6	Consultas compostas geradas por <i>templates</i> (G-C)	96
A.7	Consultas com código MI/INOM geradas por <i>templates</i> (G-M)	98
A.8	Consultas com tempo relativo geradas por <i>templates</i> (G-T)	100
A.9	Consultas com ordenação geradas por <i>templates</i> (G-O)	101
A.10	Consultas ambíguas/informais adicionais (G-A)	103
A.11	Consultas amplas, com muitos produtos (G-P)	104
A.12	Consultas de fronteira do domínio (G-F)	104
A.13	Requisitos exercitados pelas consultas	105
A.14	Auditoria do <i>dataset</i>	105
	APÊNDICE B – ESPECIFICAÇÃO DOS AMBIENTES DE EXE- CUÇÃO	109
B.1	Estação de referência	109
B.1.1	Comandos PowerShell de coleta	109
B.1.2	Saída coletada	110
B.2	Rodadas completas em GPU de nuvem	110
B.3	Referência em nuvem (Groq)	111
	APÊNDICE C – REPOSITÓRIO DE CÓDIGO E REPRODUÇÃO DOS EXPERIMENTOS	112

1 INTRODUÇÃO

1.1 Contextualização

A geoinformação ocupa papel estratégico nas atividades de defesa, planejamento territorial, gestão ambiental e tomada de decisão em diferentes esferas do Estado brasileiro. O Brasil dispõe de uma vasta infraestrutura de produção e disseminação de dados espaciais, cujo arcabouço normativo é definido pelo Decreto n.º 6.666/2008, que instituiu a Infraestrutura Nacional de Dados Espaciais (INDE) (Brasil, 2008). A INDE tem por objetivo organizar, em âmbito federal, o acesso integrado a conjuntos de dados geoespaciais produzidos e mantidos pelos órgãos governamentais, promovendo a interoperabilidade e o compartilhamento dessas informações.

No contexto do Exército Brasileiro, a Diretoria de Serviço Geográfico (DSG) tem por missão orientar, no âmbito do Exército, as atividades referentes a imagens e informações geográficas, visando à produção cartográfica nas suas mais diversas formas, ao estabelecimento de normas e instruções, ao suprimento e à manutenção do material a seu cargo e às atividades decorrentes de convênios com órgãos da Administração Pública (Diretoria de Serviço Geográfico do Exército Brasileiro, 2024b). Para o cumprimento dessa missão, a DSG mantém o Banco de Dados Geográficos do Exército (BDGEx), que constitui o nó do Exército Brasileiro na INDE (Diretoria de Serviço Geográfico do Exército Brasileiro, 2024a) e disponibiliza o acervo cartográfico oficial para consulta tanto pelo público militar quanto pela sociedade civil. A produção dos produtos cartográficos é realizada por cinco Centros de Geoinformação (CGEO) subordinados — localizados em Porto Alegre, Brasília, Olinda, Manaus e Rio de Janeiro — e segue as Especificações Técnicas para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG) (Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016a), em conformidade com o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (Brasil. Comissão Nacional de Cartografia (CONCAR), 2009), cuja adesão é obrigatória para as Organizações Militares produtoras de dados geoespaciais.

Esses produtos são indexados e disponibilizados por meio de catálogos de dados espaciais compatíveis com padrões internacionais estabelecidos pelo *Open Geospatial Consortium* (OGC), em particular o protocolo *Catalogue Service for the Web* (CSW) (Open Geospatial Consortium, 2007). Tal padronização garante interoperabilidade técnica, mas armazena os conteúdos de forma estruturada conforme um modelo de dados rígido; consequentemente, consultas formuladas sobre dados semiestruturados (texto livre, sinônimos, abreviações) requerem processamento adequado para serem traduzidas aos campos técnicos do catálogo. Esta característica torna a recuperação de dados um processo intrincado

para usuários sem formação na área cartográfica.

1.2 Problema

A busca em catálogos de dados espaciais institucionais exige que o usuário conheça previamente as convenções técnicas que organizam o acervo. No caso do acervo da DSG, isso implica saber utilizar o Índice de Nomenclatura (INOM), o Mapa Índice (MI), identificar corretamente as escalas disponíveis (1:1.000, 1:2.000, 1:5.000, 1:10.000, 1:25.000, 1:50.000, 1:100.000, 1:250.000), conhecer os tipos de produto do Sistema Cartográfico Nacional (SCN) (e.g., “SCN Carta Topográfica Matricial”, “MDT — RAM”, “SCN Carta Ortoimagem Banda P Pol HH”) e compreender as denominações dos Centros de Geoinformação responsáveis pela produção de cada folha.

Esta exigência constitui uma barreira de entrada considerável para usuários não especialistas — analistas, gestores e tomadores de decisão que precisam de acesso ágil à informação geoespacial, mas que não possuem formação técnica em cartografia. A interface atual, baseada em **busca por texto simples** (*full-text search*) contra os campos do banco, exige que o usuário conheça previamente os termos técnicos utilizados no acervo — e não usa qualquer forma de compreensão de linguagem natural. Um exemplo ilustrativo desta distinção é apresentado na Tabela 1.

Tabela 1 – Comparativo entre consulta técnica e consulta em linguagem natural

Modo atual (busca por texto simples)	Modo desejado (linguagem natural)
O usuário precisa digitar termos técnicos exatos que combinem com os campos do banco: “SCN Carta Topográfica Matricial 1:25.000 Rio de Janeiro 2023”	<i>“mapas do Rio de Janeiro em 25k publicados em 2023”</i>

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

A barreira técnica imposta por esse modelo de interação resulta em subutilização do acervo disponível e restringe a capacidade de usuários não especialistas de acessar informações geoespaciais relevantes para suas atividades. Esta é uma limitação que afeta não apenas o acervo da DSG, mas catálogos de dados espaciais em geral, conforme discutido na literatura recente sobre o tema (MAI et al., 2023).

1.3 Motivação

A Engenharia Cartográfica, enquanto disciplina, organiza-se em três eixos fundamentais: a **aquisição**, o **processamento** e a **disseminação** de dados geoespaciais. Embora boa parte do ensino e da pesquisa concentre-se nos dois primeiros eixos — com

foco em técnicas de levantamento, sensoriamento remoto e processamento digital —, a disseminação é a etapa que efetivamente conecta o acervo produzido aos seus usuários finais. É justamente nesse terceiro eixo que se insere o presente trabalho: facilitar o acesso e a disseminação dos produtos cartográficos do acervo da DSG, reduzindo a barreira técnica imposta pelas nomenclaturas especializadas e tornando o catálogo efetivamente utilizável por públicos não especialistas. Esta é a relação direta entre o trabalho desenvolvido e a Engenharia Cartográfica: traduzir, por meio de tecnologias de Processamento de Linguagem Natural, a riqueza do acervo cartográfico nacional em uma forma de acesso compatível com a linguagem cotidiana dos seus potenciais usuários.

Os Modelos de Linguagem de Grande Porte (LLMs, do inglês *Large Language Models*) representam um avanço substancial no campo do Processamento de Linguagem Natural (NLP). Baseados na arquitetura *Transformer* (VASWANI et al., 2017), esses modelos demonstram capacidade notável de compreender e gerar texto em linguagem natural, incluindo a habilidade de interpretar intenções implícitas, normalizar variações linguísticas e traduzir descrições informais para estruturas formais (BROWN et al., 2020; WEI et al., 2022).

Em particular, o mecanismo de *Tool Calling* — pelo qual um LLM pode decidir de forma autônoma quando e como invocar funções externas, estruturando os parâmetros necessários em formato JSON — abre perspectivas promissoras para a automação de tarefas de recuperação de informação (SCHICK et al., 2023; QIN et al., 2023). Este mecanismo é distinto da abordagem de Saída Estruturada (*Structured Output*), uma vez que o modelo não apenas formata uma resposta em esquema predefinido, mas decide ativamente quais ferramentas devem ser acionadas e com quais argumentos.

Um fator adicional de motivação é a viabilidade de execução local desses modelos. Plataformas como o Ollama (Ollama Team, 2023) permitem executar modelos de linguagem de código aberto — como os das famílias Qwen, Llama, Gemma e Mistral — diretamente em *hardware* local, sem dependência de serviços de nuvem. Isso é especialmente relevante para ambientes institucionais com restrições de privacidade, segurança da informação e custos operacionais, características típicas do contexto do Exército Brasileiro.

No âmbito da DSG, o 1º Centro de Geoinformação (1º CGEO) desenvolveu um protótipo inicial (1º Centro de Geoinformação, 2025), utilizando o modelo Phi-4 14B via Ollama com Saída Estruturada, confirmando a viabilidade da abordagem para o domínio cartográfico nacional, mas revelando limitações que motivaram o presente projeto.

1.4 Lacuna Identificada

Apesar da viabilidade demonstrada pelo protótipo do 1º CGEO, foram identificadas seis limitações que precisavam ser endereçadas para que a solução amadurecesse em direção ao uso operacional. Cada uma delas motivou uma frente de aprimoramento neste trabalho:

- **Backend em Node.js.** As bibliotecas de integração com LLMs, como o LangChain, têm versões mais completas em Python, o que limita a evolução da camada de tradução no *backend* original.
- **Ausência de *Tool Calling* real.** O protótipo utiliza *Structured Output*, no qual o modelo é forçado a devolver, a cada consulta, um objeto no formato do *schema* de saída, mesmo quando a consulta do usuário não contém informação para vários campos — o que pode induzir alucinações estruturadas (valores plausíveis, mas incorretos). O *Tool Calling*, ao contrário, delega ao modelo a decisão de quando invocar a ferramenta e quais argumentos preencher, o que pode ser mais adequado quando a consulta informa apenas parte dos campos.
- **Uso de um único modelo (Phi-4 14B).** Sem comparação sistemática, não há evidência que sustente a escolha do modelo em relação a alternativas mais recentes, em um ecossistema de LLMs abertos que evolui rapidamente (e.g., Qwen 3 e Gemma 4).
- **Pré-processamento manual com dicionário *hardcoded*.** O protótipo depende de um dicionário fixo de normalizações (“25k” → “1:25.000”, “primeiro cgeo” → “1º Centro de Geoinformação”). O dicionário só cobre variações previstas em tempo de desenvolvimento e exige manutenção manual sempre que aparece uma forma nova; a delegação da normalização ao LLM elimina essa dependência.
- **Fallback por expressões regulares.** Quando o modelo falha, o protótipo aciona um mecanismo que tenta extrair parâmetros com regras rígidas. O *fallback* esconde as causas reais das falhas do modelo e não é passível de análise sistemática.
- **Ausência de métricas formais de avaliação.** O protótipo dispõe apenas de 22 casos de teste, pontuados por pesos atribuídos *ad hoc* aos campos esperados, sem penalizar campos preenchidos indevidamente. Sem um conjunto de referência (*ground truth*) mais amplo, métricas padronizadas e protocolo de medição, não há como aferir objetivamente a qualidade da solução, nem sua evolução ao longo do tempo, à medida que novos modelos surjam.

O presente trabalho aprimorou o protótipo existente, endereçando diretamente cada uma dessas seis limitações.

1.5 Objetivos

1.5.1 Objetivo Geral

Aprimorar a **camada de tradução linguística** da solução de busca em linguagem natural sobre o acervo cartográfico da DSG — ou seja, a etapa em que uma consulta livre do usuário é convertida em parâmetros estruturados de busca — partindo do protótipo desenvolvido pelo 1º CGEO, por meio da reimplementação do *pipeline* com *Tool Calling* nativo, da avaliação comparativa de modelos de LLM abertos executáveis localmente via Ollama e da definição de métricas quantitativas que permitam aferir e comparar objetivamente o desempenho da tradução. A lógica de consulta ao banco e de resolução geográfica foi portada do protótipo e não é objeto de avaliação neste trabalho.

1.5.2 Entregáveis do Projeto

O projeto resultou nos seguintes entregáveis:

1. Uma nova arquitetura de *backend* em Python (FastAPI + LangChain), que substitui o *backend* Node.js do protótipo e implementa *Tool Calling* nativo via Ollama;
2. Um *dataset* de avaliação com 310 consultas de referência (*ground truth*) representativas do domínio cartográfico da DSG, acompanhado de manual de anotação e de registro de auditoria do gabarito (Capítulo 3 e Apêndice A);
3. Um *pipeline* de avaliação automatizado e reproduzível, com cálculo das métricas de acurácia, precisão, *recall*, F1-score e latência;
4. Um relatório comparativo entre quatro modelos de LLM com suporte a *Tool Calling* executáveis localmente via Ollama — Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B —, indicando o mais adequado para uso operacional no acervo da DSG, considerando o equilíbrio entre desempenho e requisitos de *hardware*.

Como referência, o mesmo protocolo foi aplicado a dois modelos de maior porte servidos em nuvem pela plataforma Groq, cujos resultados são reportados à parte por não atenderem ao requisito de execução local.

1.6 Delimitação do Trabalho

O presente Projeto Final de Curso (PFC) tem como foco o aprimoramento do *pipeline* de tradução de consultas em linguagem natural para parâmetros técnicos de busca, compreendendo o componente de LLM e sua integração com o banco de dados geoespacial. Explicitamente fora do escopo deste trabalho estão: o desenvolvimento de interface

gráfica (*frontend*) para o usuário final; a implantação em produção no sistema da DSG; a realização de pesquisa de opinião ou avaliação qualitativa com usuários; o treinamento ou ajuste fino (*fine-tuning*) de modelos de linguagem; a busca por coordenadas geográficas sem referência nominal; e o suporte a idiomas além do português. A delimitação completa do escopo é detalhada no Capítulo 3.

1.7 Estrutura do Trabalho

O presente trabalho está organizado em seis capítulos, dos quais este é o primeiro. O Capítulo 2 apresenta o referencial teórico, cobrindo os fundamentos de catálogos de dados espaciais, o mecanismo de *Tool Calling* e sua relação com a Saída Estruturada, os Modelos de Linguagem de Grande Porte e sua execução local com o Ollama, a orquestração com LangChain e as métricas de avaliação empregadas. O Capítulo 3 detalha a metodologia adotada: o escopo do trabalho (entregáveis e exclusões), os requisitos que o norteiam, a seleção dos modelos comparados, a construção e a auditoria do *dataset* de avaliação e o protocolo de avaliação, incluindo os ambientes de execução e o cálculo das métricas. O Capítulo 4 descreve a execução do aprimoramento: a análise do protótipo de referência do 1º CGEO, a elicitação de requisitos, o planejamento da migração e o desenvolvimento da solução por camadas (API, orquestração, ferramenta de busca e persistência), além do *pipeline* de avaliação. O Capítulo 5 apresenta os resultados dos experimentos com os quatro modelos locais, a validação na estação de trabalho, o resultado de referência em nuvem e a discussão comparativa que fundamenta a recomendação de modelo. O Capítulo 6 sintetiza as conclusões do trabalho, apresenta suas limitações e propõe direções para trabalhos futuros. Por fim, o Apêndice A reúne o *dataset* de avaliação, o manual de anotação e o registro da auditoria do gabarito; o Apêndice B especifica os ambientes de execução; e o Apêndice C descreve o repositório do código-fonte e os passos para a reprodução dos experimentos.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o trabalho. A organização segue a ordem em que os conceitos são utilizados na metodologia: primeiro, os *catálogos de dados espaciais* (acervo a ser consultado); em seguida, o mecanismo de *Tool Calling* (estratégia central do projeto) e sua distinção em relação à *Saída Estruturada* adotada pelo protótipo de referência; depois, os *Modelos de Linguagem de Grande Porte* executados localmente por meio do Ollama; e, por fim, as *métricas de avaliação* empregadas para comparar os modelos.

2.1 Catálogos de Dados Espaciais

A produção cartográfica brasileira é regida pelo Sistema Cartográfico Nacional (SCN) (Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016b), que define os tipos de produtos, as escalas de trabalho, os critérios de qualidade e as convenções de nomenclatura — incluindo o Índice de Nomenclatura (INOM) e o Mapa Índice (MI) — a serem seguidos pelos órgãos produtores de cartografia oficial. As Especificações Técnicas para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG) (Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016a), mantidas pela DSG, complementam o SCN ao detalhar os requisitos técnicos dos produtos.

A Infraestrutura Nacional de Dados Espaciais (INDE), instituída pelo Decreto n.º 6.666/2008 (Brasil, 2008), organiza, em âmbito federal, o acesso integrado a conjuntos de dados geoespaciais produzidos e mantidos pelos órgãos governamentais. A INDE é constituída por *nós* provedores — entendidos como pontos de acesso institucionais que catalogam e disponibilizam dados conforme o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (Brasil. Comissão Nacional de Cartografia (CONCAR), 2009). O Banco de Dados Geográficos do Exército (BDGEx) é o nó do Exército Brasileiro na INDE (Diretoria de Serviço Geográfico do Exército Brasileiro, 2024a) e, por meio dele, o acervo da DSG é disponibilizado para consulta pela sociedade. A obrigatoriedade de adesão das Organizações Militares produtoras de dados geoespaciais ao Perfil MGB e ao SCN é parte das atribuições legais decorrentes do referido decreto.

Do ponto de vista técnico, a INDE e o BDGEx adotam os padrões do OGC para a disponibilização dos dados, incluindo os serviços *Web Map Service* (WMS), *Web Feature Service* (WFS), *Web Coverage Service* (WCS) e *Catalogue Service for the Web* (CSW). O protocolo CSW (Open Geospatial Consortium, 2007) é o padrão internacional para publicação e descoberta de metadados de recursos geoespaciais e define operações de consulta (como *GetRecords*) que permitem a busca por título, data, tipo, área de abrangência ge-

ográfica e palavras-chave. A interface CSW, embora tecnicamente robusta, armazena os conteúdos de forma estruturada conforme um modelo de dados rígido; consequentemente, consultas sobre dados semiestruturados (texto livre, sinônimos, abreviações) requerem processamento adequado para serem traduzidas aos campos técnicos do catálogo.

2.2 Tool Calling

O mecanismo de *Tool Calling* (também denominado *Function Calling*) é a capacidade de um Modelo de Linguagem de Grande Porte (LLM) de, ao processar uma consulta do usuário, identificar que a resposta requer a execução de uma função externa e gerar uma chamada estruturada para essa função, incluindo todos os parâmetros necessários em formato JSON (SCHICK et al., 2023; QIN et al., 2023). Trata-se, portanto, de uma capacidade que estende o modelo para além da geração textual pura, permitindo que ele atue como o componente decisório de um sistema que orquestra ferramentas determinísticas (consultas a bancos de dados, chamadas a APIs, execuções de código).

O fluxo completo de uma interação com *Tool Calling* pode ser descrito da seguinte forma: o **usuário** formula uma consulta em linguagem natural; o **LLM** analisa a consulta e, com base nas definições de ferramentas (*tools*) fornecidas no *prompt* do sistema, decide que uma ou mais ferramentas devem ser invocadas; o LLM gera uma **chamada de ferramenta** (*tool call*) em formato JSON, especificando o nome da função e seus argumentos; o **sistema de orquestração** (e.g., LangChain) intercepta a chamada, executa a função correspondente com os argumentos fornecidos e retorna o resultado ao LLM; por fim, o LLM incorpora o resultado da execução em seu contexto e formula a resposta final ao usuário. A Figura 1 ilustra este fluxo de forma esquemática.

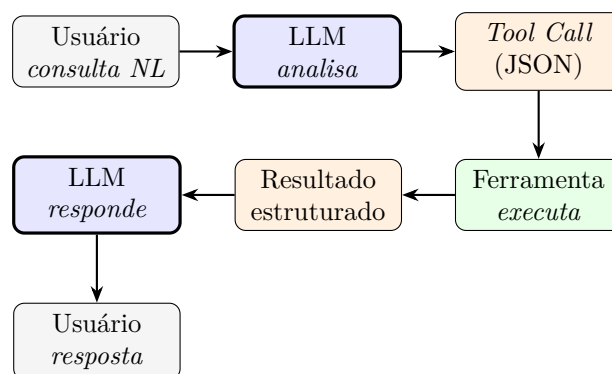


Figura 1 – Fluxo do mecanismo de *Tool Calling*

Fonte: Elaborado pelo autor.

No protocolo de *Tool Calling* adotado pelo Ollama e pelo LangChain, as ferramentas são definidas por um *schema* JSON que especifica o nome da função, sua descrição e os parâmetros aceitos, com seus tipos e descrições. O LLM recebe essas definições como

parte do contexto do sistema e as utiliza para gerar chamadas estruturalmente válidas. Um exemplo de definição de ferramenta para a busca em um catálogo de dados espaciais é apresentado na Figura 2.

```
{
  "name": "buscar_catalogo",
  "description": "Busca produtos cartograficos no catalogo",
  "parameters": {
    "type": "object",
    "properties": {
      "keyword": {"type": "string",
        "description": "Codigo MI, INOM ou nome da carta"},
      "scale": {"type": "string",
        "enum": ["1:25.000", "1:50.000", "1:100.000", ...],
        "description": "Escala cartografica"},
      "state": {"type": "string",
        "description": "Nome do estado (normalizado)"},
      "productType": {"type": "string",
        "description": "Tipo de produto SCN"}
    }
  }
}
```

Figura 2 – Exemplo simplificado de definição de ferramenta para busca em catálogo de produtos cartográficos

Fonte: Elaborado pelo autor.

2.2.1 Tool Calling *versus* Saída Estruturada

É importante distinguir *Tool Calling* da abordagem de Saída Estruturada (*Structured Output*), conceito frequentemente confundido com o primeiro. Na Saída Estruturada, o modelo é instruído a retornar sua resposta em um formato JSON validado por um *schema* predefinido (e.g., Pydantic em Python (COLVIN, 2019) ou Zod em TypeScript): o modelo *sempre* retorna uma saída estruturada, independentemente da consulta, e a decisão se restringe a quais campos preencher. Bibliotecas como o Instructor (LIU, 2023) interceptam a saída do LLM e a forçam a corresponder ao *schema*, repetindo a chamada em caso de saída inválida.

A abordagem de *Structured Output* tem uma limitação fundamental: o modelo é sempre compelido a preencher a estrutura predefinida, mesmo quando a consulta do usuário não contém informação suficiente para todos os campos, o que pode levar a alucinações estruturadas — valores plausíveis mas incorretos inseridos para satisfazer o *schema*. Além disso, o modelo não pode decidir invocar múltiplas ferramentas em sequência ou adaptar seu comportamento com base no resultado de uma operação anterior.

No *Tool Calling*, em contraste, o modelo *decide* de forma autônoma se e quando invocar uma ferramenta, selecionando entre múltiplas ferramentas disponíveis. O modelo pode responder diretamente ao usuário sem invocar ferramentas, ou pode encadear múltiplas chamadas em sequência para resolver uma tarefa complexa. Para o caso de uso deste trabalho, o *Tool Calling* *pode ser mais adequado* do que a Saída Estruturada por permitir que o modelo trate consultas ambíguas de forma mais flexível, decida quais parâmetros de busca são realmente relevantes e potencialmente encadeie operações de validação e refinamento da busca — a superioridade prática, contudo, só se estabelece empiricamente e é justamente o que a avaliação sistemática do Capítulo 5 afere.

Modelos com *thinking*.

Uma parte dos modelos abertos recentes — notadamente Gemma 4 e Qwen 3 — incorpora, opcionalmente, uma etapa de raciocínio interno (*thinking*) antes de emitir a resposta ou a chamada de ferramenta. Nesse modo, o modelo produz internamente uma sequência de tokens de raciocínio que não aparecem na saída, o que aumenta a latência total mas tende a melhorar a qualidade das decisões de *Tool Calling* em consultas com múltiplos critérios. Neste trabalho, o modo *thinking* foi mantido **desativado** em todos os modelos locais avaliados — por parâmetro da API nos modelos que declaram essa capacidade e, no Qwen 3, pela adoção da *build* oficial sem raciocínio —, garantindo a comparabilidade dos tempos de resposta e das decisões de *Tool Calling* entre modelos que oferecem e que não oferecem essa etapa (no resultado paralelo em nuvem, adotou-se o menor esforço de raciocínio aceito por cada modelo); a configuração adotada para cada modelo é detalhada no Capítulo 3. A avaliação com o modo *thinking* ativado, que permitiria medir o ganho de qualidade obtido em troca da latência adicional, não foi realizada e fica como trabalho futuro.

2.3 Modelos de Linguagem de Grande Porte e Execução Local

Os Modelos de Linguagem de Grande Porte (LLMs) baseiam-se, em sua quase totalidade, na arquitetura *Transformer* (VASWANI et al., 2017), que substituiu as arquiteturas recorrentes (RNN, LSTM) por um mecanismo puramente baseado em atenção (*self-attention*). Essa mudança permitiu o processamento paralelo de sequências longas e a captura de dependências distantes no texto, viabilizando o treinamento em corpora de grande escala. Para o presente trabalho, a propriedade mais relevante dos LLMs modernos é a capacidade emergente de *instruction following* (WEI et al., 2022; BROWN et al., 2020) — a habilidade de seguir instruções complexas em linguagem natural — que é o fundamento sobre o qual o *Tool Calling* se assenta: o modelo precisa entender a consulta, interpretar o *schema* das ferramentas disponíveis e decidir como populá-las.

2.3.1 Vantagens da Execução Local

A execução local de LLMs, em contraste com o uso de APIs de serviços na nuvem, apresenta vantagens relevantes para contextos institucionais como o da DSG (Ollama Team, 2023; llama.cpp contributors, 2023): **privacidade e segurança**, pois as consultas e os dados não trafegam por redes externas — fator crítico para organizações com restrições de sigilo; **ausência de custos recorrentes por *token***, eliminando dependência de orçamento variável; **controle e auditabilidade**, com a organização mantendo a versão exata do modelo utilizado; e **disponibilidade offline**, com o sistema funcionando independentemente da conectividade.

Essa abordagem também traz **desvantagens** que devem ser reconhecidas: (i) o **acesso limitado aos modelos de fronteira** — os LLMs proprietários mais capazes (GPT-4, Claude Sonnet, Gemini) não são liberados como código aberto e continuam superando os modelos abertos em *benchmarks* de *Tool Calling*; (ii) o **custo de infraestrutura** desloca-se do modelo de assinatura para o de *hardware*: a organização precisa adquirir e manter servidores com GPU e VRAM suficientes, além do custo operacional (energia, refrigeração, atualização de *drivers*); (iii) o **ciclo de atualização** dos modelos passa a ser responsabilidade interna — novas versões exigem repetir a avaliação sistemática.

Uma característica mitigadora é que o LangChain, adotado neste trabalho, permite **trocar o modelo local por um de terceiros** (ou vice-versa) com alteração mínima no código de aplicação — basta substituir o *wrapper* `ChatOllama` por, por exemplo, `ChatOpenAI` ou `ChatAnthropic`. Essa portabilidade permite que a decisão local × nuvem seja **gerencial**, e não técnica, no futuro. Neste trabalho, essa mesma troca (`ChatOllama` → `ChatOpenAI`, apontado para o serviço Groq) foi empregada para obter um resultado paralelo, em nuvem, com modelos de maior porte, reportado à parte como referência no Capítulo 5.

2.3.2 Ollama e Modelos com Suporte a Tool Calling

O Ollama (Ollama Team, 2023) é uma plataforma de código aberto que simplifica a instalação, a execução e o gerenciamento de modelos de linguagem localmente, expondo uma API compatível com o padrão OpenAI — o que facilita a integração com *frameworks* como o LangChain. Internamente, o Ollama utiliza o *backend* `llama.cpp` para inferência, com suporte aos formatos de quantização GGUF (*GPT-Generated Unified Format*) (llama.cpp contributors, 2023), que permitem reduzir a precisão numérica dos pesos do modelo (tipicamente de 16 bits para 4 bits por peso) com impacto pequeno na qualidade das respostas (FRANTAR et al., 2022). No presente trabalho, foram adotadas quantizações de 4 bits — `Q4_K_M`, considerada um bom equilíbrio entre compressão e qualidade, ou a quantização equivalente da *build* publicada de cada modelo (as *tags* exatas constam

da Seção 3.2) —, o que reduz o consumo de VRAM em cerca de quatro vezes em relação ao formato FP16. Como regra prática, modelos quantizados em Q4_K_M demandam aproximadamente 0,5 GB de VRAM por bilhão de parâmetros, viabilizando a execução de modelos de até cerca de 14B em GPUs de uso doméstico.

A Tabela 2 apresenta os principais modelos disponíveis no Ollama com suporte a *Tool Calling*, entre os quais foram selecionados os modelos comparados neste trabalho (Seção 3.2). A coluna “VRAM aprox. (Q4_K_M)” serviu de referência para selecionar modelos compatíveis com a estação de testes especificada na Seção 3.5.2 do Capítulo 3; modelos cuja VRAM exigida ultrapassa a disponível ainda podem ser executados com *offload* parcial para CPU, ao custo de maior latência.

Tabela 2 – Modelos com suporte a *Tool Calling* disponíveis no Ollama (seleção)

Modelo	Parâmetros	VRAM aprox. (Q4_K_M)	Organização
Gemma 4 E2B	2B efet.	~4 GB ^a	Google DeepMind
Gemma 4 E4B	4B efet.	~5,7 GB ^a	Google DeepMind
Qwen3 (4B)	4B	~2,5 GB	Alibaba Cloud
Llama 3.2 (3B)	3B	~2 GB	Meta AI
Qwen3 (8B)	8B	~5 GB	Alibaba Cloud
Mistral Nemo (12B)	12B	~7–8 GB	Mistral AI
Gemma 4 (26B MoE)	26B (4B ativ.)	~15–17 GB	Google DeepMind

Fonte: Compilado pelo autor a partir de Ollama Team (2023) e da documentação oficial dos modelos.

Nota: ^a Tamanho da *build* com quantização consciente de treinamento (QAT, 4 bits) publicada no Ollama (`gemma4:e2b-it-qat` e `gemma4:e4b-it-qat`). “Efet.” indica parâmetros efetivos; o total armazenado é maior (4,6B no E2B, segundo o Ollama), o que explica, em parte, a VRAM acima da regra prática.

2.4 Orquestração com LangChain

O LangChain (CHASE, 2022) é um *framework* de código aberto para o desenvolvimento de aplicações baseadas em LLMs, fornecendo abstrações para os componentes comuns de um *pipeline*: *wrappers* de modelos de linguagem, *prompts*, *chains* (sequências de operações), agentes, ferramentas e memória. Para o presente trabalho, o LangChain é utilizado para (a) gerenciar a integração com o Ollama via `ChatOllama`; (b) ligar ao modelo, pelo método `bind_tools()`, a ferramenta de busca descrita por seu *schema* JSON; (c) obter, em uma única chamada `invoke()`, as chamadas de ferramenta geradas pelo modelo e, no ciclo completo, devolver-lhe o resultado da busca para a resposta final; e (d) servir de interface única com os modelos no *pipeline* de avaliação.

O LangGraph (LangChain AI, 2024), extensão do LangChain que permite modelar fluxos de execução como grafos de estados, com ciclos e ramificações condicionais, é indicado quando o fluxo exige iteração explícita sobre resultados intermediários. Não foi

o caso neste trabalho: a tradução da consulta se resolve em uma única decisão do modelo (chamar ou não a ferramenta, e com quais argumentos), e o ciclo completo é implementado explicitamente no módulo de *pipeline*, sem necessidade de um grafo de estados, conforme detalhado na Seção 4.4 do Capítulo 4.

2.5 Métricas de Avaliação

Definir bem as métricas de avaliação importa por dois motivos práticos. Primeiro, elas permitem ao usuário final (a DSG e as OM subordinadas) tomar uma **decisão informada de qual modelo implantar** — há um *trade-off* entre qualidade de extração e latência, e diferentes contextos operacionais podem priorizar aspectos distintos. Segundo, elas viabilizam a **reavaliação periódica** da solução à medida que novos modelos surjam — o *pipeline* de avaliação (entregável do projeto, descrito na Seção 4.4 do Capítulo 4) foi projetado para ser rerodado com custo marginal baixo sobre qualquer LLM compatível com *Tool Calling*.

As métricas adotadas, todas oriundas da literatura de recuperação de informação (MANNING; RAGHAVAN; SCHÜTZE, 2008), devem ser lidas como **pontos de vista complementares sobre o mesmo resultado**: cada uma revela um aspecto (correção total, correção por campo, tempo de resposta) e as decisões operacionais nascem da análise conjunta, não do ranking de uma métrica isolada. Modelos diferentes podem vencer em métricas diferentes.

Nota sobre o critério de comparação.

A comparação entre a resposta do modelo e o *ground truth* é feita **campo a campo** sobre o objeto JSON estruturado, e não byte a byte: enums (`scale`, `productType`, `supplyArea`, `project`) e campos de ordenação (`sortField`, `sortDirection`) exigem correspondência exata; *strings* livres (`keyword`, `state`, `city`) são normalizadas antes da comparação (*lowercase*, remoção de acentos e pontuação); intervalos de data são expressos pelos limites `start` e `end` em ISO 8601 e exigem coincidência exata desses limites; e `limit` é comparado como inteiro. Quando a consulta admite mais de uma leitura legítima (por exemplo, “últimos 3 meses” como noventa dias ou como três meses de calendário), o gabarito lista explicitamente as leituras aceitas, e a resposta é considerada correta se coincidir com alguma delas; os critérios de anotação constam do Capítulo 3, Seção 3.3. Detalhes do protocolo constam do Capítulo 3, Seção 3.5.

A **acurácia** mede a proporção de consultas para as quais o modelo extraiu corretamente *todos* os parâmetros esperados, sem acrescentar parâmetros ausentes do gabarito, em relação ao total de consultas avaliadas; nas consultas cujo gabarito prevê que nenhuma

ferramenta seja chamada, a resposta é correta quando o modelo não a chama:

$$\text{Acurácia} = \frac{\text{Número de consultas completamente corretas}}{\text{Total de consultas}}.$$

Esta é a métrica mais estrita, pois qualquer erro em qualquer parâmetro conta como uma consulta incorreta.

Para a análise por parâmetro (campo a campo) do *schema* de busca, utilizam-se as métricas clássicas de **precisão**, **recall** e **F1-score**:

$$\text{Precisão} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}},$$

onde *TP* (*true positives*) são os parâmetros corretamente extraídos, *FP* (*false positives*) são parâmetros extraídos incorretamente, e *FN* (*false negatives*) são parâmetros esperados que não foram extraídos. O *F1-score* é a média harmônica entre precisão e *recall* e é especialmente útil quando ambas as métricas são igualmente importantes.

A **latência** é o tempo de resposta do modelo para uma consulta. Na avaliação, mede-se o tempo da tradução — do envio da consulta ao modelo até o recebimento da resposta com as chamadas de ferramenta —, que é a etapa comparada entre os modelos. É uma métrica operacional importante para a viabilidade de uso em produção e é reportada por mínimo, mediana, média, percentil 95 e máximo sobre o conjunto de consultas avaliadas. Para diagnóstico, o tempo de execução da ferramenta (a consulta ao banco) é registrado à parte, quando a busca é executada, e não entra na comparação entre modelos.

3 METODOLOGIA

Este capítulo descreve o método de trabalho adotado para o aprimoramento da solução de busca em linguagem natural sobre o acervo cartográfico da DSG. O trabalho parte de um protótipo de referência desenvolvido pelo 1º CGEO, descrito e analisado em detalhe no Capítulo 4, e adota a estratégia de **substituição dirigida**: identificam-se os componentes responsáveis pelas limitações documentadas pelo cliente e substituem-se esses componentes por equivalentes mais robustos, preservando o que já funciona (esquema do banco, conjunto de parâmetros de busca, lógica das consultas espaciais).

O capítulo apresenta os elementos transversais ao projeto: o escopo (entregáveis e exclusões), os modelos comparados, a construção e a auditoria do *dataset* de avaliação e o protocolo de medição. A execução do aprimoramento — análise do protótipo, requisitos, planejamento da migração, desenvolvimento por camadas e cronograma — é apresentada no Capítulo 4.

Foco da avaliação: a camada de tradução linguística.

O objeto de avaliação deste trabalho é o desempenho do LLM na **tradução de uma consulta em linguagem natural para os parâmetros estruturados** de busca no catálogo. A execução dessa consulta contra o banco (PostgreSQL/PostGIS), a resolução de nomes geográficos e a interpretação das *bounding boxes* das cartas são componentes herdados do protótipo do 1º CGEO; sua correção é assumida como *caixa-preta* e não é medida. Métricas sobre o *conjunto de produtos retornados* ficam fora do escopo (Seção 3.1.2) e são apontadas como direção de trabalho futuro no Capítulo 6. O LLM é comparado contra um *ground truth* de **parâmetros extraídos**, não de produtos recuperados.

3.1 Escopo do Projeto

3.1.1 Entregáveis

O projeto entregou:

- uma nova arquitetura de *backend* em Python (FastAPI + LangChain), que substituiu o *backend* Node.js do protótipo e implementa *Tool Calling* nativo via Ollama;
- um *dataset* de avaliação com 310 consultas de referência (*ground truth*), distribuídas em seis categorias de consulta e uma de consultas fora do domínio, acompanhado de um manual de anotação e de um relatório de auditoria;

- um *pipeline* de avaliação automatizado, que executa o *dataset* sobre cada modelo e produz as métricas de acurácia, precisão, *recall*, F1-score e latência, com os testes estatísticos correspondentes;
- um relatório comparativo entre quatro modelos de LLM com suporte a *Tool Calling*, executados localmente, com a recomendação fundamentada do modelo mais adequado para uso operacional sobre o acervo da DSG, complementado por uma referência paralela com modelos maiores servidos em nuvem.

3.1.2 Exclusões

Estão fora do escopo deste trabalho:

- **Avaliação da qualidade de recuperação em nível de produto:** uma vez extraídos os parâmetros, o conjunto de produtos retornado depende do motor de busca (*buscar_catalogo*) e da qualidade do banco cartográfico, ambos herdados do protótipo. O trabalho **não mede** precisão e *recall* em nível de produto. A execução SQL permanece no *pipeline* apenas como demonstração de que a arquitetura fecha ponta a ponta;
- **Resolução avançada de nomes geográficos:** o cruzamento entre *strings* livres (estado, município) e as *bounding boxes* das cartas depende de recursos externos (malha do IBGE, Nominatim/OSM) e da qualidade das geometrias no banco; é tratado como trabalho futuro (Capítulo 6);
- desenvolvimento de interface gráfica (*frontend*) para o usuário final;
- implantação em produção em ambiente da DSG ou de suas OM subordinadas;
- pesquisa de opinião ou avaliação qualitativa com usuários;
- treinamento ou ajuste fino (*fine-tuning*) de modelos de linguagem;
- busca por coordenadas geográficas brutas (*bounding box*) sem referência nominal a um lugar;
- suporte a idiomas além do português brasileiro.

3.1.3 Requisitos que norteiam o escopo

O escopo é norteado por requisitos funcionais (RF) e não funcionais (RNF) elicitados a partir da análise do protótipo e das conversas com o cliente. O Quadro 1 traz o resumo; a discussão completa está no Capítulo 4 (Seção 4.2).

Quadro 1 – Resumo dos requisitos que norteiam o escopo

Requisitos Funcionais (RF)		Requisitos Não Funcionais (RNF)	
RF1	Receber consulta em linguagem natural via REST	RNF1	Execução totalmente local (privacidade)
RF2	Traduzir consulta em parâmetros do <i>schema</i>	RNF2	Latência mediana operacional aceitável
RF3	Executar busca no Post-greSQL/PostGIS	RNF3	Conformidade a ET-PCDG e Perfil MGB
RF4	Selecionar modelo por requisição (<i>dev only</i>)	RNF4	Reprodutibilidade dos experimentos
RF5	Registrar traços para as métricas	RNF5	Manutenibilidade (módulos, testes)
		RNF6	Substituibilidade de modelo por configuração

Fonte: Elaborado pelo autor. Detalhes no Cap. 4, Seção 4.2.

Requisitos ideais são mensuráveis, mas alguns dos RNF — em particular o RNF2, latência — dependem de fatores externos ao escopo deste PFC (*hardware* do ambiente de implantação, carga concorrente). Por isso os valores de latência são reportados nas condições de cada ambiente de execução (Seção 3.5.2), sem limiar operacional fixado *a priori*.

3.2 Seleção de Modelos

3.2.1 Critérios de Seleção

Os modelos foram selecionados segundo os critérios: (i) disponibilidade no Ollama com suporte nativo a *Tool Calling*, verificada no catálogo oficial da biblioteca (Ollama Team, 2023), que mantém rankings de popularidade e uma etiqueta explícita de compatibilidade com *tool calling*; (ii) disponibilidade em versões quantizadas compatíveis com estações de trabalho modestas; (iii) lançamento ou atualização entre 2024 e 2026; e (iv) diversidade de arquiteturas e organizações desenvolvedoras.

3.2.2 Modelos Avaliados

Foram avaliados quatro modelos abertos, executados localmente pelo Ollama:

1. **Qwen 3 4B** (Qwen Team, 2025) (Alibaba Cloud), *tag* `qwen3:4b-instruct-2507-q4_K_M`: 4 bilhões de parâmetros, quantização Q4_K_M, 2,3 GB — o único dos quatro que cabe integralmente nos 4 GB de VRAM da estação de referência. Usou-se a *build* oficial sem raciocínio interno (*Instruct-2507*), pela razão explicada na Seção 3.5.3;

2. **Gemma 4 E4B** (Gemma Team, 2026) (Google DeepMind), *tag* `gemma4:e4b-it-qat`: variante *edge* de 4 bilhões de parâmetros efetivos, com quantização consciente de treinamento. É a menor *build* publicada do E4B ($\sim 5,7$ GB) e não cabe em 4 GB de VRAM — a variante `Q4_K_M` ocupa ~ 9 GB —, o que exige *offload* parcial para a CPU na estação de referência;
3. **Gemma 4 E2B** (Gemma Team, 2026), *tag* `gemma4:e2b-it-qat`: variante menor da mesma família ($\sim 4,0$ GB), incluída justamente porque o E4B não cabe na VRAM da estação de referência. Permite observar, dentro de uma mesma família, o efeito do tamanho do modelo;
4. **Mistral Nemo 12B** (JIANG et al., 2023) (Mistral AI e NVIDIA), *tag* `mistral-nemo:12b`: 12 bilhões de parâmetros ($\sim 6,6$ GB), com suporte consolidado a *Tool Calling* e bom desempenho em português. Representa a faixa superior de tamanho executável com *offload* parcial numa estação modesta.

3.2.3 Referência paralela em nuvem

Como referência — e não como candidatos à implantação, uma vez que o requisito RNF1 exige execução local —, a mesma avaliação foi executada sobre dois modelos maiores servidos pelo provedor Groq e disponíveis com suporte a *Tool Calling* em setembro de 2026: `openai/gpt-oss-20b` e `openai/gpt-oss-120b`. O objetivo é situar os modelos locais em relação a modelos de maior porte sob o mesmo *prompt*, a mesma ferramenta, o mesmo *dataset* e as mesmas métricas. A troca de provedor usa exatamente a portabilidade descrita no Capítulo 2: o `ChatOllama` do LangChain é substituído pelo `ChatOpenAI` apontado para o *endpoint* compatível do provedor, sem alteração no restante do *pipeline*. Esses resultados são sempre reportados à parte e identificados como execução em nuvem.

3.3 Construção do *Dataset* de Avaliação

A comparação objetiva dos modelos exige um *dataset* de referência representativo do espectro de consultas que usuários do catálogo tenderiam a formular, cobrindo diferentes níveis de complexidade e combinações de parâmetros, e cujo gabarito seja **auditável**: cada resposta esperada deve poder ser justificada a partir do texto da consulta e de regras escritas, e qualquer leitor deve poder refazer essa verificação.

3.3.1 Estrutura de cada entrada

Cada consulta do *dataset* é registrada com os campos da Tabela 3. O arquivo `dataset.json` e o Apêndice A são gerados a partir da mesma fonte, de modo que não podem divergir.

Tabela 3 – Estrutura de cada entrada do *dataset* de avaliação

Campo	Descrição
id	Identificador (P01–P22, N01–N40, GS001 ... GF007)
fonte	Rastreabilidade: arquivo e linha de origem (camada P), autores (camada N) ou função geradora, modelo de frase e semente (camada G)
categorias	Uma ou mais das categorias S, C, M, T, O, A; ou F (fora do domínio), que não se combina com as demais
consulta	Texto em linguagem natural
esperado	Leitura preferencial do gabarito: parâmetros de busca na forma canônica, com períodos relativos expressos por regras nomeadas
alternativas	Demais leituras aceitas, quando a consulta admite mais de uma (Seção 3.3.4)
espera_tool_call	Falso quando o comportamento correto é não chamar a ferramenta
observacional	Verdadeiro quando o caso fica fora das métricas principais
notas	Justificativa das leituras alternativas e observações do anotador

Fonte: Elaborada pelo autor.

3.3.2 Categorias de Consultas

O *dataset* é organizado em seis categorias, alinhadas aos exemplos do protótipo e às demandas do cliente: **simples** (S, um único parâmetro: “cartas de São Paulo”); **compostas** (C, dois ou mais parâmetros: “ortoimagens de Manaus em 1:50.000”); **com código MI/INOM** (M: “INOM SG-22-X-A”); **com referência temporal relativa** (T: “nos últimos 3 meses”, “depois de 2022”); **com ordenação** (O: “as 3 cartas mais antigas”); e **ambíguas ou informais** (A: siglas, grafias sem acento, abreviações e expressões qualitativas). Uma consulta pode pertencer a mais de uma categoria. Uma sétima categoria, **fora do domínio** (F: “cartas de Marte”, “qual a previsão do tempo para amanhã em Manaus?”), reúne as consultas em que a resposta correta é não chamar a ferramenta; ela mede a capacidade de o modelo recusar pedidos que o catálogo não atende.

3.3.3 Camadas de Construção

O *dataset* tem 310 consultas, construídas em três camadas complementares (Tabela 4):

1. **Camada P (protótipo, 22 casos)**: as consultas do arquivo de testes do protótipo (`backend/evaluation/test-cases.ts`), cada uma rastreada até a linha de origem. A anotação original da equipe do 1º CGEO é mantida como leitura preferencial — o *baseline* não é reanotado —, e as leituras alternativas acrescentadas seguem o manual de anotação, com justificativa registrada. Os gabaritos de datas relativas do

arquivo original, fixados em datas de 2025, são substituídos pelas regras nomeadas da Seção 3.3.4, que se resolvem na data da execução.

2. **Camada N (autores, 40 casos)**: redigidos para preencher lacunas de distribuição — em especial a categoria Simples, ausente na camada P — e para exercitar ambiguidades e casos de fronteira do domínio.
3. **Camada G (geração programática, 248 casos)**: consultas sintetizadas pela combinação de catálogos fechados do domínio — 27 UFs, 12 municípios de referência, 8 escalas do SCN, 9 tipos de produto das ET-PCDG, 5 CGEOs, 9 projetos, códigos MI e INOM, 15 expressões de tempo relativo e padrões de ordenação — em modelos de frase do português corrente (com preposição adequada a cada UF e apelidos usuais de escala, como “25k” ou “escala de 25 mil”). O gabarito é conhecido **por construção**: a frase é montada a partir dos parâmetros que deve produzir. A geração é determinística (semente 42) e rejeita qualquer consulta que repita, após normalização, outra do *dataset*, inclusive das camadas P e N. A camada inclui sete consultas de fronteira do domínio (família G-F): seis fora do domínio e uma observacional.

Tabela 4 – Composição do *dataset* de avaliação por origem e família

Origem / família	Consultas	Observacionais	Com mais de uma leitura aceita
Protótipo do 1º CGEO (P)	22	0	11
Autores (N)	40	3	15
Geradas — simples (G-S)	61	0	0
Geradas — compostas (G-C)	55	0	3
Geradas — código MI/INOM (G-M)	28	0	0
Geradas — tempo relativo (G-T)	41	0	20
Geradas — ordenação (G-O)	19	0	16
Geradas — informais (G-A)	24	0	2
Geradas — amplas (G-P)	13	0	0
Geradas — fronteira (G-F)	7	1	0
Total	310	4	67

Fonte: Elaborada pelo autor a partir de `dataset.json`.

Justificativa da geração programática.

Consultas escritas à mão são caras de redigir e anotar e tendem a variar sobre um pequeno conjunto de padrões familiares aos autores. A geração por modelos de frase sobre catálogos fechados oferece cobertura sistemática de combinações raras (pares UF × escala pouco frequentes, por exemplo), escala sem custo adicional de anotação e volume suficiente para estimar intervalos de confiança e comparar modelos com testes de significância. A

contrapartida é a regularidade linguística: por isso os resultados são reportados também por camada, permitindo distinguir o desempenho nas consultas geradas do desempenho nas consultas redigidas por pessoas (P e N).

Tabela 5 – Consultas por categoria e por camada (métricas principais; uma consulta pode ter mais de uma categoria)

Categoria	P	N	G	Total
Simples (S)	0	11	89	100
Compostas (C)	22	21	137	180
Código MI/INOM (M)	10	5	42	57
Tempo relativo (T)	6	9	45	60
Ordenação (O)	4	6	19	29
Ambíguas/informais (A)	16	19	162	197
Fora do domínio (F)	0	2	6	8

Fonte: Elaborada pelo autor.

3.3.4 Manual de Anotação e Leituras Múltiplas

O gabarito é construído segundo um manual de anotação escrito (Apêndice A.1) que fixa seis princípios — evidência explícita no texto, forma canônica, leituras múltiplas explícitas, tratamento das consultas fora do domínio, consistência e casos observacionais — e regras para cada um dos doze campos da ferramenta.

O princípio mais importante para a confiabilidade da avaliação é o das **leituras múltiplas explícitas**. Uma parte das consultas admite, legitimamente, mais de uma resposta correta: “cartas de São Paulo” pode referir-se ao estado ou à capital; “produtos publicados nos últimos 3 meses” pode significar noventa dias ou três meses de calendário; “as cartas mais recentes” pode ser ordenado pela data de publicação ou pela de criação, pois a consulta não diz qual. Um gabarito que escolhesse uma única leitura penalizaria como erro uma resposta razoável e mediria a coincidência com a preferência do anotador, não a qualidade da tradução. Nesses casos o gabarito lista **todas** as leituras aceitas, por três mecanismos: *valor alternativo* (o campo aceita mais de um valor), *campo opcional* (pode faltar sem erro, como `limit = 1` em “a carta mais recente”) e *leitura estrutural alternativa* (o mesmo trecho pode ocupar campos diferentes, como `state` ou `city`). As expressões de tempo relativo são registradas como regras nomeadas, resolvidas na data da execução — a mesma data informada ao modelo no *prompt* de sistema — com as leituras aceitas do Quadro 5.

A ambiguidade não é, porém, tratada com leniência genérica: toda leitura aceita está escrita no gabarito, justificada por uma regra do manual e visível no Apêndice A; fora delas, a comparação é estrita (Seção 3.5.4). Das 306 consultas das métricas principais, a Tabela 4 indica quantas admitem mais de uma leitura.

Consultas fora do domínio e casos observacionais.

Oito consultas estão fora do domínio (N36, N37 e seis da família G-F): o comportamento correto é determinado — não chamar a ferramenta — e elas entram nas métricas principais, na categoria F. Quatro consultas são observacionais (N38, N39, N40 e GF007), porque o comportamento correto não é determinável a partir do texto: valor impossível de representar (“escala 1:10.000.000”, “cartas do estado 42”) ou pedido de resposta discutível (“cartas topo no oceano atlântico”, “cartas do futuro (ano 2050)”). Elas são executadas e o comportamento de cada modelo é reportado à parte; as métricas principais cobrem as 306 consultas restantes.

Tabela 6 – Frequência de cada campo na leitura preferencial do gabarito (métricas principais)

Campo	Ocorr.	Campo	Ocorr.
keyword	62	project	11
scale	84	publicationPeriod	51
productType	44	creationPeriod	10
state	130	sortField	29
city	27	sortDirection	29
supplyArea	40	limit	21

Fonte: Elaborada pelo autor.

3.4 Auditoria do *Dataset*

A confiabilidade das métricas depende diretamente da confiabilidade do gabarito. Por isso o *dataset* foi submetido a uma auditoria em três etapas, repetidas até não restar divergência sem decisão registrada. O procedimento, as ferramentas e os resultados completos estão no Apêndice A.14 e no repositório ([data/auditoria/](#)).

1. **Checagens automáticas** (`pfc-auditar`): unicidade de identificadores e de consultas (após normalização de caixa, acentos e pontuação); validade de todos os valores contra os enumerados da ferramenta; resolução de todas as regras de tempo relativo em datas diversas, inclusive anos bissextos e viradas de ano; rastreabilidade das consultas da camada P até a linha citada do arquivo de origem; um **anotador por regras**, que localiza na consulta o trecho que justifica cada campo do gabarito e aponta campo sem evidência, evidência sem campo e valor divergente; e uma verificação de **consistência** de convenções entre casos (a mesma expressão, a mesma anotação).
2. **Anotação independente às cegas**: todas as 310 consultas foram anotadas novamente, a partir do zero, por um modelo de linguagem de propósito geral de grande

porte (Claude Opus 5.5, da Anthropic), que não integra o conjunto de modelos avaliados. A anotação foi feita em dez lotes de 31 consultas, em sessões independentes, cada uma com acesso apenas ao manual de anotação, à definição da ferramenta, à data de referência e ao próprio lote — sem acesso ao gabarito, ao gerador ou aos demais lotes. Para cada consulta, o anotador registrou a decisão (chamar a ferramenta, não chamar ou caso observacional), a leitura preferencial, as leituras alternativas previstas pelo manual e uma justificativa por referência às regras.

3. **Adjudicação:** cada consulta com divergência em qualquer das medidas de concordância recebeu uma decisão registrada — gabarito mantido, corrigido ou ampliado com leitura alternativa — com justificativa por referência ao manual. O gabarito anterior a cada decisão fica registrado, de modo que a concordância anterior à adjudicação pode ser recalculada a partir do repositório.

A concordância entre o gabarito e a anotação independente foi medida em rigor crescente (Tabela 7): a decisão sobre chamar a ferramenta, com o kappa de Cohen (COHEN, 1960; ARTSTEIN; POESIO, 2008), que desconta a concordância esperada ao acaso; a compatibilidade das leituras nos dois sentidos (a leitura do anotador é aceita pelo gabarito e a leitura preferencial do gabarito é aceita pelo anotador); a identidade da leitura preferencial; a presença e o valor de cada campo; e a detecção de ambiguidade, isto é, se a consulta admite mais de uma leitura. A coluna “antes da adjudicação” mede a concordância propriamente dita; a coluna “após” mostra o que restou depois das decisões.

As medidas de campo e de ambiguidade mostram concordância quase perfeita já antes da adjudicação. A principal divergência foi de classificação: o gabarito tratava como observacionais oito consultas fora do domínio cujo comportamento correto é determinado pelo próprio manual (não chamar a ferramenta). A decisão foi corrigir o gabarito e incluí-las nas métricas principais, na categoria F. As demais divergências foram uma leitura ampliada (N33, em que “cartografia sistemática” passou a admitir a leitura sem o campo `project`, pois a expressão não consta da descrição da ferramenta) e cinco divergências mantidas com justificativa, entre elas três escolhas de leitura preferencial dentro do mesmo conjunto de leituras aceitas, que não afetam a pontuação. A lista completa está no Quadro 7 do Apêndice A.14.

Duas ressalvas delimitam o alcance da auditoria. O anotador por regras foi escrito pelos autores e, portanto, verifica evidência e consistência, mas não constitui uma anotação independente. A anotação às cegas é independente do gabarito, mas foi feita por um modelo de linguagem, e não por uma pessoa: ela identifica divergências com alta sensibilidade, e a decisão sobre cada uma é tomada na adjudicação, por referência ao manual. Na camada G, além disso, o gabarito é conhecido por construção, e a auditoria

Tabela 7 – Resultados da auditoria do *dataset*

Verificação	Antes da adjudicação	Após
<i>Checagens automáticas</i>		
Problemas estruturais (ids, duplicatas, enumerados, datas)	—	0
Consultas da camada P conferidas na linha de origem	—	22/22
Apontamentos do anotador por regras sem resolução	—	0
Expressões com anotação inconsistente sem justificativa	—	0
<i>Anotação independente às cegas (310 consultas)</i>		
Decisão (chamar / não chamar / observacional)	302/310 (97,4%)	310/310 (100,0%)
kappa de Cohen	0,658	1,000
Leituras compatíveis nos dois sentidos	300/310 (96,8%)	309/310 (99,7%)
Leitura preferencial idêntica ^a	294/298 (98,7%)	294/298 (98,7%)
Presença de cada campo na leitura preferencial ^{a,b} (kappa)	0,999	0,999
Valor do campo, quando ambos o anotam ^a	534/537 (99,4%)	534/537 (99,4%)
Consulta com mais de uma leitura aceita ^a	296/298 (99,3%)	297/298 (99,7%)
kappa de Cohen	0,981	0,990
Consultas com divergência em alguma medida, sem decisão registrada	14	0

Fonte: Elaborada pelo autor a partir de `data/auditoria/`. (a) Sobre as consultas em que gabarito e anotador pedem a chamada da ferramenta nas métricas principais. (b) Doze campos por consulta.

verifica sobretudo se a frase gerada sustenta, em português corrente, os parâmetros que a geraram.

3.5 Protocolo de Avaliação

3.5.1 Visão Geral

Cada modelo executa todas as consultas do *dataset*, em ordem fixa, sob condições controladas e reproduzíveis. A execução é automatizada pelo comando `pfc-avaliar` (Capítulo 4), que grava um registro por consulta e repetição — consulta, gabarito resolvido, parâmetros emitidos pelo modelo, *tool calls* brutos, texto de resposta, latência, contagem de *tokens* e classe de erro — e um manifesto de reprodutibilidade por rodada.

3.5.2 Ambientes de Execução

Estação de referência.

A estação que define o cenário de implantação modesta tem a seguinte configuração (especificação completa no Apêndice B):

- **CPU:** Intel Core i7-12700H (12.^a geração, 14 núcleos / 20 *threads*, base 2,3 GHz);
- **RAM:** 32 GB DDR4 a 3200 MHz;
- **GPU dedicada:** NVIDIA GeForce RTX 3050 Laptop GPU com 4 GB de VRAM;
- **Disco:** SSD NVMe de 512 GB;
- **Sistema operacional:** Microsoft Windows 11 Pro (versão 10.0.26200, 64 bits);
- **Software:** Python 3.14.4; Ollama 0.34.0; LangChain 1.x (`langchain-core` 1.6.3, `langchain-ollama` 1.1.0, cliente `ollama` 0.6.2); PostgreSQL 16.4 com PostGIS 3.4 em contêiner Docker, usado apenas na demonstração ponta a ponta.

Com 4 GB de VRAM, só modelos quantizados de até cerca de 4 bilhões de parâmetros executam integralmente em GPU; os demais recebem *offload* parcial para a CPU, com penalidade de latência.

Rodadas completas.

As rodadas completas dos quatro modelos locais, com três repetições por modelo, foram executadas numa GPU de nuvem NVIDIA T4 com 16 GB de VRAM, suficiente para cada modelo inteiro, com Ollama (na versão registrada no manifesto de cada rodada), as mesmas *tags* e quantizações e o mesmo `pfc-avaliar` usados na estação de referência. Isso separa o efeito do modelo do efeito do *offload* e evita ocupar a estação de referência durante horas de execução. A qualidade da tradução depende essencialmente do modelo, e não do *hardware* — diferenças numéricas entre execução em GPU e em CPU podem alterar respostas isoladas, o que a validação na estação de referência quantifica —; a latência, ao contrário, depende fortemente do *hardware* e é reportada separadamente por ambiente.

Validação na estação de referência.

As 62 consultas das camadas P e N foram executadas também na estação de referência pelos dois modelos de menor porte (Qwen 3 4B e Gemma 4 E2B), com uma

repetição, para medir a latência no cenário de implantação modesta e verificar a concordância das respostas com as rodadas completas. As duas rodadas foram feitas na mesma sessão; antes de cada uma, todos os modelos residentes no Ollama foram descarregados, para que cada modelo fosse carregado a frio e dispusesse da mesma VRAM livre. O manifesto registra a VRAM livre no início da rodada e a parcela do modelo que o Ollama alocou na GPU, porque a latência nessa estação depende diretamente desses dois valores.

Referência em nuvem.

Os dois modelos do Groq (Seção 3.2) executaram as 310 consultas com uma repetição, respeitando os limites de taxa do provedor.

3.5.3 Configuração dos Modelos

Todos os modelos foram avaliados com a mesma configuração: **temperatura zero**; limite de **1024 *tokens*** de saída; **nenhum contexto de conversações anteriores** (*zero-shot*); **nenhum exemplo resolvido no *prompt*** (*few-shot*) e **nenhum dicionário de normalização**; o mesmo *prompt* de sistema, com a data de referência injetada; e a mesma definição da ferramenta `buscar_catalogo`. Não há nova tentativa nem mecanismo de *fallback*: cada falha do modelo é registrada e contada.

Modo *thinking*.

O modo de raciocínio interno foi desativado por parâmetro (`think=false` na API do Ollama) nos modelos que o declaram, para comparabilidade das latências. Na *build* padrão do Qwen 3 (`qwen3:4b`) não foi possível desativá-lo de forma limpa — nem pela diretiva `/no_think` no *prompt* nem pelo parâmetro `think=false` —: nos testes preliminares, o modelo consumia integralmente o limite de 1024 *tokens* de saída e, em parte das consultas, sequer emitia a chamada de ferramenta. Adotou-se, por isso, a *build* oficial sem raciocínio (*Qwen3-4B-Instruct-2507*), de mesmo porte e mesma quantização. Na referência em nuvem, o esforço de raciocínio foi fixado no mínimo aceito por cada modelo (`none` para o Qwen; `low` para a família GPT-OSS, que não permite desligá-lo), o que é registrado no manifesto.

3.5.4 Métricas

Comparação.

A resposta do modelo é comparada com o gabarito campo a campo (Capítulo 2): enumerados exigem igualdade exata; *strings* livres (`keyword`, `state`, `city`) são comparadas após normalização de caixa, acentos e pontuação; períodos exigem coincidência

exata dos limites; *limit* é inteiro. Quando o gabarito tem leituras múltiplas, a resposta é comparada com cada leitura aceita e avaliada contra a de melhor casamento: correta se coincidir com alguma; por campo, contra a leitura que maximiza acertos menos erros. Campos opcionais ausentes não contam como erro.

Acurácia por consulta.

Uma consulta é correta se o conjunto de parâmetros emitidos coincide com uma leitura aceita — nem campos a mais, nem a menos. Nas consultas cujo gabarito é não chamar a ferramenta, correta é a resposta sem *tool call*. A acurácia é reportada com intervalo de confiança de 95% pelo método de Wilson (WILSON, 1927), adequado a proporções com amostras moderadas.

Precisão, *recall* e F1 por campo.

Para cada campo: verdadeiro positivo (TP) quando o campo é emitido e corresponde a um valor aceito; falso positivo (FP) quando é emitido e não corresponde, ou não deveria ter sido emitido; falso negativo (FN) quando é obrigatório no gabarito e não foi emitido. Um valor errado conta, portanto, como FP. O F1 médio é ponderado pelo número de ocorrências de cada campo no gabarito; reportam-se também as médias *macro* e *micro*.

Latência.

Tempo da chamada de tradução: do envio da consulta ao modelo até o recebimento da resposta com os *tool calls*, medido com relógio monotônico. O tempo de execução da ferramenta (SQL) é registrado à parte e não entra na comparação entre modelos. Reportam-se mínimo, mediana, média, percentil 95 e máximo, por ambiente de execução.

Comparação entre modelos.

Diferenças de acurácia entre dois modelos, sobre as mesmas consultas, são testadas pelo teste de McNemar exato (MCNEMAR, 1947), que usa apenas os pares discordantes (consultas em que só um dos modelos acerta). As repetições de um mesmo modelo entram como pares independentes, aproximação declarada nos resultados.

Análise de erros.

Cada resposta incorreta é classificada em: campo inventado (emitido sem estar no gabarito), campo omitido, valor errado, erro misto, não chamou a ferramenta quando devia, chamou quando não devia e argumentos fora do *schema* (tipo inválido ou campo

inexistente). Erros de infraestrutura (*timeout*, indisponibilidade) são separados dos erros do modelo e não entram nas estatísticas de latência. Como discutido no Capítulo 2, **não se busca um ranking único**: cada métrica revela um aspecto da tradução, e a recomendação final pondera qualidade e latência.

3.5.5 Reprodutibilidade

Cada rodada registra em manifesto: *tag* e *digest* do modelo no Ollama, versões do Ollama e do LangChain, configuração de inferência, data de referência, *hardware* e os *hashes* SHA-256 do *dataset* e da definição da ferramenta. A pontuação é sempre recalculada contra o gabarito vigente: uma correção de gabarito não exige nova execução dos modelos, e uma consulta cujo texto tenha mudado é descartada da pontuação, com registro. O *dataset*, o gerador, o manual, os relatórios de auditoria, os registros de execução e os *scripts* estão no repositório do projeto (Apêndice C).

4 APRIMORAMENTO DO PROTÓTIPO: ESTRUTURA ANALÍTICA DO PROJETO

Este capítulo apresenta a execução do projeto de aprimoramento da solução de busca em linguagem natural sobre o acervo cartográfico da DSG. Diferentemente de uma seção de *trabalhos relacionados* no sentido clássico — que tipicamente serve ao levantamento de hipóteses em pesquisas de iniciação científica —, este capítulo é organizado segundo uma **Estrutura Analítica do Projeto (EAP)**, que decompõe o aprimoramento em quatro grandes etapas, sequenciais mas com pontos de realimentação: (1) *Análise do Protótipo*; (2) *Elicitação de Requisitos*; (3) *Planejamento da Migração*; e (4) *Desenvolvimento*. A EAP é apresentada graficamente na Figura 3 e norteia tanto a redação deste capítulo quanto o cronograma de execução, apresentado ao final.

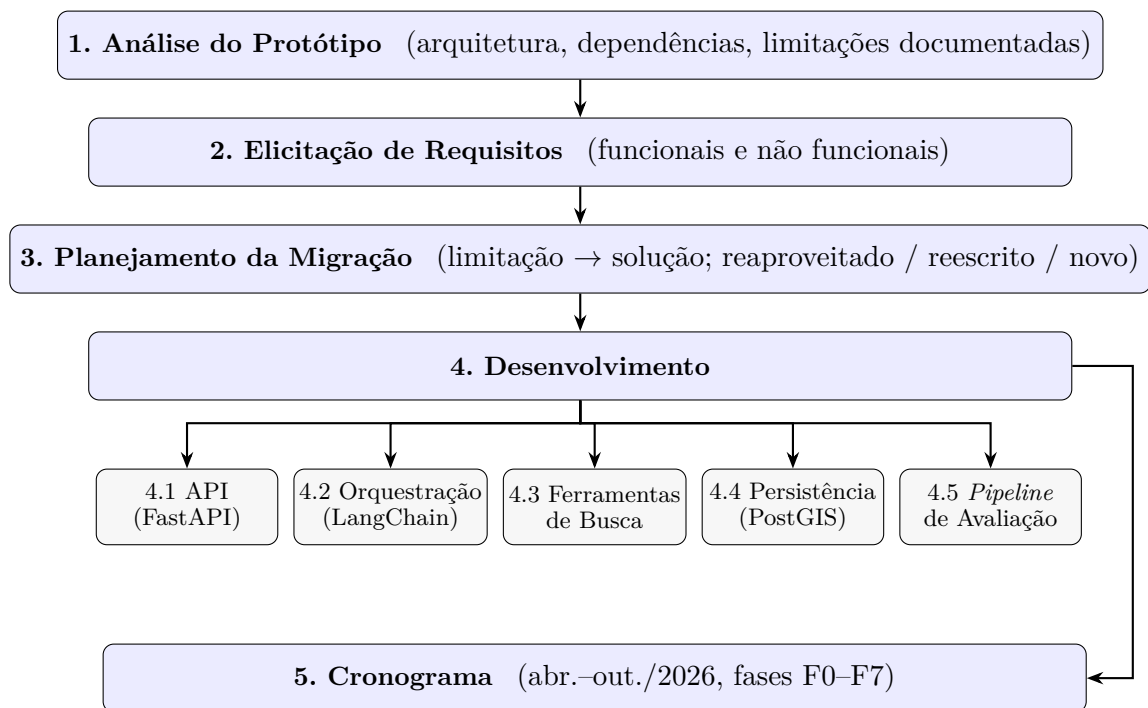


Figura 3 – Estrutura Analítica do Projeto (EAP) adotada no aprimoramento

Fonte: Elaborado pelo autor.

4.1 Análise do Protótipo do 1º CGEO

4.1.1 Visão Geral

O 1º Centro de Geoinformação (1º CGEO), por meio de sua Divisão de Levantamento “Gen Augusto Tasso Fragoso”, desenvolveu um protótipo funcional de busca em

linguagem natural sobre o catálogo de dados geoespaciais sob sua guarda (1º Centro de Geoinformação, 2025). O protótipo foi concebido como prova de conceito para avaliar a viabilidade do uso de LLMs locais no contexto cartográfico brasileiro e está disponível publicamente no repositório GitHub do 1º CGEO. Este protótipo é o *benchmark* de referência do presente trabalho.

4.1.2 Arquitetura do Protótipo

A arquitetura do protótipo é composta por quatro camadas principais, conforme ilustrado na Figura 4.

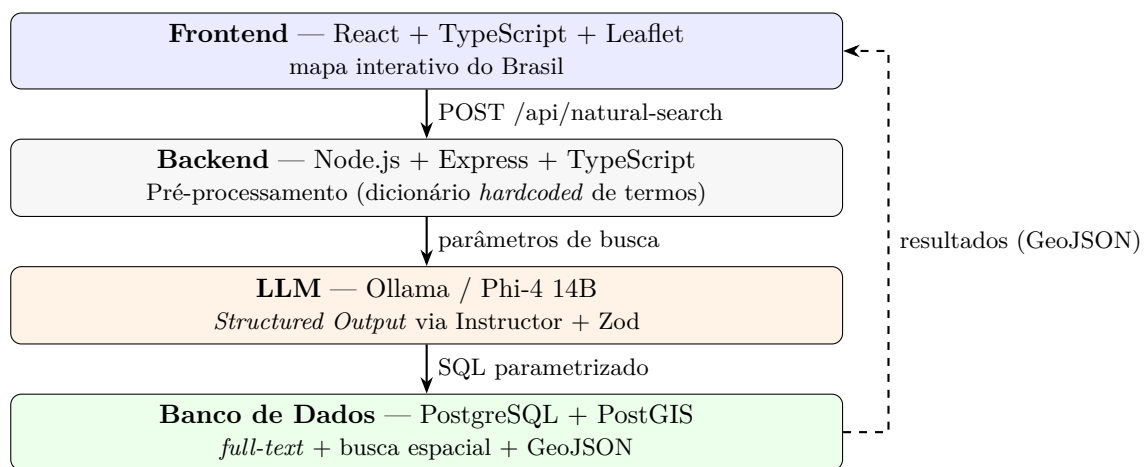


Figura 4 – Arquitetura do protótipo de busca do 1º CGEO

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

Frontend. Implementado em React com TypeScript, utiliza a biblioteca Leaflet para exibição de um mapa interativo do Brasil. O usuário digita sua consulta em linguagem natural em um campo de texto, e os resultados são exibidos em lista paginada e como marcadores no mapa.

Backend. Implementado em Node.js com Express e TypeScript, recebe as consultas via rota REST (`POST /api/natural-search`), aplica pré-processamento de termos e encaminha a consulta ao serviço de LLM, cujo resultado é convertido em SQL parametrizado para o banco PostgreSQL/PostGIS.

Pré-processamento. Antes de enviar a consulta ao LLM, o protótipo aplica um módulo de pré-processamento (`preprocessor.ts`) que normaliza variações ortográficas comuns no domínio cartográfico. O módulo utiliza um dicionário *hardcoded* de sinônimos e variações, com mapeamentos como “25k” → “1:25.000” ou “primeiro cgeo” → “1º Centro de Geoinformação”.

Serviço de LLM com *Structured Output*. O núcleo do protótipo é o serviço de LLM (`service.ts`), que utiliza o modelo *Phi-4 14B* executado localmente via Ol-

lama. A integração é feita pela biblioteca Instructor para TypeScript, que usa Zod para definir o *schema* de saída. O *schema* prevê os campos: `keyword`, `scale`, `productType`, `state`, `city`, `supplyArea`, `project`, `publicationPeriod`, `creationPeriod`, `sortField`, `sortDirection` e `limit`. O *prompt* do sistema fornece ao modelo exemplos detalhados (*few-shot prompting*) para guiar a extração.

Fallback com expressões regulares. Em caso de falha do LLM (exceção após `max_retries = 3`), o protótipo aciona um mecanismo de *fallback* baseado em expressões regulares (`utils.ts`), que tenta extrair os parâmetros mais simples (escalas, datas) diretamente do texto.

Banco de dados. O banco utiliza PostgreSQL com PostGIS para armazenar os metadados dos produtos cartográficos e suas geometrias (*bounding boxes* das folhas). As consultas, montadas no serviço `search.ts` e executadas pela biblioteca `pg-promise`, combinam pesquisa textual (*full-text search*) com operações espaciais do PostGIS para retornar resultados em GeoJSON. O repositório do protótipo contém o esquema do banco (`postgres/init.sql`, com as tabelas `datasets`, `estados`, `municipios` e `areas_suprimento`), mas não os dados do acervo.

4.1.3 Contribuições do Protótipo

O protótipo representa contribuição pioneira no contexto brasileiro, demonstrando que: (i) a abordagem baseada em LLMs locais é tecnicamente viável para o domínio cartográfico; (ii) o modelo Phi-4 14B, via Ollama, é capaz de extrair parâmetros de busca relevantes a partir de consultas em linguagem natural em português; (iii) a combinação de *few-shot prompting* com *Structured Output* produz resultados utilizáveis para a maioria das consultas; e (iv) a integração de LLM com PostgreSQL/PostGIS para busca espacial é viável em ambiente controlado.

4.1.4 Limitações Identificadas

O próprio cliente identificou explicitamente seis limitações do protótipo, que constituem as frentes de aprimoramento deste Projeto Final de Curso:

1. **Backend em Node.js.** O ecossistema Node.js tem suporte consideravelmente menor do que Python para bibliotecas de IA e aprendizado de máquina; as bibliotecas de integração com LLMs (LangChain, LlamaIndex, Instructor) têm versões mais completas em Python.
2. **Apenas Saída Estruturada.** A abordagem força o modelo a sempre retornar a estrutura predefinida, independentemente da consulta, e não permite encadear ferramentas nem adaptar o comportamento ao resultado de operações anteriores.

3. **Modelo único e defasado (Phi-4 14B).** O uso de um único modelo impede a seleção embasada do mais adequado ao caso de uso; o ecossistema evolui rapidamente, e a escolha precisa ser revista à luz dos modelos mais recentes.
4. **Pré-processamento manual com dicionário *hardcoded*.** O protótipo mantém um dicionário fixo de 141 pares (“25k” → “1:25.000”, “primeiro cgeo” → “1º Centro de Geoinformação”, “carta ortoimg” → “SCN Carta Ortoimagem”) que é aplicado **antes** de o texto chegar ao LLM. Isso traz três problemas: (i) o dicionário só cobre variações *previstas em tempo de desenvolvimento* — grafias novas que emergem no uso operacional (“dois cgeo” em vez de “segundo cgeo”, “25 mil” em vez de “25k”) falham silenciosamente; (ii) sua manutenção é **manual** e depende do desenvolvedor observar erros no *log* e propor novas entradas; (iii) o pré-processamento pode **deturpar** consultas legítimas quando um termo do dicionário aparece em contexto ambíguo. A delegação ao LLM elimina o dicionário fixo em troca do conhecimento semântico do modelo, que reconhece variações inéditas sem alteração de código.
5. **Fallback com expressões regulares.** Sua presença é sintoma de falhas do LLM que não foram sistematicamente documentadas, dificultando a análise de causa raiz.
6. **Ausência de métricas de avaliação.** O protótipo dispõe apenas de 22 casos de teste, pontuados por pesos atribuídos *ad hoc* aos campos esperados, sem penalizar campos preenchidos indevidamente, e não há protocolo de medição, o que impede comparar objetivamente modelos, *prompts* ou configurações.

4.2 Elicitação de Requisitos

A análise do protótipo e as conversas com o cliente conduziram ao seguinte conjunto de requisitos para a solução aprimorada.

4.2.1 Requisitos Funcionais

- RF1** Receber, por uma rota REST, uma consulta em linguagem natural em português.
- RF2** Traduzir a consulta em parâmetros técnicos compatíveis com os campos do catálogo (`keyword`, `scale`, `productType`, `state`, `city`, `supplyArea`, `project`, `publicationPeriod`, `creationPeriod`, `sortField`, `sortDirection`, `limit`).
- RF3** Executar a busca técnica resultante no banco PostgreSQL/PostGIS, preservando a lógica de filtros e a saída em GeoJSON do protótipo.
- RF4** Permitir a seleção, em tempo de requisição, do modelo de LLM utilizado para a tradução, viabilizando a avaliação comparativa.

RF5 Registrar, para cada requisição, os parâmetros extraídos, a sequência de *tool calls*, os tempos de cada etapa e eventuais erros, em formato adequado ao cálculo automatizado das métricas.

4.2.2 Requisitos Não Funcionais

RNF1 Execução totalmente local: nenhuma chamada a serviços de LLM em nuvem (privacidade e segurança).

RNF2 Latência aceitável: a latência por consulta deve ser compatível com uso interativo; ela é medida e reportada por modelo, de modo que o limite operacional possa ser fixado pelo cliente conforme o contexto de implantação.

RNF3 Conformidade com normas: aderência ao Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) e às ET-PCDG no que diz respeito à estrutura dos metadados consultados.

RNF4 Reprodutibilidade: *tags* dos modelos, versões das bibliotecas e *hashes* do *dataset* e da definição da ferramenta registrados em cada rodada de avaliação.

RNF5 Manutenibilidade: código modular, com testes automatizados das funções de busca e do cálculo das métricas.

RNF6 Substituibilidade de modelo: troca do modelo sem alteração da camada de orquestração, apenas por configuração.

4.3 Planejamento da Migração

A migração não parte do zero: o protótipo fornece decisões de projeto já validadas (estrutura do banco, catálogo de parâmetros, lógica de *queries* espaciais) que são integralmente aproveitadas. A estratégia adotada é, portanto, de **substituição dirigida**: identificar os componentes responsáveis pelas limitações documentadas e substituí-los, preservando o que já funciona.

4.3.1 Mapeamento Limitação → Solução

O Quadro 2 relaciona cada limitação identificada na Seção 4.1.4 ao componente do novo sistema que a endereça.

4.3.2 Componentes Reaproveitados, Reescritos e Novos

O Quadro 3 classifica cada componente do sistema final em relação ao protótipo.

Quadro 2 – Mapeamento entre limitações do protótipo e soluções adotadas

#	Limitação	Solução adotada	Componente afetado
1	<i>Backend</i> em Node.js	Reimplementação em Python com FastAPI	Nova API REST
2	Apenas <i>Structured Output</i>	<i>Tool Calling</i> nativo via LangChain	Serviço de LLM
3	Modelo único (Phi-4 14B)	Avaliação comparativa de quatro modelos atuais (Qwen 3 4B, Gemma 4 E2B, Gemma 4 E4B, Mistral Nemo 12B)	Configuração e testes
4	Pré-processamento manual	Normalização delegada ao LLM, orientada pelas descrições do <i>schema</i> da ferramenta	Remoção do pré-processador
5	<i>Fallback</i> via regex	Falhas registradas como métrica, sem <i>fallback</i> oculto	Serviço de LLM
6	Ausência de métricas	<i>Dataset</i> auditado com gabarito + <i>pipeline</i> de avaliação automatizado	Novo módulo de avaliação

Fonte: Elaborado pelo autor.

4.4 Desenvolvimento

A arquitetura da solução aprimorada é apresentada na Figura 5, com as camadas detalhadas nas subseções seguintes.

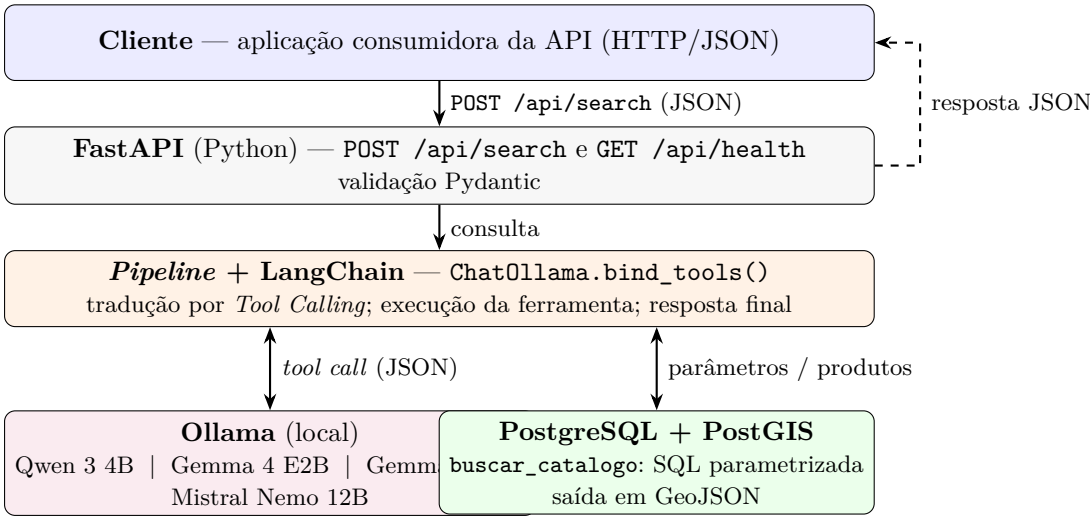


Figura 5 – Arquitetura da solução aprimorada de busca

Fonte: Elaborado pelo autor.

Quadro 3 – Classificação dos componentes do sistema final em relação ao protótipo

Componente	Categoria	Observação
Esquema do banco PostgreSQL/PostGIS	Reaproveitado	Mesmo esquema do protótipo (<code>init.sql</code> , quatro tabelas), sem alterações
Conjunto de parâmetros de busca	Reaproveitado	Os mesmos doze campos, compatíveis com a lógica das consultas SQL; enumerados grafados conforme as ET-PCDG
Lógica das consultas SQL	Reaproveitado	Portada de <code>search.ts</code> ; sem o filtro por retângulo do mapa e a paginação, próprios do <i>frontend</i>
<i>Frontend</i> React/Leaflet	Fora do escopo	Não é parte deste trabalho
API REST	Reescrito	Node.js/Express → Python/FastAPI
Serviço de LLM	Reescrito	<i>Structured Output</i> (Instructor/Zod) → <i>Tool Calling</i> (LangChain)
Camada de acesso ao banco	Reescrito	<code>pg-promise</code> → <code>psycopg</code>
Pré-processamento de termos	Eliminado	Função transferida ao LLM
<i>Fallback</i> com regex	Eliminado	Falhas viram métricas
Semente sintética do banco	Novo	Apenas para a demonstração ponta a ponta; nenhuma métrica depende dela
<i>Dataset</i> de avaliação	Novo	310 consultas com gabarito auditado (Cap. 3 e Ap. A)
<i>Pipeline</i> de avaliação	Novo	Comandos <code>pfc-dataset</code> , <code>pfc-auditar</code> , <code>pfc-avaliar</code> e <code>pfc-relatorio</code>

Fonte: Elaborado pelo autor.

4.4.1 Camada de API: FastAPI

O FastAPI (RAMÍREZ, 2018) é o *framework* da API REST, que expõe duas rotas. A rota `POST /api/search` recebe um objeto JSON com o campo `query` (consulta em linguagem natural) e devolve os parâmetros técnicos extraídos pelo LLM, a sequência de *tool calls*, os produtos cartográficos encontrados (total e itens, com geometria em GeoJSON), a resposta textual ao usuário, as latências de cada etapa (tradução, execução da SQL, resposta final e total) e eventuais erros e avisos, atendendo a RF1, RF2 e RF3 e expondo os traços de RF5, cujo registro para o cálculo das métricas é feito pelo *pipeline* de avaliação (Seção 4.4.5). A rota `GET /api/health` informa a versão do servidor Ollama, a disponibilidade do banco e o modelo configurado. A escolha do FastAPI justifica-se pela validação automática das requisições via Pydantic, pela documentação OpenAPI (Swagger) gerada automaticamente e pela integração direta com o ecossistema Python de IA. A camada de API é mantida minimalista; toda a lógica de orquestração reside no módulo de *pipeline*. O *pipeline* de avaliação não passa pela API: invoca diretamente a

camada de tradução, de modo que a latência medida corresponde à chamada ao modelo, sem o custo do transporte HTTP.

Seleção de modelo.

O serviço aceita um campo opcional `model` na requisição (RF4), habilitado apenas no ambiente de desenvolvimento por variável de ambiente; fora dele, a requisição que solicita um modelo diferente do configurado é recusada. Na configuração operacional, o modelo é fixado por variável de ambiente no servidor: alternar o modelo entre requisições implicaria desalocar e recarregar pesos na VRAM, com custo de segundos por chamada e problemas de concorrência em cenários multiusuário.

Tratamento de falhas.

Falhas do LLM não são recuperadas por regras determinísticas nem por novas tentativas: cada falha é **registrada como métrica**, o que permite atribuir corretamente os erros ao modelo (limitação 5 da Seção 4.1.4). As falhas são classificadas em duas famílias: **erros de infraestrutura** (tempo esgotado, servidor ou modelo indisponível, falha de execução) e **erros do modelo** (argumentos fora do *schema*, chamada a ferramenta inexistente). Uma chamada com argumentos fora do tipo esperado é preservada como veio, e a comparação campo a campo com o gabarito contabiliza o erro. Para uma versão de produção destinada aos usuários finais, é desejável um caminho de *fallback* suave (por exemplo, delegar a consulta bruta à busca textual do PostgreSQL), preservando comportamento útil diante de falhas; essa evolução é apontada como trabalho futuro no Capítulo 6.

4.4.2 Camada de Orquestração: LangChain + Ollama

A integração com o Ollama é feita pela classe `ChatOllama` do LangChain (CHASE, 2022), que encapsula a API nativa do Ollama: envia a consulta ao modelo junto com a definição da ferramenta e devolve, de forma estruturada, as chamadas de ferramenta que o modelo decidiu gerar — desativando, por parâmetro (`reasoning=False`, que envia `think=false` à API do Ollama), o modo *thinking* nos modelos que declaram essa capacidade. O objeto `ChatOllama` é instanciado com o nome do modelo, temperatura zero, limite de 1024 *tokens* de saída e o *base URL* do Ollama; a ligação da ferramenta é feita pelo método `bind_tools()`, que recebe a definição de `buscar_catalogo` em JSON Schema; a invocação é uma única chamada `invoke()`, cujo retorno traz a lista de *tool calls*. Na versão 1.x do LangChain, adotada neste trabalho, o antigo `AgentExecutor` foi descontinuado; como a etapa avaliada é exatamente essa única decisão do modelo (chamar ou não a ferramenta, e com quais argumentos), o ciclo completo — executar a ferramenta e devolver

o resultado ao modelo para a resposta final — é implementado explicitamente no módulo de *pipeline*, sem necessidade de um grafo de estados como o LangGraph (LangChain AI, 2024).

No caso do Qwen 3 4B, empregou-se a *build* oficial sem raciocínio, de mesmo porte e mesma quantização (*tag qwen3:4b-instruct-2507-q4_K_M*), porque na *build qwen3:4b* o raciocínio não pôde ser desligado nem pela diretiva */no_think* nem pelo parâmetro *think=false* (Capítulo 3, Seção 3.5.3).

A mesma camada admite outro provedor sem alteração do restante do código: substituindo ChatOllama por ChatOpenAI, apontado para um *endpoint* compatível com a API da OpenAI, o *bind_tools()* e a interpretação das *tool calls* permanecem idênticos. Essa portabilidade, discutida no Capítulo 2, foi empregada para obter o resultado paralelo com modelos maiores servidos em nuvem pelo Groq (Seção 4.4.5), sem qualquer efeito sobre a configuração local avaliada.

4.4.3 Ferramentas de Busca

A solução expõe ao LLM uma única ferramenta, *buscar_catalogo*, em linha direta com a funcionalidade do protótipo. Ela é descrita por um *schema* JSON explícito — nome, descrição geral e os doze parâmetros de busca (*keyword*, *scale*, *productType*, *state*, *city*, *supplyArea*, *project*, *publicationPeriod*, *creationPeriod*, *sortField*, *sortDirection*, *limit*), cada um com tipo, enumeração dos valores válidos quando aplicável e instruções de normalização na própria descrição (por exemplo, “25k” corresponde a “1:25.000”) — mantido em um único módulo do repositório (*schema.py*). O mesmo módulo é a fonte dos campos e tipos usados na validação dos argumentos devolvidos pelo modelo e dos enumerados usados na validação do *dataset*. Manter o *schema* explícito, em vez de derivá-lo de *docstrings*, garante que o texto lido pelo modelo é exatamente o que está versionado no repositório; seu *hash* é registrado no manifesto de cada rodada de avaliação.

As descrições dos parâmetros são o único lugar em que o modelo recebe exemplos de normalização: o *prompt* de sistema apenas instrui a converter cada valor para a forma canônica descrita na ferramenta, sem dicionário de termos nem exemplos resolvidos (*few-shot*). As 141 entradas do dicionário do protótipo são, assim, substituídas pela capacidade semântica do modelo (limitação 4). A função Python que implementa a ferramenta recebe os argumentos decididos pelo modelo e executa a consulta parametrizada descrita na Seção 4.4.4.

Ferramentas auxiliares.

Ferramentas como `validar_inom` e `validar_mi` (verificação sintática de códigos INOM e MI antes da consulta ao banco) e `listar_tipos_produto` (retorno dos valores válidos de `productType`, útil quando o modelo precisa deduzir o tipo a partir de um sinônimo informal) não integram a solução avaliada, cujo objeto é a decisão única do modelo sobre `buscar_catalogo`. Sua inclusão, orientada pelos padrões de erro por campo observados no Capítulo 5, fica como trabalho futuro.

4.4.4 Camada de Persistência

O esquema do banco é o do protótipo: PostgreSQL com PostGIS (PostGIS Development Team, 2001), com as quatro tabelas definidas em `postgres/init.sql` — `datasets` (metadados e geometria de cada produto, com um vetor de busca textual gerado a partir dos campos descritivos), `estados`, `municipios` e `areas_suprimento` —, reaproveitado sem alterações. Como o repositório do protótipo contém apenas esse esquema, sem os dados do acervo, a demonstração ponta a ponta povoa o mesmo esquema com uma **semente sintética**, gerada por *script* com semente fixa e rotulada como sintética nos comentários de cada tabela: 27 unidades da federação representadas como retângulos numa grade, 14 municípios, as cinco áreas de suprimento dos CGEO e 117 produtos. A semente serve apenas para demonstrar que a arquitetura fecha — da consulta em linguagem natural ao resultado em GeoJSON —; **nenhuma métrica deste trabalho depende do banco**, pois a avaliação incide sobre os parâmetros extraídos pelo modelo (Capítulo 3). O ambiente é levantado por Docker Compose, com um contêiner PostgreSQL 16 com PostGIS 3.4 que aplica o esquema e a semente na inicialização.

A camada de acesso foi reimplementada em Python com `psycopg`, em substituição à biblioteca `pg-promise` do protótipo. A lógica de construção das consultas, portada de `backend/src/services/search.ts`, foi preservada: busca textual (*full-text search* em português) combinada com a identificação de códigos MI e INOM por expressão regular; filtros exatos de escala, tipo de produto e projeto; intervalos de datas de publicação e de criação; interseções espaciais (`ST_Intersects`) com estados, municípios e áreas de suprimento; ordenação e limite de resultados. Para preservar a fidelidade ao protótipo, as mudanças limitaram-se à sintaxe do *driver*, à restrição da direção de ordenação a `ASC/DESC` e à retirada do filtro por retângulo envolvente e da paginação, próprios do *frontend* de mapa (fora do escopo); o limite de resultados passou a vir do parâmetro `limit` da ferramenta. Todas as consultas são parametrizadas: o modelo nunca escreve SQL, apenas preenche os parâmetros da ferramenta, e a função monta a consulta com os valores passados como argumentos do *driver*.

Resolução dos campos geográficos: fora do escopo.

Os campos `state` e `city` extraídos pelo LLM são *strings* livres em português, que a consulta casa por nome com as tabelas de estados e municípios antes da interseção espacial com a geometria de cada folha. A correção desse resultado depende de geometrias reais das unidades territoriais (malhas do IBGE, Nominatim/OSM) e da qualidade das geometrias dos produtos importadas do catálogo (PYCSW), o que constitui uma frente de trabalho própria e **fica fora do escopo deste PFC**: a avaliação mede se o LLM extraiu o nome correto do estado ou do município, e não se a consulta espacial resultante retorna o conjunto exato de folhas. O tratamento completo dessa questão é apontado como trabalho futuro no Capítulo 6.

4.4.5 Pipeline de Avaliação

O *pipeline* de avaliação automatiza a execução do *dataset* de avaliação (Capítulo 3, Seção 3.3; Apêndice A) sobre cada modelo e a produção dos números apresentados no Capítulo 5. É composto por quatro comandos, declarados como pontos de entrada do pacote Python:

- **pfc-dataset**: constrói, a partir de uma fonte única — os casos das camadas P e N, mantidos em módulo próprio, e o gerador da camada G, com semente fixa —, o arquivo JSON executado pelo avaliador e os blocos do Apêndice A, de modo que o texto e o arquivo avaliado não podem divergir; os enumerados e campos de cada gabarito são validados contra o *schema* da ferramenta;
- **pfc-auditar**: executa as checagens automáticas da auditoria do *dataset* — estrutura (identificadores únicos, consultas não repetidas, enumerados válidos, expressões de tempo relativo que resolvem em diferentes datas de referência); rastreabilidade (toda consulta tem fonte, e as da camada P aparecem literalmente na linha citada do arquivo de casos de teste do protótipo); evidência, por meio de um anotador baseado em regras, escrito sem acesso ao gabarito, que aponta campos anotados sem evidência no texto da consulta, evidências sem campo anotado e valores divergentes; e consistência, exigindo a mesma anotação para a mesma expressão em todos os casos. O comando também compara o gabarito com a anotação independente feita às cegas e lista cada divergência; esses apontamentos alimentam a adjudicação descrita no Capítulo 3;
- **pfc-avaliar**: executa as consultas sobre um modelo, em ordem fixa e sequencialmente, e grava um registro por consulta $\times$ repetição em formato JSON Lines (texto da consulta, data de referência, *tool calls*, argumentos, tempos, *tokens* e erro classificado). A execução é retomável: se interrompida, é continuada pelo mesmo

comando, sem repetir o que já foi registrado. Cada rodada gera um **manifesto de reprodutibilidade** (RNF4) com a *tag* e o *digest* do modelo, as versões de *software* (Python, Ollama, LangChain), o *hardware*, a data de referência injetada no *prompt* e os *hashes* SHA-256 do *dataset* e do *schema* da ferramenta. A pontuação é **sempre recalculada contra o gabarito vigente**: o registro guarda a resposta do modelo e a data de referência, e as métricas (Capítulo 3, Seção 3.5.4) são refeitas a partir do *dataset* atual. Assim, uma correção de gabarito decidida na auditoria não exige nova execução dos modelos, e registros cuja consulta teve o texto alterado são descartados e listados no resumo;

- **pfc-relatorio**: consolida os resultados dos modelos nas tabelas LaTeX e nas figuras do Capítulo 5 (comparativo geral, F1 por categoria e por campo, latência), calcula o intervalo de confiança de Wilson para a acurácia e o teste exato de McNemar pareado entre modelos, e extrai uma amostra de erros representativos por tipo para a análise qualitativa.

Provedor alternativo e resultado paralelo.

O comando `pfc-avaliar` aceita a opção `--provedor groq`, que substitui `ChatOllama` por `ChatOpenAI` apontado para o *endpoint* compatível do Groq, mantendo o mesmo *prompt*, a mesma ferramenta, o mesmo *dataset* e as mesmas métricas. Com essa opção foram avaliados dois modelos maiores servidos em nuvem (`openai/gpt-oss-20b` e `openai/gpt-oss-120b`). Esses resultados são gravados em diretório próprio, com manifesto rotulado como execução em nuvem, e reportados à parte no Capítulo 5 como referência: por dependerem de serviço externo, estão fora do requisito de execução local (RNF1) e não concorrem à recomendação de modelo para implantação. Duas particularidades do provedor são tratadas explicitamente: as respostas de limite de taxa do plano gratuito são aguardadas e a chamada é repetida — assim como, até três vezes, as falhas transitórias de conexão ou do servidor, antes de serem registradas como erro de infraestrutura —, registrando-se apenas a latência da tentativa respondida; e o servidor valida os argumentos contra o *schema* e rejeita a *tool call* inválida, o que é contado como erro do modelo, com os argumentos gerados — quando recuperáveis da mensagem de rejeição — avaliados campo a campo, como na execução local.

4.4.6 Ferramentas de Apoio ao Desenvolvimento

Em apoio ao desenvolvimento e à reprodutibilidade, foram empregados: `git`, para controle de versão; `pytest`, com 78 testes automatizados que cobrem o *schema* da ferramenta, a montagem das consultas SQL, a resolução das expressões de tempo relativo, as invariantes do *dataset*, o cálculo das métricas e a geração dos relatórios (RNF5); `ruff`, para análise estática do código; Docker Compose, para subir o contêiner PostgreSQL/PostGIS

de forma reproduzível; e o arquivo `pyproject.toml`, que declara as dependências e os comandos do pacote, instalados com `pip` em ambiente virtual Python. A política de divulgação pública do repositório é definida em conjunto com o 1º CGEO.

4.5 Cronograma

O projeto foi executado entre abril e outubro de 2026, em oito fases (F0–F7) alinhadas às etapas da EAP, conforme o Quadro 4.

Quadro 4 – Cronograma de execução do projeto

Fase	Atividade	Período	Marco
F0	Análise do protótipo: código-fonte, dependências, esquema do banco e limitações documentadas	abr/2026	Limitações mapeadas
F1	Elicitação de requisitos e planejamento da migração (limitação → solução; componentes reaproveitados, reescritos e novos)	mai/2026	Requisitos e plano de migração
F2	Referencial teórico; seleção dos modelos; definição das métricas e do protocolo de avaliação	mai–jun/2026	Protocolo de avaliação
F3	Construção do <i>dataset</i> de avaliação: camadas P, N e G e respectivo gabarito (Apêndice A)	jun–jul/2026	310 consultas anotadas
F4	Prova de conceito do <i>Tool Calling</i> sobre consultas representativas, com modelo local e em nuvem	ago/2026	Viabilidade do <i>Tool Calling</i> demonstrada
F5	Implementação: API, orquestração, ferramenta <code>buscar_catalogo</code> , persistência e <i>pipeline</i> de avaliação, com testes automatizados	set/2026	Sistema ponta a ponta
F6	Experimentos (modelos locais, validação na estação local e resultado paralelo em nuvem); auditoria do <i>dataset</i> ; análise de erros	set/2026	Tabelas e figuras do Cap. 5
F7	Redação dos resultados e da conclusão; preparação da defesa	set–out/2026	Defesa (out/2026)

Fonte: Elaborado pelo autor.

A fase F6 foi conduzida em três ambientes, com os mesmos comandos de avaliação: (i) as rodadas completas dos quatro modelos locais, com Ollama em uma GPU de nuvem (NVIDIA T4, 16 GB) e o mesmo *software*; (ii) um subconjunto de validação (camadas P e N) na estação local com RTX 3050 Laptop de 4 GB de VRAM (Capítulo 3, Seção 3.5.2); e (iii) o resultado paralelo com os modelos servidos pelo Groq, fora do requisito de execução local e reportado à parte. A configuração de cada ambiente e os resultados correspondentes são apresentados no Capítulo 5 (Seção 5.1).

A configuração do *hardware* da estação local foi coletada por meio dos comandos PowerShell apresentados no Apêndice B; a saída na íntegra está reproduzida na Seção B.1.2 do mesmo apêndice.

5 RESULTADOS

Este capítulo apresenta os resultados da execução do protocolo descrito no Capítulo 3 sobre o *dataset* do Apêndice A. Todos os valores são produzidos pelo *pipeline* de avaliação (Seção 4.4.5) a partir dos registros de execução, com a pontuação recalculada contra o gabarito vigente; cada rodada carrega um manifesto com a *tag* e o *digest* do modelo, as versões de *software*, o *hardware*, a data de referência e os *hashes* do *dataset* e da definição da ferramenta.

5.1 Configuração Experimental

Os resultados vêm de três conjuntos de rodadas, descritos na Seção 3.5.2: as **rodadas completas** dos quatro modelos locais, com três repetições, executadas com Ollama numa GPU de nuvem NVIDIA T4; a **validação na estação de referência**, com as 62 consultas das camadas P e N; e a **referência em nuvem**, com dois modelos de maior porte servidos pelo Groq, com uma repetição. As Tabelas 8, 13 e 19 transcrevem dos manifestos o ambiente de cada rodada. Nessas rodadas, a camada G foi executada a partir de uma versão anterior do gerador, cujas consultas diferem em redação das do *dataset* final. As métricas das rodadas completas cobrem, por isso, as 75 consultas cujo texto coincide com o do *dataset* final — todas as 62 das camadas P e N e 13 das 248 da camada G —, das quais 71 entram nas métricas principais.

Tabela 8 – Ambiente de execução de cada rodada

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
Qwen 3 4B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	225	—	—
Gemma 4 E4B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	225	—	—
Gemma 4 E2B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	225	—	—
Mistral Nemo 12B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	225	—	—

Fonte: Elaborado pelo autor a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-14. Python 3.13.15; langchain-core 1.6.1, langchain-ollama 1.1.0.

Cada modelo foi avaliado com temperatura zero, limite de 1024 *tokens* de saída, sem contexto de conversações anteriores (*zero-shot*), sem exemplos no *prompt* e sem dicionário de normalização, com o mesmo *prompt* de sistema e a mesma definição da ferramenta `buscar_catalogo`; o modo de raciocínio foi desativado nos modelos locais que o oferecem (Seção 3.5.3). As *tags* exatas são `qwen3:4b-instruct-2507-q4_K_M`, `gemma4:e4b-it-qat`, `gemma4:e2b-it-qat` e `mistral-nemo:12b`. Nas rodadas completas, as métricas principais cobrem 71 consultas, pontuadas em cada uma das três repetições —

213 execuções por modelo —, incluindo as da categoria F, em que a resposta correta é não chamar a ferramenta; as consultas observacionais são reportadas à parte (Seção 5.5.1). Nenhuma execução das rodadas completas terminou em erro de infraestrutura.

5.2 Comparativo Geral entre Modelos

A Tabela 9 sumariza as métricas principais por modelo, e a Tabela 10 detalha a distribuição da latência da tradução.

Tabela 9 – Comparativo de desempenho entre os modelos avaliados

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
Qwen 3 4B	52,1%	45,4%–58,7%	0,725	0,756	0,716	993
Gemma 4 E4B	55,4%	48,7%–61,9%	0,866	0,648	0,713	820
Gemma 4 E2B	50,7%	44,0%–57,3%	0,801	0,730	0,722	427
Mistral Nemo 12B	22,5%	17,4%–28,6%	0,775	0,295	0,406	3191

Fonte: Elaborado pelo autor. IC 95%: intervalo de confiança de Wilson para a acurácia. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 10 – Latência da tradução, em milissegundos, por modelo

Modelo	n	Mín.	Mediana	Média	p95	Máx.
Qwen 3 4B	213	64	993	1119	2398	4059
Gemma 4 E4B	213	420	820	1061	3504	4936
Gemma 4 E2B	213	237	427	496	956	1284
Mistral Nemo 12B	213	387	3191	3912	9662	20639

Fonte: Elaborado pelo autor. Tempo da chamada de tradução, medido no ambiente de cada rodada (Tabela 8); execuções com erro de infraestrutura excluídas.

Qualidade.

Três dos quatro modelos ficaram num mesmo patamar de acurácia: 55,4% para o Gemma 4 E4B, 52,1% para o Qwen 3 4B e 50,7% para o Gemma 4 E2B, com intervalos de confiança que se sobrepõem amplamente. Pelo teste de McNemar (Seção 5.8), nenhuma das diferenças entre esses três é estatisticamente significativa. O Mistral Nemo 12B, o maior dos quatro, ficou muito abaixo, com 22,5% (IC 95%: 17,4% a 28,6%), e a diferença para cada um dos outros três é significativa. O tamanho do modelo, portanto, não explica o desempenho na tarefa: a Seção 5.5 mostra que a queda do Mistral Nemo decorre quase inteiramente de um modo de falha no uso da ferramenta.

Precisão e *recall*.

Os três modelos do patamar superior têm F1 ponderado semelhante (0,716, 0,713 e 0,722), mas chegam a ele por caminhos diferentes. O Gemma 4 E4B tem a maior precisão

(0,866) e o menor *recall* (0,648) dos três: quando chama a ferramenta, raramente preenche um campo errado, mas deixa de chamá-la com frequência. O Qwen 3 4B tem o maior *recall* (0,756) e a menor precisão (0,725), porque acrescenta com mais frequência campos que a consulta não pediu.

Latência.

Na T4, o Gemma 4 E2B foi o mais rápido, com mediana de 0,4 s e percentil 95 de 1,0 s; o Gemma 4 E4B e o Qwen 3 4B tiveram medianas de 0,8 s e 1,0 s; e o Mistral Nemo 12B, 3,2 s, cerca de 7,5 vezes a mediana do Gemma 4 E2B. A cauda do Gemma 4 E4B é mais longa que a do Qwen (percentil 95 de 3,5 s, contra 2,4 s), como mostra a Figura 6. Esses tempos valem para uma GPU com VRAM suficiente para cada modelo inteiro; na estação de referência, com 4 GB, a latência é maior e depende da parcela do modelo que cabe na GPU (Seção 5.6).

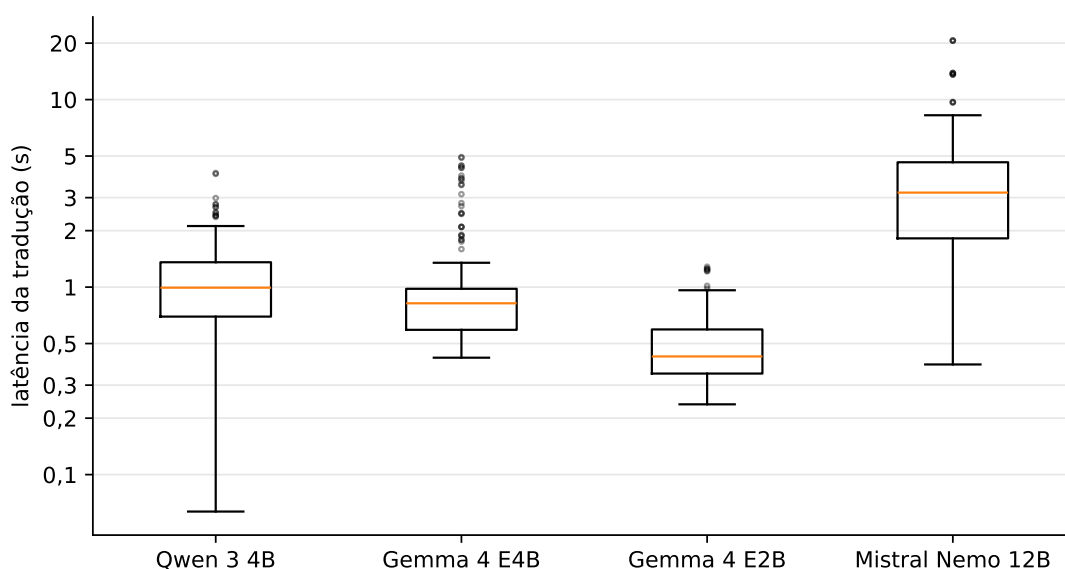


Figura 6 – Distribuição da latência da tradução nas rodadas completas, em escala logarítmica, por modelo

Fonte: Elaborado pelo autor.

Estabilidade entre repetições.

Numa mesma sessão da T4, as três repetições produziram exatamente a mesma resposta em 71 das 71 consultas para o Qwen 3 4B, em 71 para o Gemma 4 E2B, em 71 para o Mistral Nemo 12B e em 67 para o Gemma 4 E4B. Com temperatura zero e o mesmo ambiente, a variação entre repetições foi, portanto, pequena; a Seção 5.6 mostra que ela é maior entre sessões diferentes na estação de referência.

5.3 Análise por Categoria de Consulta

A Tabela 11 apresenta o F1 ponderado por categoria, e a Figura 7, a acurácia por consulta. A categoria F aparece só na figura: como o gabarito dessas consultas não tem campos, a acurácia nela é a taxa de recusa correta.

Tabela 11 – F1-score por categoria de consulta e por modelo

Categoria	Qwen 3 4B	Gemma 4 E4B	Gemma 4 E2B	Mistral Nemo 12B
Simples (n=66)	0,798	0,903	0,770	0,442
Compostas (n=132)	0,705	0,690	0,721	0,392
Código MI/INOM (n=45)	0,872	0,721	0,910	0,232
Tempo relativo (n=45)	0,718	0,530	0,714	0,511
Ordenação (n=30)	0,740	0,510	0,566	0,464
Ambíguas/informais (n=123)	0,642	0,687	0,654	0,391
Média ponderada	0,716	0,713	0,722	0,406

Fonte: Elaborado pelo autor. Uma consulta pode pertencer a mais de uma categoria; n = consultas da categoria nas métricas principais. As consultas fora do domínio (categoria F), cujo gabarito é não chamar a ferramenta, não têm campos e aparecem na Figura de acurácia por categoria.

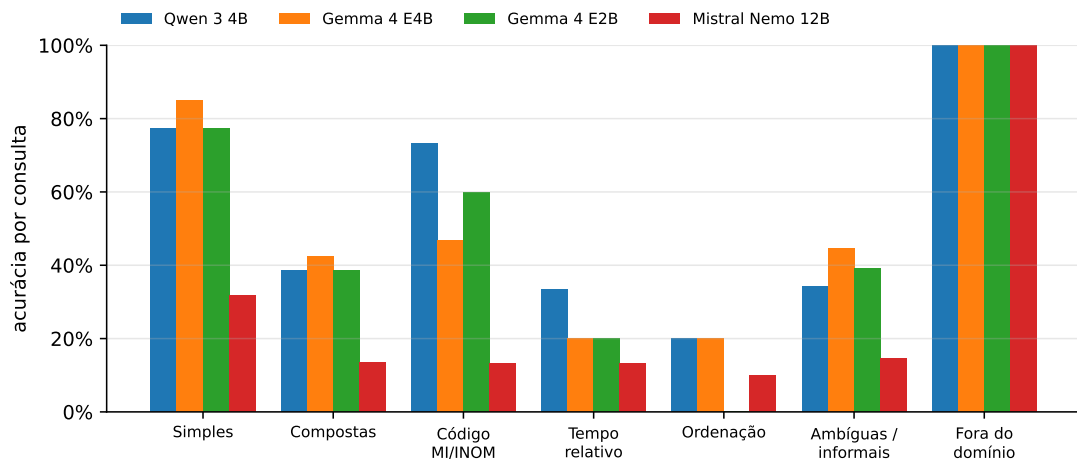


Figura 7 – Acurácia por consulta, por categoria e por modelo, nas rodadas completas

Fonte: Elaborado pelo autor.

O que é fácil.

As consultas simples, com um único parâmetro, tiveram as maiores acurácias entre as categorias com campos: 84,8% no Gemma 4 E4B e 77,3% no Qwen 3 4B e no Gemma 4 E2B. Os quatro modelos recusaram corretamente as consultas fora do domínio — a acurácia na categoria F foi de 100,0%, 100,0%, 100,0% e 100,0% para Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B —, o que confirma que o *Tool Calling* permite ao modelo decidir não buscar, algo que a Saída Estruturada do protótipo não permitia.

O que é difícil para todos.

As categorias de tempo relativo e de ordenação foram as mais difíceis para os quatro modelos. No tempo relativo, a melhor acurácia foi a do Qwen 3 4B, 33,3%; na ordenação, nenhum modelo passou de 20,0%, e o Gemma 4 E2B não acertou nenhuma consulta dessa categoria. As consultas compostas, que exigem vários campos corretos ao mesmo tempo, ficaram em torno de 38,6% a 42,4% nos três melhores modelos. Essas quedas simultâneas indicam dificuldades da tarefa, e não de um modelo específico: resolver uma expressão temporal em datas ISO e decidir se, e por qual data, ordenar são as etapas em que a tradução mais depende de raciocínio além do reconhecimento de nomes.

Onde os modelos divergem.

Nas consultas com código MI ou INOM, o Qwen 3 4B foi o mais consistente (73,3%, contra 60,0% do Gemma 4 E2B e 46,7% do Gemma 4 E4B). Nas consultas ambíguas e informais, o Gemma 4 E4B teve a maior acurácia (44,7%).

5.4 Análise por Campo do *Schema*

Tabela 12 – F1-score por campo do *schema* de busca, por modelo

Campo	Ocorr.	Qwen 3 4B	Gemma 4 E4B	Gemma 4 E2B	Mistral Nemo 12B
keyword	60	0,895	0,792	0,739	0,333
scale	51	0,640	0,771	0,828	0,455
productType	42	0,774	0,734	0,815	0,636
state	57	0,757	0,871	0,765	0,500
city	15	0,750	1,000	0,923	0,500
supplyArea	51	0,938	0,851	1,000	0,300
project	15	0,833	0,909	0,800	0,000
publicationPeriod	42	0,593	0,353	0,600	0,526
creationPeriod	6	0,000	0,667	0,667	0,000
sortField	30	0,500	0,462	0,182	0,353
sortDirection	30	0,500	0,462	0,462	0,133
limit	18	0,556	0,571	0,571	0,600

Fonte: Elaborado pelo autor. Ocorr. = vezes que o campo aparece no gabarito.

Os campos de identificação — **supplyArea**, **city**, **project** e **keyword** — tiveram, em geral, os maiores F1 nos três melhores modelos: o Gemma 4 E2B acertou todas as ocorrências de **supplyArea** (F1 de 1,000), e o Qwen 3 4B teve o maior F1 em **keyword** (0,895), coerente com seu desempenho nas consultas com código MI/INOM. São campos em que a tarefa se reduz a reconhecer e normalizar um nome, justamente o que o dicionário fixo do protótipo fazia; aqui, a normalização foi feita pelo próprio modelo, a partir das descrições da ferramenta.

Os campos de período e de ordenação concentraram as perdas. O F1 de **publicationPeriod** ficou entre 0,353 (Gemma 4 E4B) e 0,600 (Gemma 4 E2B), e os de

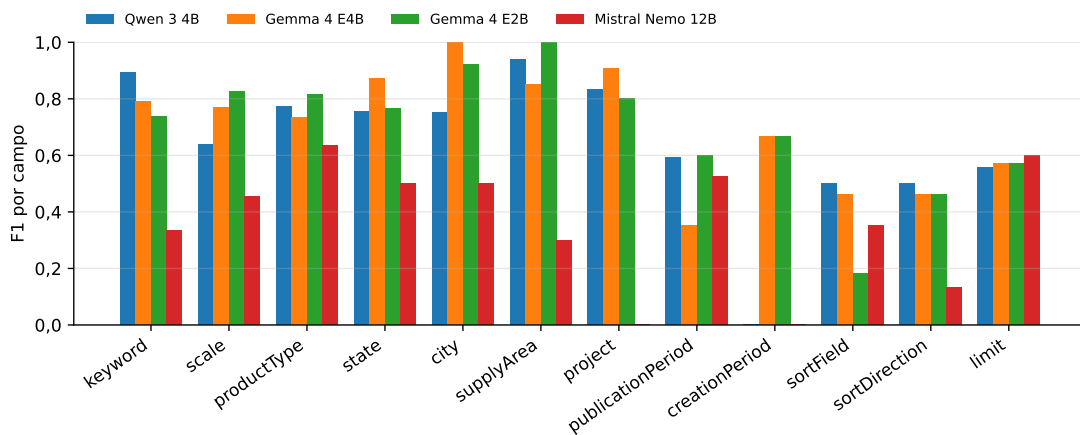


Figura 8 – F1-score por campo do *schema* de busca, por modelo, nas rodadas completas

Fonte: Elaborado pelo autor.

`sortField` e `sortDirection` não passaram de 0,500 em nenhum dos quatro modelos. O campo `creationPeriod` tem poucas ocorrências nas consultas pontuadas, o que torna seu F1 instável.

5.5 Análise de Erros

A classificação dos erros (Seção 3.5.4) mostra um modo de falha dominante diferente em cada modelo. Os exemplos citados são da primeira repetição e estão, com os demais, no arquivo `results/consolidado/erros_representativos.md` do repositório.

Mistral Nemo 12B: a chamada descrita em texto.

O Mistral Nemo 12B deixou de chamar a ferramenta em 141 das 213 execuções pontuadas. Em 84 delas, a resposta cita a ferramenta `buscar_catalogo` e descreve, em texto corrido, a chamada que faria. Para “preciso da carta MI 2965-2-NE do segundo cgeo”, por exemplo, o modelo respondeu que chamaria a ferramenta com `keyword` “2965-2-NE” e `supplyArea` “2º Centro de Geoinformação”, mas não emitiu a chamada estruturada. Em outras 30, pediu mais informações ao usuário. Nesses casos, o modelo identifica a ferramenta e os parâmetros, mas não emite a chamada no canal de *tool calls* da integração com o Ollama; como o protocolo não interpreta texto livre como chamada, cada um deles conta como falha.

Gemma 4 E4B e E2B: o pedido de esclarecimento.

Os dois modelos Gemma deixaram de chamar a ferramenta em 49 (E4B) e 27 (E2B) execuções, quase sempre pedindo ao usuário que especificasse melhor a busca —

41 e 27 casos, respectivamente. Isso ocorreu sobretudo em consultas amplas, com um único critério: “cartas topográficas”, “produtos em escala 1:50.000”, “modelos digitais de terreno”. O Gemma 4 E2B cometeu ainda um erro próprio, o campo acrescentado mais frequente nas suas respostas: copiar para **keyword** palavras comuns da consulta, como “mdt” em “mdt do para em 25k” ou “mapeamento detalhado” em “mapeamento detalhado (25k ou 50k) de brasil”.

Qwen 3 4B: a recusa indevida e os campos acrescentados.

O Qwen 3 4B chamou a ferramenta com mais regularidade, mas declarou fora do escopo 15 execuções de consultas legítimas, como “primeiro mapeamento feito em cuiaba” e “ultima atualizacao do para”. Seu erro mais frequente, porém, foi acrescentar campos não pedidos, sobretudo ordenação e limite (**sortField**, **sortDirection** e **limit** são os campos mais frequentes entre os seus falsos positivos): para “todas as cartas em escala maior que 100k do rio grande do sul”, emitiu ordenação decrescente por data de publicação e limite de 100 resultados, e omitiu a escala.

Datas relativas.

Os erros em períodos seguem padrões comuns aos modelos: tratar “esse ano” como começando na data de referência, e não em 1º de janeiro; reduzir “2 anos atrás” a um único dia; e trocar o trimestre-calendário por três meses corridos, ou pelo trimestre errado, em “trimestre passado” e “último trimestre do ano passado”. A Tabela 12 mostra o efeito desses erros no F1 dos campos de período.

Os tipos de erro têm, assim, remédios diferentes. A chamada descrita em texto do Mistral Nemo é uma limitação da integração do modelo com o canal de ferramentas. O pedido de esclarecimento dos modelos Gemma e a recusa indevida do Qwen são decisões de chamar ou não, sensíveis à redação do *prompt*. Os erros de período e de ordenação são erros de raciocínio sobre a consulta. Nenhum deles decorre de valores fora do *schema*: não houve, em nenhum modelo, argumento com campo inexistente ou tipo inválido.

5.5.1 Casos observacionais

As quatro consultas observacionais testam o comportamento diante de pedidos que a ferramenta não consegue representar. O Qwen 3 4B foi o único que chamou a ferramenta nas duas consultas com valor impossível, em 6 das 12 execuções observacionais: para “escala 1:10.000.000”, emitiu a escala 1:10.000, que existe no enumerado mas não corresponde ao pedido; para “cartas do estado 42”, emitiu **state** “Rio Grande do Sul”, um valor sem nenhum apoio no texto. Os demais modelos não chamaram a ferramenta

nesses dois casos. Nas consultas de resposta discutível — “cartas topo no oceano atlântico” e “cartas do futuro (ano 2050)” —, os modelos em geral também não chamaram a ferramenta; o Gemma 4 E2B buscou cartas topográficas sem restrição de área na primeira delas.

5.6 Validação na Estação de Referência

As 62 consultas das camadas P e N foram executadas na estação de referência pelos dois modelos de menor porte, Qwen 3 4B e Gemma 4 E2B, com uma repetição e a mesma data de referência das rodadas completas. As duas rodadas foram feitas em sequência, na mesma sessão. Antes de cada uma, o *pipeline* descarregou do Ollama todos os modelos residentes, de modo que cada modelo foi carregado a frio, com a VRAM que a estação tinha livre (Tabela 13). Com 4 GB de VRAM, nenhum dos dois coube inteiro na GPU: o Ollama alocou na VRAM 66% do Qwen 3 4B carregado, contando pesos e contexto, e 38% do Gemma 4 E2B; o restante executou na CPU.

Tabela 13 – Ambiente de execução de cada rodada — estação de referência (camadas P e N)

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
Qwen 3 4B	RTX 3050 Laptop, 4096 MiB	Ollama 0.34.0	1	62	66%	3,0 GB
Gemma 4 E2B	RTX 3050 Laptop, 4096 MiB	Ollama 0.34.0	1	62	38%	3,1 GB

Fonte: Elaborado pelo autor a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-14. Python 3.14.4; langchain-core 1.6.4, langchain-ollama 1.1.0.

Tabela 14 – Comparativo de desempenho entre os modelos avaliados — estação de referência (camadas P e N)

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
Qwen 3 4B	49,2%	36,8%–61,6%	0,741	0,762	0,733	2596
Gemma 4 E2B	47,5%	35,3%–60,0%	0,795	0,722	0,719	709

Fonte: Elaborado pelo autor. IC 95%: intervalo de confiança de Wilson para a acurácia. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 15 – Latência da tradução, em milissegundos, por modelo — estação de referência (camadas P e N)

Modelo	n	Mín.	Mediana	Média	p95	Máx.
Qwen 3 4B	59	1251	2596	2857	5702	8098
Gemma 4 E2B	59	372	709	777	1198	1439

Fonte: Elaborado pelo autor. Tempo da chamada de tradução, medido no ambiente de cada rodada (Tabela 13); execuções com erro de infraestrutura excluídas.

Qualidade.

Nas 59 consultas pontuadas, os dois modelos tiveram acurácias estatisticamente indistinguíveis: 49,2% para o Qwen 3 4B (IC 95%: 36,8% a 61,6%) e 47,5% para o Gemma 4 E2B (IC 95%: 35,3% a 60,0%). Entre as consultas acertadas por apenas um deles, 8 foram do Qwen e 7 do Gemma, com p -valor de 1,000 no teste de McNemar exato (Tabela 16).

Tabela 16 – Teste de McNemar exato entre pares de modelos — estação de referência (camadas P e N)

Modelo A	Modelo B	Só A acerta	Só B acerta	p -valor
Qwen 3 4B	Gemma 4 E2B	8	7	1,0000

Fonte: Elaborado pelo autor. Pares discordantes sobre as mesmas consultas e repetições das métricas principais; p -valor bicaudal exato.

Latência.

A latência, ao contrário, separou nitidamente os dois modelos. O Qwen 3 4B teve mediana de 2,6 s e percentil 95 de 5,7 s; o Gemma 4 E2B, 0,7 s e 1,2 s, cerca de 3,7 vezes mais rápido, apesar da menor parcela na GPU. Dois fatores são compatíveis com essa diferença: o Gemma 4 E2B computa com cerca de 2 bilhões de parâmetros efetivos (Seção 3.2), e suas respostas foram mais curtas, com mediana de 35 *tokens* de saída, contra 50 do Qwen 3 4B. Em relação à T4, a latência mediana do Qwen 3 4B subiu de 1,0 s para 2,6 s, e a do Gemma 4 E2B, de 0,4 s para 0,7 s.

Comportamento.

Na categoria F, a acurácia foi de 100,0% para o Qwen 3 4B e de 100,0% para o Gemma 4 E2B: os dois recusaram as consultas fora do domínio das camadas P e N. Os modos de falha foram os mesmos das rodadas completas: o Qwen declarou fora do escopo pedidos como “atualizações deste mês”, e o Gemma pediu mais critérios em consultas como “cartas topográficas” e “produtos publicados nos últimos 3 meses”, contrariando a instrução do *prompt* de sistema de que um único parâmetro basta para chamar a ferramenta.

Comparação com as rodadas completas.

A Tabela 17 compara, para os mesmos modelos e consultas, as respostas da estação com as da primeira repetição das rodadas completas. A resposta foi idêntica em 50 das 59 consultas para o Qwen 3 4B e em 53 para o Gemma 4 E2B, e a pontuação coincidiu em 57 e 57, respectivamente. A acurácia nessas consultas foi de 49,2% e 47,5% nas

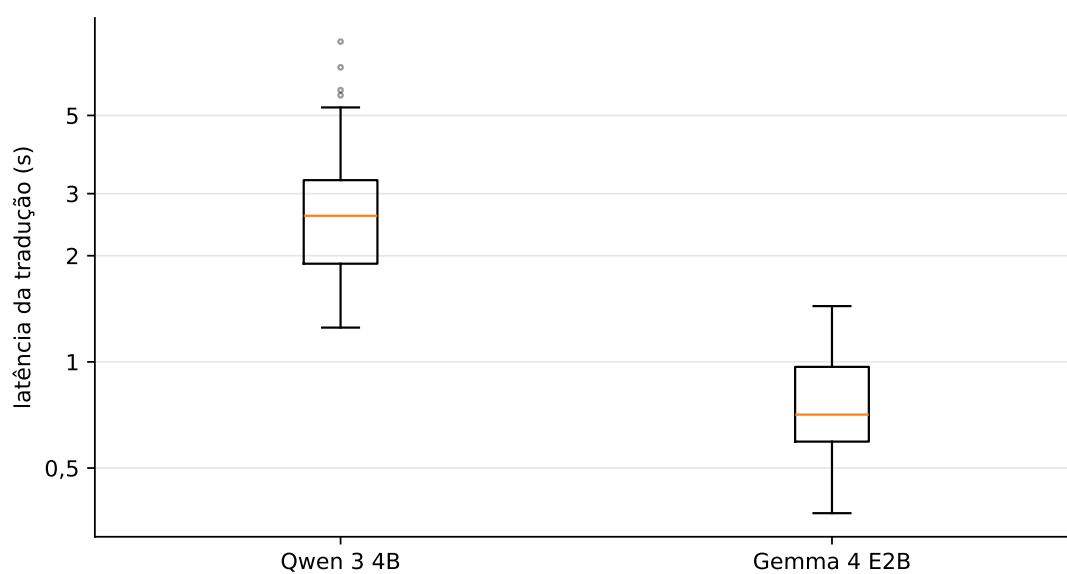


Figura 9 – Distribuição da latência da tradução na estação de referência, em escala logarítmica

Fonte: Elaborado pelo autor.

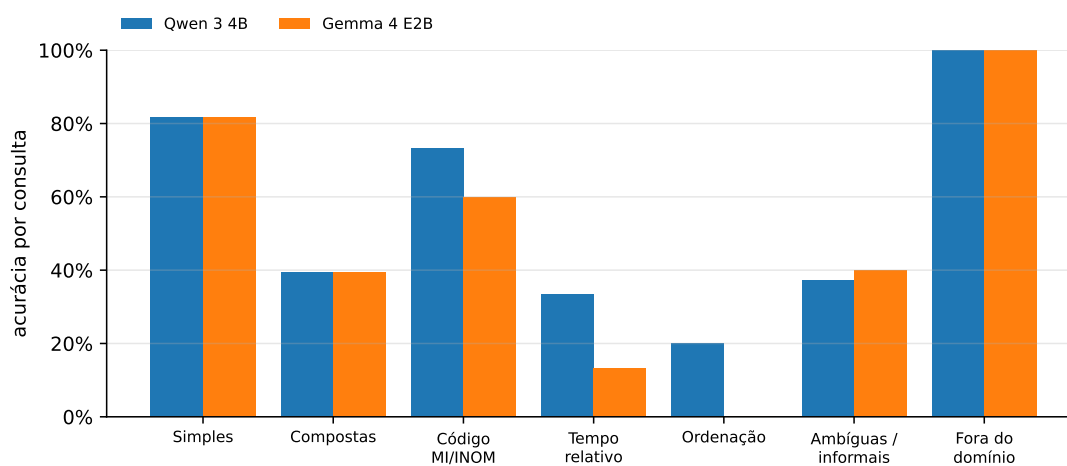


Figura 10 – Acurácia por consulta, por categoria, na estação de referência

Fonte: Elaborado pelo autor.

rodadas completas, e de 49,2% e 47,5% na estação. A qualidade da tradução, portanto, praticamente não mudou com o *hardware*; a latência, sim.

Tabela 17 – Mesmo modelo nas rodadas completas e na estação de referência (camadas P e N, primeira repetição)

Modelo	n	Resposta idêntica	Mesma pontuação	Acurácia (rodada completa)	Acurácia (estação)
Qwen 3 4B	59	50	57	49,2%	49,2%
Gemma 4 E2B	59	53	57	47,5%	47,5%

Fonte: Elaborado pelo autor. Resposta idêntica: mesmos parâmetros emitidos (ou nenhuma chamada nos dois ambientes).

Reexecução na mesma estação.

O Qwen 3 4B já havia executado as mesmas consultas na estação em 14 de setembro de 2026, com os mesmos pesos, *prompt*, ferramenta, data de referência e versão do Ollama. A Tabela 18 compara as duas execuções: mesmo com temperatura zero, a resposta se repetiu em 51 das 59 consultas, e a pontuação, em 56. A explicação mais provável são diferenças numéricas de ponto flutuante na inferência, que dependem da divisão do modelo entre GPU e CPU e da ordem das operações e bastam para mudar a escolha de um *token* em respostas isoladas. A variação é maior entre sessões na estação, com parte do modelo na CPU, do que entre as repetições de uma mesma sessão na T4 (Seção 5.2), e é por causa dela que as rodadas completas usam três repetições e que a acurácia é reportada com intervalo de confiança.

Tabela 18 – Qwen 3 4B executado duas vezes na estação de referência, com os mesmos pesos, *prompt*, ferramenta e data de referência (camadas P e N)

Modelo	n	Resposta idêntica	Mesma pontuação	Acurácia (24/set)	Acurácia (14/set)
Qwen 3 4B	59	51	56	49,2%	47,5%

Fonte: Elaborado pelo autor. Resposta idêntica: mesmos parâmetros emitidos (ou nenhuma chamada nos dois ambientes).

5.7 Referência em Nuvem

Dois modelos de maior porte servidos pelo Groq executaram o mesmo protocolo, com uma repetição: `openai/gpt-oss-20b` e `openai/gpt-oss-120b`. Eles não atendem ao requisito de execução local (RNF1) e servem como referência de quanto modelos maiores, sem restrição de *hardware*, acertam nas mesmas consultas; a latência inclui a rede e não é comparável à dos modelos locais.

Tabela 19 – Ambiente de execução de cada rodada — referência em nuvem (Groq)

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
GPT-OSS 20B (Groq)	servidores do provedor	Groq (API)	1	310	—	—
GPT-OSS 120B (Groq)	servidores do provedor	Groq (API)	1	310	—	—

Fonte: Elaborado pelo autor a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-24. Python 3.14.4; langchain-core 1.6.4, langchain-ollama 1.1.0, langchain-openai 1.6.6.

Tabela 20 – Comparativo de desempenho entre os modelos avaliados — referência em nuvem (Groq)

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
GPT-OSS 20B (Groq)	89,2%	85,2%–92,2%	0,959	0,945	0,946	530
GPT-OSS 120B (Groq)	89,9%	86,0%–92,8%	0,965	0,951	0,956	655

Fonte: Elaborado pelo autor. IC 95%: intervalo de confiança de Wilson para a acurácia. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 21 – F1-score por categoria de consulta e por modelo — referência em nuvem (Groq)

Categoria	GPT-OSS 20B (Groq)	GPT-OSS 120B (Groq)
Simples (n=100)	0,985	0,979
Compostas (n=180)	0,937	0,950
Código MI/INOM (n=57)	0,985	0,986
Tempo relativo (n=60)	0,938	0,933
Ordenação (n=29)	0,837	0,926
Ambíguas/informais (n=197)	0,952	0,951
Média ponderada	0,946	0,956

Fonte: Elaborado pelo autor. Uma consulta pode pertencer a mais de uma categoria; n = consultas da categoria nas métricas principais. As consultas fora do domínio (categoria F), cujo gabarito é não chamar a ferramenta, não têm campos e aparecem na Figura de acurácia por categoria.

Sobre todas as 306 consultas das métricas principais, o GPT-OSS 20B acertou 89,2% (IC 95%: 85,2% a 92,2%) e o GPT-OSS 120B, 89,9% (IC 95%: 86,0% a 92,8%). Restrita às mesmas 71 consultas pontuadas nas rodadas completas dos modelos locais, a acurácia foi de 81,7% e 77,5%, respectivamente, contra 55,4% do melhor modelo local. A distância é maior justamente nas categorias difíceis para os modelos locais: nas consultas compostas e de tempo relativo desse mesmo conjunto, o GPT-OSS 20B acertou 75,0% e 80,0%, contra no máximo 42,4% e 33,3% entre os modelos locais. A ordenação continuou sendo a categoria mais difícil também para eles: sobre todo o *dataset*, o GPT-OSS 20B acertou 48,3% dessas consultas e o GPT-OSS 120B, 79,3%. Como nas rodadas locais, todas as consultas fora do domínio foram recusadas.

A referência mostra que a tarefa, com a mesma ferramenta, o mesmo *prompt* e o mesmo gabarito, é resolvível com alta acurácia por modelos de maior porte. A distância

para os modelos locais mede, assim, a limitação dos modelos que cabem em estações modestas, e não um defeito do protocolo ou do gabarito.

5.8 Análise Estatística

A Tabela 22 traz o teste de McNemar exato entre cada par de modelos das rodadas completas, sobre as mesmas consultas e repetições. Entre os três modelos do patamar superior, nenhuma diferença é significativa: o p -valor foi de 0,464 entre Qwen 3 4B e Gemma 4 E4B, de 0,766 entre Qwen 3 4B e Gemma 4 E2B e de 0,164 entre os dois Gemma. Entre o Mistral Nemo 12B e cada um dos outros três, o p -valor foi inferior a 0,001. Como as três repetições de cada consulta entram como pares, e elas são quase sempre idênticas numa mesma sessão, os testes tratam como independentes observações que não o são inteiramente; os p -valores devem ser lidos como aproximação, o que não altera as conclusões, dada a distância entre os dois grupos de modelos e a proximidade dentro do grupo superior.

Tabela 22 – Teste de McNemar exato entre pares de modelos

Modelo A	Modelo B	Só A acerta	Só B acerta	p -valor
Qwen 3 4B	Gemma 4 E4B	30	37	0,4638
Qwen 3 4B	Gemma 4 E2B	24	21	0,7660
Qwen 3 4B	Mistral Nemo 12B	63	0	0,0000
Gemma 4 E4B	Gemma 4 E2B	26	16	0,1641
Gemma 4 E4B	Mistral Nemo 12B	82	12	0,0000
Gemma 4 E2B	Mistral Nemo 12B	69	9	0,0000

Fonte: Elaborado pelo autor. Pares discordantes sobre as mesmas consultas e repetições das métricas principais; p -valor bicaudal exato.

5.9 Discussão

Qualidade e latência.

Os resultados não apontam um modelo dominante em qualidade entre Qwen 3 4B, Gemma 4 E4B e Gemma 4 E2B: as acurácias são estatisticamente indistinguíveis, e cada um tem pontos fortes próprios — o Qwen nos códigos MI/INOM e no *recall*, o Gemma 4 E4B na precisão e nas consultas informais, o Gemma 4 E2B nos campos de escala, tipo de produto e área de suprimento. A latência os separa com mais clareza: o Gemma 4 E2B foi o mais rápido tanto na T4 quanto na estação de referência, onde sua mediana ficou abaixo de um segundo.

O que o *Tool Calling* entregou.

Em relação à Saída Estruturada do protótipo, três ganhos aparecem diretamente nos dados. O modelo passou a poder recusar: todas as consultas fora do domínio foram recusadas por todos os modelos. A normalização de siglas, grafias sem acento, apelidos de escala e nomes de projeto passou a ser feita pelo modelo, sem dicionário, com F1 elevado nos campos de identificação. E toda falha passou a ser registrada e classificada, o que permitiu identificar o modo de falha de cada modelo — inclusive o do Mistral Nemo 12B, que um mecanismo de *fallback* teria mascarado.

O que continua difícil.

Datas relativas, ordenação e consultas com muitos campos permanecem difíceis para todos os modelos locais, e a ordenação o é também para os modelos em nuvem. Parte da dificuldade é inerente à tarefa: nem sempre a consulta diz se a ordem é por publicação ou por criação, e o gabarito aceita as duas leituras, mas os modelos ainda assim acrescentam ou omitem campos. Outra parte é sensível ao *prompt*: a recusa indevida e o pedido de esclarecimento em consultas amplas contrariam uma instrução explícita e são candidatos naturais a ajuste de *prompt* ou a exemplos no *prompt*, que este trabalho não usou por desenho.

Hardware.

Com 4 GB de VRAM, nenhum modelo coube inteiro na GPU da estação, e a latência depende diretamente da parcela alocada. A qualidade, por outro lado, praticamente não mudou entre a estação e a T4. Para a DSG, isso significa que a escolha do modelo pode ser feita pela qualidade medida em qualquer ambiente, e que o *hardware* de implantação define a latência: uma GPU com 6 a 8 GB de VRAM já comportaria inteiros o Qwen 3 4B e o Gemma 4 E2B.

5.9.1 Recomendação de Modelo para Uso Operacional

Para o uso operacional sobre o acervo da DSG, em estações com GPU modesta, recomenda-se o **Gemma 4 E2B**: sua acurácia é estatisticamente equivalente à dos melhores modelos avaliados, e ele teve a menor latência nos dois ambientes, inclusive na estação de referência de 4 GB. O **Qwen 3 4B** é a alternativa quando as consultas com códigos MI/INOM forem predominantes, categoria em que teve o melhor desempenho, ao custo de uma latência maior na estação. O Gemma 4 E4B, de qualidade equivalente, exige mais VRAM que os outros dois e não foi medido na estação de referência. O Mistral Nemo 12B não é recomendado na configuração avaliada, pela falha sistemática no uso do canal de ferramentas.

A recomendação tem três ressalvas. Ela vale para a configuração avaliada — sem raciocínio, sem exemplos no *prompt* e com as versões registradas nos manifestos —, e o desempenho com raciocínio ativado não foi medido. A distância para os modelos em nuvem indica margem de melhoria considerável, que pode vir de novos modelos abertos: o *pipeline* permite repetir a comparação com o mesmo critério. E os modos de falha identificados, sobretudo o pedido de esclarecimento em consultas amplas, devem ser observados no uso real antes da implantação em larga escala.

6 CONCLUSÃO

6.1 Síntese do Trabalho

Este Projeto Final de Curso aprimorou a camada de tradução linguística da solução de busca em linguagem natural sobre o acervo cartográfico da Diretoria de Serviço Geográfico (DSG), partindo do protótipo desenvolvido pelo 1º Centro de Geoinformação (1º CGEO). O protótipo demonstrou a viabilidade da abordagem, mas apresentava limitações documentadas pelo próprio cliente: *backend* em Node.js, Saída Estruturada via Instructor e Zod, um único modelo (Phi-4 14B), dicionário de normalização fixo no código e ausência de um protocolo de avaliação.

A partir da análise crítica do protótipo, o trabalho entregou:

1. um *backend* em Python, com FastAPI e LangChain, que substitui o do protótipo e preserva o esquema do banco e o conjunto de parâmetros de busca;
2. a substituição da Saída Estruturada por *Tool Calling* nativo, em que o modelo decide se chama a ferramenta de busca e com quais argumentos, sem dicionário de normalização e sem exemplos resolvidos no *prompt*;
3. um *dataset* de avaliação com 310 consultas, construído em três camadas (protótipo, autores e geração programática), com gabarito definido por um manual de anotação escrito, leituras múltiplas explícitas para as consultas ambíguas e uma auditoria em três etapas — checagens automáticas, anotação independente às cegas e adjudicação registrada — que deixou todas as divergências com decisão justificada;
4. um *pipeline* de avaliação automatizado e reproduzível, que registra manifesto e traços de cada rodada, recalcula a pontuação contra o gabarito vigente e produz acurácia com intervalo de confiança, precisão, *recall* e F1 por campo, latência e testes de significância entre modelos;
5. a comparação de quatro modelos abertos executáveis localmente — Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B —, complementada por uma referência paralela com dois modelos de maior porte servidos em nuvem.

6.2 Retomada dos Objetivos

O objetivo geral — aprimorar a camada de tradução linguística por meio do *Tool Calling* nativo, da avaliação comparativa de modelos locais e de métricas quantitativas

— foi atendido pelos entregáveis definidos na Introdução:

- **Nova arquitetura de *backend***: descrita no Capítulo 4, com as camadas de API, orquestração, persistência e avaliação, e com a lógica de consulta ao banco portada do protótipo;
- ***Dataset* de avaliação**: 310 consultas com manual de anotação e registro de auditoria (Capítulo 3 e Apêndice A); a auditoria mostrou concordância quase perfeita entre o gabarito e uma anotação independente feita às cegas, e cada divergência recebeu decisão registrada;
- ***Pipeline* de avaliação**: implementado nos comandos `pfc-dataset`, `pfc-auditar`, `pfc-avaliar` e `pfc-relatorio`, cobertos por testes automatizados (Capítulo 4);
- **Relatório comparativo e recomendação**: apresentados no Capítulo 5, com a recomendação de modelo para uso operacional na Seção 5.9.

6.3 Principais Resultados

Os resultados do Capítulo 5 permitem responder às perguntas que motivaram o trabalho.

- **Qualidade dos modelos locais**: três dos quatro modelos — Gemma 4 E4B, Qwen 3 4B e Gemma 4 E2B — tiveram acurácias estatisticamente indistinguíveis, de 55,4%, 52,1% e 50,7%, com F1 ponderado entre 0,713 e 0,722. O Mistral Nemo 12B, o maior dos quatro, ficou em 22,5%, porque em grande parte das consultas descreveu a chamada da ferramenta em texto, em vez de emití-la; o porte do modelo, portanto, não determinou o desempenho.
- **O que o *Tool Calling* entregou**: todos os modelos recusaram corretamente as consultas fora do domínio, a normalização de nomes, siglas e escalas foi feita sem dicionário, e cada falha foi registrada e classificada, sem *fallback* que a mascarasse.
- **O que continua difícil**: datas relativas, ordenação e consultas com vários campos foram as categorias de menor acurácia para todos os modelos locais. Os modos de falha dominantes foram deixar de chamar a ferramenta em consultas amplas — por recusa indevida, no Qwen 3 4B, ou por pedido de esclarecimento, nos modelos Gemma — e acrescentar ordenação ou limite não pedidos.
- **Hardware**: a qualidade praticamente não mudou entre a estação de referência e a GPU de nuvem; a latência, sim. Na estação de 4 GB, o Gemma 4 E2B teve latência mediana de 0,7 s, contra 2,6 s do Qwen 3 4B, com a mesma acurácia.

- **Referência em nuvem:** nas mesmas consultas, os modelos servidos pelo Groq acertaram 77,5% (GPT-OSS 120B) e 81,7% (GPT-OSS 20B), o que mostra que a tarefa é resolvível com o mesmo protocolo e indica a margem de melhoria disponível para os modelos locais.
- **Recomendação:** para uso operacional em estações com GPU modesta, recomenda-se o Gemma 4 E2B, pela qualidade equivalente à dos melhores modelos e pela menor latência nos dois ambientes; o Qwen 3 4B é a alternativa quando predominarem consultas com códigos MI/INOM.

6.4 Limitações do Trabalho

Os resultados devem ser interpretados à luz das seguintes limitações:

- **Regularidade da camada gerada:** 248 das 310 consultas foram geradas por modelos de frase. A geração garante cobertura sistemática e volume para os testes estatísticos, mas essas consultas são linguisticamente mais regulares que as de usuários reais, o que pode favorecer os modelos; por isso os resultados são reportados também por camada.
- **Consultas não coletadas de usuários:** nenhuma consulta do *dataset* foi coletada do uso real do catálogo; as camadas P e N foram redigidas pela equipe do protótipo e pelos autores.
- **Alcance da auditoria:** a anotação independente às cegas foi feita por um modelo de linguagem de propósito geral, e não por anotadores humanos; ela verifica a consistência do gabarito com o manual, mas não substitui uma anotação dupla por especialistas do domínio.
- **Ambientes de execução:** as rodadas completas foram executadas em GPU de nuvem, e a estação de referência executou as camadas P e N. A latência depende fortemente do *hardware* e é reportada por ambiente; valores obtidos em outras GPUs, com outra quantidade de VRAM, podem diferir.
- **Configuração dos modelos:** todos os modelos foram avaliados sem modo de raciocínio, com temperatura zero e sem exemplos no *prompt*. O desempenho com raciocínio ativado, com *few-shot* ou com *prompts* ajustados a cada modelo não foi medido.
- **Cobertura das rodadas completas:** as métricas das rodadas completas cobrem 75 das 310 consultas do *dataset*, com a camada G representada por 13 consultas; os resultados por categoria e por campo dessas rodadas refletem, sobretudo, as consultas redigidas por pessoas (camadas P e N).

- **Referência em nuvem:** os modelos do Groq executaram uma repetição, sob os limites de taxa do plano gratuito, e servem apenas como referência de porte; não atendem ao requisito de execução local.
- **Versões dos modelos:** os resultados valem para as versões registradas nos manifestos; o ecossistema de LLMs abertos evolui rapidamente, e o *pipeline* foi construído para que a avaliação possa ser refeita com novos modelos.

6.5 Trabalhos Futuros

Os resultados e a experiência adquirida apontam as seguintes direções. As três primeiras decorrem diretamente das exclusões de escopo declaradas no Capítulo 3.

1. **Avaliação da recuperação em nível de produto:** medir a fidelidade do conjunto de produtos retornado pela busca (precisão, *recall* e F1 sobre os códigos das folhas), o que exige um gabarito de quais folhas cada consulta deveria devolver.
2. **Resolução avançada de nomes geográficos:** resolver os campos `state` e `city` em geometrias, para operações espaciais (`ST_Intersects`, `ST_Within`) contra as *bounding boxes* das folhas, com integração à malha do IBGE e a serviços de geocodificação.
3. **Fallback para produção:** neste trabalho, falhas do modelo são registradas como métrica, sem recuperação por regras determinísticas — decisão adequada à avaliação. Em produção, convém um caminho alternativo, como encaminhar a consulta bruta à busca textual do PostgreSQL.
4. **Ferramentas auxiliares:** acrescentar ferramentas de validação de códigos INOM e MI e de listagem dos tipos de produto, que o *Tool Calling* permite oferecer ao modelo sem alterar a arquitetura.
5. **Modos de raciocínio e prompts:** avaliar os modelos com raciocínio ativado e com exemplos no *prompt*, medindo o ganho de qualidade contra o custo de latência.
6. **Dataset com consultas reais:** ampliar o *dataset* com consultas coletadas de usuários do catálogo, anotadas por especialistas do domínio segundo o manual de anotação, com medida de concordância entre anotadores humanos.
7. **Interface e implantação:** desenvolver a interface gráfica, com mapa interativo, e integrar o *pipeline* ao sistema de produção da DSG, com autenticação, controle de acesso e monitoramento.

8. **Ajuste fino:** treinar por ajuste fino um modelo de menor porte para o domínio das consultas cartográficas brasileiras, o que pode reduzir o requisito de *hardware*.
9. **Estudo com usuários:** avaliar a solução com usuários reais do catálogo, medindo impacto na produtividade e na satisfação.
10. **Conversação em múltiplos turnos:** permitir que o usuário refine a busca a partir dos resultados, mantendo o contexto das consultas anteriores.

6.6 Considerações Finais

O trabalho mostra que a busca em linguagem natural sobre o acervo cartográfico da DSG pode ser construída com modelos de linguagem abertos, executados localmente e integrados por *Tool Calling*, sem dependência de serviços de nuvem e com o controle sobre os dados que o contexto da DSG e de suas organizações subordinadas exige.

Para o cliente, o resultado mais duradouro não é a classificação dos modelos avaliados, que envelhece com o ecossistema, mas o instrumento de avaliação: um *dataset* auditável, com gabarito justificado por regras escritas e divergências adjudicadas, e um *pipeline* reproduzível que permite comparar qualquer novo modelo, *prompt* ou configuração com o mesmo critério.

REFERÊNCIAS

- 1º Centro de Geoinformação. *Protótipo de busca de geoinformação utilizando Large Language Models*. 2025. <https://github.com/1cgeo/prototipo_busca_llm>. Repositório GitHub. Acesso em: mar. 2026.
- ARTSTEIN, R.; POESIO, M. Inter-coder agreement for computational linguistics. *Computational Linguistics*, v. 34, n. 4, p. 555–596, 2008.
- Brasil. *Decreto n. 6.666, de 27 de novembro de 2008: Institui, no âmbito do Poder Executivo federal, a Infra-Estrutura Nacional de Dados Espaciais – INDE*. [S.l.], 2008. Disponível em: <https://www.planalto.gov.br/ccivil_03/_Ato2007-2010/2008/Decreto/D6666.htm>.
- Brasil. Comissão Nacional de Cartografia (CONCAR). *Perfil de Metadados Geoespaciais do Brasil (Perfil MGB)*. [S.l.], 2009. Disponível em: <<https://www.gov.br/inde/pt-br/composicao/perfil-mgb>>.
- Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico. *Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG)*. [S.l.], 2016. 2.ª edição. Integra o arcabouço de normas técnicas do SCN e da INDE (junto com ET-EDGV, ET-RDG, ET-ADGV e ET-CQDG). Disponível em: <<https://geoportal.eb.mil.br/portal/index.php/et-edgv/et-pcdg>>.
- Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico. *Normas para Cartografia Sistemática Terrestre e Publicação para Registro Público*. [S.l.], 2016. Normas técnicas em vigor no Sistema Cartográfico Nacional (SCN). Estabelecidas pelo Decreto n.º 89.817/1984 e mantidas/atualizadas pela DSG. Disponível em: <https://bdgex.eb.mil.br/portal/index.php?option=com_content&view=article&id=83&Itemid=355&lang=pt>.
- BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. D.; DHARIWAL, P.; NEELAKANTAN, A.; SHYAM, P.; SASTRY, G.; ASKELL, A. et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, v. 33, p. 1877–1901, 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>.
- CHASE, H. *LangChain: Building applications with LLMs through composability*. 2022. <<https://github.com/langchain-ai/langchain>>. Acesso em: mar. 2026.
- COHEN, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, v. 20, n. 1, p. 37–46, 1960.
- COLVIN, S. *Pydantic: Data validation using Python type hints*. 2019. <<https://docs.pydantic.dev>>. Acesso em: mar. 2026.
- Diretoria de Serviço Geográfico do Exército Brasileiro. *Banco de Dados Geográfico do Exército — BDGEx: o nó do Exército na INDE*. 2024. <<https://www.geoportal.eb.mil.br/portal/bdgex-1/bdgex-generalidades>>. Acesso em: maio 2026.

Diretoria de Serviço Geográfico do Exército Brasileiro. *Missão da Diretoria de Serviço Geográfico*. 2024. <<https://geoportal.eb.mil.br/portal/index.php/chefia/missao>>. Acesso em: maio 2026.

FRANTAR, E.; ASHKBOOS, S.; HOEFLER, T.; ALISTARH, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. Disponível em: <<https://arxiv.org/abs/2210.17323>>.

Gemma Team. *Gemma 4: byte for byte, the most capable open models*. 2026. <<https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>>. Google DeepMind. Acesso em: maio 2026.

JIANG, A. Q.; SABLAYROLLES, A.; MENSCH, A.; BAMFORD, C.; CHAPLOT, D. S.; CASAS, D. de las; BRESSAND, F.; LENGUEL, G.; LAMPLE, G.; SAULNIER, L. et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023. Disponível em: <<https://arxiv.org/abs/2310.06825>>.

LangChain AI. *LangGraph: Building stateful, multi-actor applications with LLMs*. 2024. <<https://github.com/langchain-ai/langgraph>>. Acesso em: mar. 2026.

LIU, J. *Instructor: Structured outputs for LLMs*. 2023. <<https://python.useinstructor.com>>. Acesso em: mar. 2026.

llama.cpp contributors. *GGUF: GPT-Generated Unified Format*. 2023. <<https://github.com/ggerranov/llama.cpp>>. Acesso em: mar. 2026.

MAI, G.; HUANG, W.; SUN, J.; SONG, S.; MISHRA, D.; LIU, N.; GAO, S.; LIU, T.; CONG, G.; HU, Y. et al. On the opportunities and challenges of foundation models for geospatial artificial intelligence. *arXiv preprint arXiv:2304.06798*, 2023. Disponível em: <<https://arxiv.org/abs/2304.06798>>.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Disponível em: <<https://nlp.stanford.edu/IR-book/>>.

MCNEMAR, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, v. 12, n. 2, p. 153–157, 1947.

Ollama Team. *Ollama: Get up and running with large language models locally*. 2023. <<https://ollama.com>>. Acesso em: mar. 2026.

Open Geospatial Consortium. *OGC Catalogue Services Standard 2.0.2*. 2007. <<https://www.ogc.org/standard/cat/>>. OGC 07-006r1. Acesso em: mar. 2026.

PostGIS Development Team. *PostGIS: Spatial and geographic objects for PostgreSQL*. 2001. <<https://postgis.net>>. Acesso em: mar. 2026.

QIN, Y.; LIANG, S.; YE, Y.; ZHU, K.; YAN, L.; LU, Y.; LIN, Y.; CONG, X.; TANG, X.; QIAN, B. et al. ToolLLM: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023. Disponível em: <<https://arxiv.org/abs/2307.16789>>.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Alibaba Cloud. Coleção de LLMs de código aberto (0,6B a 235B parâmetros), incluindo modelos densos e MoE. Disponível em: <<https://arxiv.org/abs/2505.09388>>.

RAMÍREZ, S. *FastAPI: Modern, fast web framework for building APIs with Python*. 2018. <<https://fastapi.tiangolo.com>>. Acesso em: mar. 2026.

SCHICK, T.; DWIVEDI-YU, J.; DESSI, R.; RAILEANU, R.; LOMELI, M.; ZETTLEMOYER, L.; CANCEDDA, N.; SCIALOM, T. Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems*. [s.n.], 2023. v. 36. Disponível em: <<https://arxiv.org/abs/2302.04761>>.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>.

WEI, J.; TAY, Y.; BOMMASANI, R.; RAFFEL, C.; ZOPH, B.; BORGEAUD, S.; YOGATAMA, D.; BOSMA, M.; ZHOU, D.; METZLER, D. et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. Disponível em: <<https://arxiv.org/abs/2206.07682>>.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, v. 22, n. 158, p. 209–212, 1927.

APÊNDICE A – DATASET DE AVALIAÇÃO

Este apêndice documenta integralmente o *dataset* de avaliação descrito na Seção 3.3: o manual de anotação que define o gabarito, as 310 consultas com as respectivas leituras aceitas, o mapeamento das consultas aos requisitos funcionais e o registro da auditoria. As mesmas informações estão, em formato legível por máquina, no arquivo `data/dataset.json` do repositório (Apêndice C). As listagens deste apêndice são geradas a partir desse arquivo pelo comando `pfc-dataset -apendice`, sem transcrição manual.

Como ler as entradas.

Cada consulta traz o identificador, as categorias (Seção 3.3), o texto exatamente como é enviado ao modelo e as **leituras aceitas** do gabarito, na forma canônica da ferramenta. A notação segue o princípio P3 do manual: “A ou B” indica valor alternativo; “(opcional)” indica campo que pode faltar sem erro, mas que, se presente, precisa ter o valor indicado; “aceita-se também *city* no lugar de *state*” indica leitura estrutural alternativa. Expressões de tempo relativo aparecem pelo nome da regra (“ano corrente”, “semana passada”) e são resolvidas na data de referência da execução, segundo o Quadro 5. Nas camadas P e N, uma nota registra a linha de origem no arquivo do protótipo e a justificativa das leituras alternativas.

Escopo do gabarito.

O gabarito descreve os **parâmetros** que a consulta deve produzir, e não o conjunto de produtos que a busca resultante devolveria. A avaliação de recuperação em nível de produto está fora do escopo do trabalho (Seção 3.1.2).

A.1 Manual de anotação

O gabarito de cada consulta foi construído segundo as regras abaixo. Elas são a referência para a redação e a revisão das consultas, para a auditoria automática e para a anotação independente descritas na Seção A.14; qualquer anotação que as contrarie é considerada erro do gabarito, e não do modelo.

Princípios

P1 — Evidência explícita. O gabarito contém somente parâmetros para os quais existe um trecho da consulta que os justifica. Nada é inferido além do que o texto

diz (“cartas de Manaus” não implica `state = "Amazonas"`).

- P2 — Forma canônica.** Todo valor é escrito na forma canônica da ferramenta `buscar_catalogo`: enumerados exatamente como no *schema*; estados e municípios por extenso, com acentos; datas em ISO 8601.
- P3 — Leituras múltiplas explícitas.** Quando a consulta admite mais de uma leitura razoável e nem o texto nem a descrição da ferramenta desempatam, o gabarito lista todas as leituras aceitas, sendo a primeira a preferencial. Há três mecanismos: *valor alternativo* (o campo aceita mais de um valor), *campo opcional* (pode faltar; se aparecer, precisa ter o valor indicado) e *leitura estrutural alternativa* (o mesmo trecho pode ocupar campos diferentes, como estado ou município).
- P4 — Fora do domínio.** Consultas sem intenção de busca no acervo cartográfico brasileiro têm como gabarito “não chamar a ferramenta”.
- P5 — Consistência.** A mesma expressão recebe a mesma anotação em todo o *dataset*.
- P6 — Casos observacionais.** Consultas cujo comportamento correto não é determinável a partir do texto (valor impossível de representar, comportamento esperado discutível) são executadas e reportadas, mas ficam fora das métricas principais. Consultas fora do domínio (P4) não são observacionais, pois o comportamento correto é determinado.

Regras por campo

keyword Código MI com seus sufixos, sem o prefixo “MI”; código INOM na grafia com hífen (grafias compactadas, como “sf22yd”, são normalizadas); nome próprio de carta quando precedido de “carta” ou “folha”. Se o nome da carta coincide com um município, aceita-se também `city`. Havendo dois códigos, aceita-se qualquer um deles, pois o *schema* comporta um só.

scale Escala explícita em qualquer grafia (“1:25.000”, “1:25000”, “25k”, “25 mil”). As convenções informadas na descrição da ferramenta valem como regra (“detalhada” → 1:25.000; “pequena escala” → 1:250.000). Expressões qualitativas não informadas ao modelo aceitam o conjunto de escalas que as satisfazem: “grande escala” → 1:25.000 ou maior; “média escala” → 1:50.000 ou 1:100.000; “maior que 100k” → 1:50.000 ou maior. Duas escalas explícitas: aceita-se qualquer uma.

productType Sinônimos informados na ferramenta (“topo” e “topográfica”; “orto” e “ortoing”; “mdt”) e demais tipos pelo nome. “Cartas”, “mapas”, “produtos” e “folhas” sozinhos não definem tipo.

state, city Siglas e grafias sem acento são normalizadas. “Rio de Janeiro”, “São Paulo” e “rio” sem qualificador admitem as leituras estado ou município; siglas são sempre estado. Regiões (“sul”, “Nordeste”, “Amazônia Legal”) não são estados e não são anotadas, pois o *schema* não tem campo de região.

supplyArea Qualquer menção ao Centro de Geoinformação (“1º CGEO”, “1o cgeo”, “primeiro cgeo”, “dois cgeo”).

project Pelo nome ou pelos apelidos informados na ferramenta, com ou sem acento. Expressão que designa o projeto pelo termo distintivo do nome, sem ser o nome nem um apelido informado (“cartografia sistemática” para **Mapeamento Sistemático**), torna o campo opcional: a consulta sustenta a leitura, mas a ferramenta não a informa.

Períodos Verbo de publicação (“publicad-”) decide **publicationPeriod**; verbo de criação (“criad-”, “feit-”, “elaborad-”, “produzid-”) decide **creationPeriod**; sem verbo que desempate, a leitura preferencial é **publicationPeriod** e **creationPeriod** é leitura estrutural alternativa, com o mesmo intervalo. O intervalo é resolvido com a data de referência da execução — a mesma informada ao modelo — segundo o Quadro 5.

Ordenação “Mais recente(s)”, “último(s)” → **DESC**; “mais antiga(s)”, “primeiro(s)”, “ordem cronológica” → **ASC**. Pista de criação ligada à ordenação (“primeiro mapeamento feito”, “última atualização”, “criadas mais recentemente”) → **creationDate**; pista de publicação (“últimas publicações”, “última carta publicada”) → **publicationDate**; sem pista, aceita-se qualquer dos dois campos. Número explícito → **limit**; singular sem número (“a carta mais recente”) → **limit** = 1 opcional; plural sem número → sem **limit**. Ordenação nunca é anotada por padrão.

Consultas fora do domínio

A ferramenta não deve ser chamada quando a consulta trata de outro assunto (clima, preço, ficção), pede território fora do Brasil ou fora da Terra, ou não tem intenção de busca. Essas consultas formam a categoria F (fora do domínio) e entram nas métricas principais: a resposta é correta quando a ferramenta não é chamada. Consultas com valor impossível de representar (“escala 1:10.000.000”, “estado 42”) ou cujo comportamento esperado é discutível (“cartas do futuro”, “cartas topo no oceano atlântico”) são observacionais.

Quadro 5 – Leituras aceitas para as expressões de tempo relativo (D = data de referência da execução)

Expressão	Leituras aceitas (a primeira é a preferencial)
esse ano / este ano / deste ano	ano inteiro; 1º de janeiro até D
ano passado	ano anterior inteiro
2 anos atrás	o ano de dois anos antes, inteiro; D menos 2 anos até D
mês passado	mês anterior inteiro
este mês / deste mês	dia 1 até D; mês inteiro
semana passada	D-7 até D-1; D-7 até D; semana-calendário anterior (segunda a domingo)
esta semana / nesta semana	segunda-feira até D; segunda a domingo
hoje	D a D
últimos 3 (6) meses	D-90 (180) dias até D; mesmo dia, 3 (6) meses antes, até D
últimos 5 anos	D menos 5 anos até D; 1º de janeiro do ano-5 até D
desde AAAA	AAAA-01-01 até D; AAAA-01-01 sem limite final
depois de AAAA	início em AAAA-01-01 ou (AAAA+1)-01-01; fim em D ou sem limite final
antes de AAAA	sem limite inicial até (AAAA-1)-12-31
primeiro trimestre (deste ano)	janeiro a março do ano de D
segundo semestre do ano passado	julho a dezembro do ano anterior
último trimestre do ano passado	outubro a dezembro do ano anterior
trimestre passado	trimestre-calendário anterior ao de D

Fonte: Elaborado pelo autor. Implementação: `relative_time.py` do repositório.

A.2 Consultas do protótipo (P01–P22)

As 22 consultas do arquivo de testes do protótipo do 1º CGEO (`backend/evaluation/test-cases.ts`), na ordem do arquivo. A anotação original da equipe do 1º CGEO é mantida como leitura preferencial; as leituras alternativas acrescentadas seguem o manual. Os períodos relativos, que no arquivo original estavam fixados em datas de 2025, são expressos pelas regras nomeadas do Quadro 5.

ID	P01
Categorias	M · C
Consulta	<i>“preciso da carta MI 2965-2-NE do segundo cgeo”</i>
Leituras aceitas	keyword: "2965-2-NE" supplyArea: "2º Centro de Geoinformação"
Nota	origem: test-cases.ts do protótipo, linha 6

ID	P02
Categorias	M · C
Consulta	<i>“folha MI 2866-3 em escala 50k”</i>
Leituras aceitas	keyword: "2866-3" scale: "1:50.000"
Nota	origem: test-cases.ts do protótipo, linha 16

ID	P03
Categorias	M · C · A
Consulta	<i>“MI 2901 do projeto olimpíadas”</i>
Leituras aceitas	keyword: "2901" project: "Olimpíadas Rio 2016"
Nota	origem: test-cases.ts do protótipo, linha 26

ID	P04
Categorias	M · C · A
Consulta	<i>“carta 530 em pequena escala”</i>
Leituras aceitas	keyword: "530" scale: "1:250.000"
Nota	origem: test-cases.ts do protótipo, linha 36

ID	P05
Categorias	M · C · T
Consulta	<i>“buscar folha SF-22-Y-D-II-4 do RS publicada esse ano”</i>
Leituras aceitas	keyword: "SF-22-Y-D-II-4" state: "Rio Grande do Sul" publicationPeriod: ano corrente
Nota	origem: test-cases.ts do protótipo, linha 48

ID	P06
Categorias	M · C · A
Consulta	<i>“carta SF-22-Y-D do tipo ortoimg”</i>
Leituras aceitas	keyword: "SF-22-Y-D" productType: "SCN Carta Ortoimagem"
Nota	origem: test-cases.ts do protótipo, linha 62

ID	P07
Categorias	M · C · A
Consulta	<i>“inom SF-22-Y-D-II-4-SE criado pelo primeiro cgeo”</i>
Leituras aceitas	keyword: "SF-22-Y-D-II-4-SE" supplyArea: "1º Centro de Geoinformação"
Nota	origem: test-cases.ts do protótipo, linha 72

ID	P08
Categorias	C
Consulta	<i>“carta topográfica Passo da Seringueira em 1:25.000”</i>
Leituras aceitas	keyword: "Passo da Seringueira" scale: "1:25.000" productType: "SCN Carta Topográfica Matricial"
Nota	origem: test-cases.ts do protótipo, linha 84

ID	P09
Categorias	C
Consulta	<i>“folha Porto Velho publicada em 2024”</i>
Leituras aceitas	keyword: "Porto Velho" publicationPeriod: 2024-01-01 a 2024-12-31 aceita-se também city no lugar de keyword
Nota	origem: test-cases.ts do protótipo, linha 95; nome de carta coincide com município: aceita-se também city

ID	P10
Categorias	C · A
Consulta	<i>“carta Vale do Guaporé do projeto rondonia”</i>
Leituras aceitas	keyword: "Vale do Guaporé" project: "Base Cartográfica Digital de Rondônia"
Nota	origem: test-cases.ts do protótipo, linha 108

ID	P11
Categorias	M · C · A
Consulta	<i>“MI 2965-2-NE e SF-22-Y-D do terceiro cgeo em grande escala”</i>
Leituras aceitas	keyword: "2965-2-NE"ou "SF-22-Y-D" supplyArea: "3° Centro de Geoinformação" scale: "1:25.000"ou "1:10.000"ou "1:5.000"ou "1:2.000"ou "1:1.000"
Nota	origem: test-cases.ts do protótipo, linha 120; dois códigos (o schema comporta um): aceita-se qualquer um; 'grande escala' aceita as escalas grandes

ID	P12
Categorias	M · C · T · A
Consulta	<i>“cartas do tipo topo com MI 530 ou SF-22 publicadas esse ano”</i>
Leituras aceitas	keyword: "530"ou "SF-22" productType: "SCN Carta Topográfica Matricial" publicationPeriod: ano corrente
Nota	origem: test-cases.ts do protótipo, linha 131; dois códigos: aceita-se qualquer um

ID	P13
Categorias	C · A
Consulta	<i>“todas as cartas em escala maior que 100k do rio grande do sul”</i>
Leituras aceitas	state: "Rio Grande do Sul" scale: "1:25.000"ou "1:50.000"ou "1:10.000"ou "1:5.000"ou "1:2.000"ou "1:1.000"
Nota	origem: test-cases.ts do protótipo, linha 147; 'maior que 100k' aceita qualquer escala maior que 1:100.000

ID	P14
Categorias	C · A
Consulta	<i>“mapeamento detalhado (25k ou 50k) de brasília”</i>
Leituras aceitas	scale: "1:25.000"ou "1:50.000" city: "Brasília"
Nota	origem: test-cases.ts do protótipo, linha 156; duas escalas explícitas: aceita-se qualquer uma

ID	P15
Categorias	T · C · A
Consulta	<i>“cartas criadas no último trimestre do ano passado do segundo cgeo”</i>
Leituras aceitas	supplyArea: "2º Centro de Geoinformação" creationPeriod: 4º trimestre do ano anterior
Nota	origem: test-cases.ts do protótipo, linha 167

ID	P16
Categorias	T · C · A
Consulta	<i>“produtos da semana passada do terceiro cgeo”</i>
Leituras aceitas	supplyArea: "3º Centro de Geoinformação" publicationPeriod: semana passada aceita-se também creationPeriod no lugar de publicationPeriod
Nota	origem: test-cases.ts do protótipo, linha 179; sem verbo que decida publicação ou criação

ID	P17
Categorias	C · A
Consulta	<i>“cartas do mapeamento sistematico em goias”</i>
Leituras aceitas	project: "Mapeamento Sistemático" state: "Goiás"
Nota	origem: test-cases.ts do protótipo, linha 193

ID	P18
Categorias	C · A
Consulta	<i>“produtos da copa das confederacoes em fortaleza”</i>
Leituras aceitas	project: "Copa das Confederações" city: "Fortaleza"
Nota	origem: test-cases.ts do protótipo, linha 202

ID	P19
Categorias	O · C · A
Consulta	<i>“primeiro mapeamento feito em cuiaba”</i>
Leituras aceitas	city: "Cuiabá" sortField: "creationDate" sortDirection: "ASC" limit: 1 (opcional)
Nota	origem: test-cases.ts do protótipo, linha 213; singular sem número: limit 1 opcional

ID	P20
Categorias	O · C · A
Consulta	<i>“ultima atualizacao do para”</i>
Leituras aceitas	state: "Pará" sortField: "creationDate" sortDirection: "DESC" limit: 1 (opcional)
Nota	origem: test-cases.ts do protótipo, linha 223; singular sem número: limit 1 opcional

ID	P21
Categorias	O · T · C · A
Consulta	<i>“preciso das 5 cartas mais antigas do tipo ortoimg em escala detalhada do terceiro cgeo em pernambuco publicadas depois de 2020”</i>
Leituras aceitas	limit: 5 productType: "SCN Carta Ortoimagem" scale: "1:25.000" supplyArea: "3° Centro de Geoinformação" state: "Pernambuco" publicationPeriod: depois de 2020 sortField: "creationDate"ou "publicationDate" sortDirection: "ASC"
Nota	origem: test-cases.ts do protótipo, linha 235; 'mais antigas' sem pista ligada à ordenação: aceita-se publicationDate além do creationDate original

ID	P22
Categorias	M · O · T · C
Consulta	<i>“MI 2901 ou SF-22 do quarto cgeo no sudeste em pequena escala criado entre 2022 e 2023 ordem cronológica”</i>
Leituras aceitas	keyword: "2901"ou "SF-22" supplyArea: "4° Centro de Geoinformação" scale: "1:250.000" creationPeriod: 2022-01-01 a 2023-12-31 sortField: "creationDate"ou "publicationDate" sortDirection: "ASC"
Nota	origem: test-cases.ts do protótipo, linha 253; dois códigos; 'sudeste' é região (nada a anotar); ordem cronológica sem pista ligada

A.3 Consultas redigidas pelos autores (N01–N40)

Consultas redigidas para preencher lacunas de distribuição — em especial a categoria Simples, ausente na camada P —, exercitar as ambiguidades previstas no manual (estado ou capital, ordenação sem pista, regiões, expressões qualitativas de escala) e cobrir a fronteira do domínio: N36 e N37 estão fora do domínio (categoria F), e N38 a N40 são observacionais.

ID	N01
Categorias	S
Consulta	<i>“cartas de São Paulo”</i>
Leituras aceitas	state: "São Paulo" aceita-se também city no lugar de state
Nota	'São Paulo' sem qualificador: estado ou capital

ID	N02
Categorias	S
Consulta	<i>“produtos em escala 1:50.000”</i>
Leituras aceitas	scale: "1:50.000"

ID	N03
Categorias	S
Consulta	<i>“ortoimagens”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem"

ID	N04
Categorias	S · A
Consulta	<i>“mapas do rj”</i>
Leituras aceitas	state: "Rio de Janeiro"

ID	N05
Categorias	S · A
Consulta	<i>“carta 25k”</i>
Leituras aceitas	scale: "1:25.000"

ID	N06
Categorias	S
Consulta	<i>“folha SF-22-Y-D”</i>
Leituras aceitas	keyword: "SF-22-Y-D"

ID	N07
Categorias	S · A
Consulta	<i>“modelos digitais de terreno”</i>
Leituras aceitas	productType: "MDT - RAM"

ID	N08
Categorias	S · A
Consulta	<i>“cartas topográficas”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial"

ID	N09
Categorias	S
Consulta	<i>“produtos do Amazonas”</i>
Leituras aceitas	state: "Amazonas"

ID	N10
Categorias	S · A
Consulta	<i>“mi 2901”</i>
Leituras aceitas	keyword: "2901"

ID	N11
Categorias	C
Consulta	<i>“cartas do 1º CGEO em Minas Gerais”</i>
Leituras aceitas	supplyArea: "1º Centro de Geoinformação" state: "Minas Gerais"

ID	N12
Categorias	C
Consulta	<i>“ortoimagens de Manaus em 1:50.000”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" city: "Manaus" scale: "1:50.000"

ID	N13
Categorias	C · A
Consulta	<i>“mdt do para em 25k”</i>
Leituras aceitas	productType: "MDT - RAM" state: "Pará" scale: "1:25.000"

ID	N14
Categorias	M · C
Consulta	<i>“folha 2965-2 do Rio Grande do Sul”</i>
Leituras aceitas	keyword: "2965-2" state: "Rio Grande do Sul"

ID	N15
Categorias	M · C
Consulta	<i>“INOM SG-22-X-A”</i>
Leituras aceitas	keyword: "SG-22-X-A"

ID	N16
Categorias	M · C · T
Consulta	<i>“carta MI 3010 publicada em 2023”</i>
Leituras aceitas	keyword: "3010" publicationPeriod: 2023-01-01 a 2023-12-31

ID	N17
Categorias	T · A
Consulta	<i>“produtos publicados nos últimos 3 meses”</i>
Leituras aceitas	publicationPeriod: últimos 3 meses

ID	N18
Categorias	T · C
Consulta	<i>“cartas do ano passado em Roraima”</i>
Leituras aceitas	state: "Roraima" publicationPeriod: ano anterior, inteiro aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N19
Categorias	T · A
Consulta	<i>“atualizações deste mês”</i>
Leituras aceitas	publicationPeriod: mês corrente aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N20
Categorias	T · C · A
Consulta	<i>“cartas dos últimos 5 anos em escala 100k”</i>
Leituras aceitas	scale: "1:100.000" publicationPeriod: últimos 5 anos aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N21
Categorias	T · A
Consulta	<i>“qualquer coisa de hoje”</i>
Leituras aceitas	publicationPeriod: dia da execução aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N22
Categorias	T · C · A
Consulta	<i>“ortoimagens de 2 anos atrás”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" publicationPeriod: ano de dois anos antes aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N23
Categorias	T · C · A
Consulta	<i>“produtos do trimestre passado do 4º CGEO”</i>
Leituras aceitas	supplyArea: "4º Centro de Geoinformação" publicationPeriod: trimestre anterior aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N24
Categorias	O · C
Consulta	<i>“3 cartas mais recentes de Curitiba”</i>
Leituras aceitas	city: "Curitiba" limit: 3 sortField: "publicationDate"ou "creationDate" sortDirection: "DESC"
Nota	'mais recentes' sem pista: aceita-se publicationDate ou creationDate

ID	N25
Categorias	O · C
Consulta	<i>“10 primeiras ortoimagens do Nordeste”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" limit: 10 sortField: "publicationDate"ou "creationDate" sortDirection: "ASC"
Nota	'Nordeste' é região (nada a anotar); 'primeiras' sem pista

ID	N26
Categorias	O · C · A
Consulta	<i>“mais antigas em rondonia”</i>
Leituras aceitas	state: "Rondônia" sortField: "publicationDate"ou "creationDate" sortDirection: "ASC"
Nota	'mais antigas' sem pista

ID	N27
Categorias	O · C
Consulta	<i>“últimas 5 publicações”</i>
Leituras aceitas	limit: 5 sortField: "publicationDate" sortDirection: "DESC"

ID	N28
Categorias	O · M · C
Consulta	<i>“carta MI 2901 mais recente”</i>
Leituras aceitas	keyword: "2901" sortField: "publicationDate"ou "creationDate" sortDirection: "DESC" limit: 1 (opcional)
Nota	singular sem número: limit 1 opcional; 'mais recente' sem pista

ID	N29
Categorias	O · C · T
Consulta	<i>“três produtos mais antigos deste ano”</i>
Leituras aceitas	limit: 3 publicationPeriod: ano corrente sortField: "publicationDate"ou "creationDate" sortDirection: "ASC" aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida o período; 'mais antigos' sem pista

ID	N30
Categorias	A · C
Consulta	<i>“cartas topo do primeiro cgeo”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial" supplyArea: "1° Centro de Geoinformação"

ID	N31
Categorias	A · S
Consulta	<i>“orto banda p”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem Banda P Pol HH"

ID	N32
Categorias	A · C
Consulta	<i>“mapas do rio em cem k”</i>
Leituras aceitas	state: "Rio de Janeiro" scale: "1:100.000" aceita-se também city no lugar de state
Nota	'rio' é ambíguo entre o estado e a capital

ID	N33
Categorias	A · C
Consulta	<i>“cartografia sistematica sp”</i>
Leituras aceitas	project: "Mapeamento Sistemático"(opcional) state: "São Paulo"
Nota	'cartografia sistemática' designa o programa pelo termo distintivo do nome, sem ser nome nem apelido informado na ferramenta: project é opcional (regra de project; adjudicação)

ID	N34
Categorias	A · C
Consulta	<i>“ortos da amazonia legal”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem"
Nota	'Amazônia Legal' é região (nove estados), não estado: nada a anotar para ela

ID	N35
Categorias	A · M · C
Consulta	<i>“sf22yd do dois cgeo”</i>
Leituras aceitas	keyword: "SF-22-Y-D" supplyArea: "2° Centro de Geoinformação"

ID	N36
Categorias	F
Consulta	<i>“cartas fora do brasil”</i>
Leituras aceitas	não chamar a ferramenta (fora do território brasileiro, P4)

ID	N37
Categorias	F
Consulta	<i>“quero um mapa bonito”</i>
Leituras aceitas	não chamar a ferramenta (sem critério de busca, P4)

ID	N38
Categorias	S · A
Consulta	<i>“escala 1:10.000.000”</i>
Leituras aceitas	não chamar a ferramenta (observacional, P6: escala fora do enumerado da ferramenta)

ID	N39
Categorias	C · A
Consulta	<i>“cartas topo no oceano atlântico”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial" caso observacional (P6): executado e reportado, fora das métricas principais
Nota	área não terrestre: comportamento esperado discutível

ID	N40
Categorias	C · T
Consulta	<i>“cartas do futuro (ano 2050)”</i>
Leituras aceitas	publicationPeriod: 2050-01-01 a 2050-12-31 aceita-se também creationPeriod no lugar de publicationPeriod caso observacional (P6): executado e reportado, fora das métricas principais
Nota	data futura: comportamento esperado discutível

A.4 Geração programática da camada G

As 248 consultas da camada G são produzidas pelo *script* `generate_dataset.py` do repositório, a partir de catálogos fechados do domínio: as 27 unidades da federação

(com sigla, preposição e grafias usuais), 12 municípios de referência sem homônimo entre as UFs, as 8 escalas do SCN (com apelidos como “25k”, “25 mil” e as convenções “detalhada” e “pequena escala”), os 9 tipos de produto das ET-PCDG, os 5 CGEOs, os 9 projetos institucionais, 12 códigos MI e 10 códigos INOM, 15 expressões de tempo relativo e os padrões de ordenação. Oito funções geradoras, uma por família, combinam esses catálogos em modelos de frase do português corrente.

Três propriedades tornam a camada auditável. A geração é **determinística**: um único gerador pseudoaleatório, com semente 42, consumido em ordem fixa, de modo que a mesma versão do *script* produz sempre as mesmas consultas. O gabarito é conhecido **por construção**: a frase é montada a partir dos parâmetros que deve produzir, e cada consulta registra no *dataset* a função geradora e o modelo de frase que a originaram. E **nenhuma consulta se repete**: a comparação é feita após normalização de caixa, acentos e pontuação, dentro da camada e em relação às camadas P e N. As convenções de anotação são as mesmas do manual; em particular, “São Paulo” e “Rio de Janeiro” por extenso recebem as leituras estado ou município, e frases sem verbo de publicação ou criação aceitam os dois campos de período.

As seções seguintes listam as 248 consultas por família. Os identificadores seguem o padrão $G\{\text{família}\}\{\text{sequência}\}$, em que a família é S (simples), C (compostas), M (código MI/INOM), T (tempo relativo), O (ordenação), A (informais), P (amplas, com muitos produtos) ou F (fronteira do domínio).

A.5 Consultas simples geradas por *templates* (G-S)

Total desta família: **61 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GS001	S	cartas do Acre	state: "Acre"
GS002	S · A	cartas de AL	state: "Alagoas"
GS003	S · A	cartas do AP	state: "Amapá"
GS004	S	cartas do Amazonas	state: "Amazonas"
GS005	S · A	cartas da bahia	state: "Bahia"
GS006	S · A	cartas do CE	state: "Ceará"
GS007	S · A	cartas do DF	state: "Distrito Federal"
GS008	S · A	cartas do ES	state: "Espírito Santo"
GS009	S	cartas de Goiás	state: "Goiás"
GS010	S · A	cartas do MA	state: "Maranhão"
GS011	S	cartas de Mato Grosso	state: "Mato Grosso"
GS012	S	cartas de Mato Grosso do Sul	state: "Mato Grosso do Sul"
GS013	S · A	cartas de MG	state: "Minas Gerais"
GS014	S	cartas do Pará	state: "Pará"

ID	Cat.	Consulta	Leituras aceitas
GS015	S · A	cartas da paraíba	state: "Paraíba"
GS016	S · A	produtos na escala 1:1000	scale: "1:1.000"
GS017	S · A	produtos na escala 1:2000	scale: "1:2.000"
GS018	S · A	produtos na escala 1:5000	scale: "1:5.000"
GS019	S · A	produtos na escala 1:10000	scale: "1:10.000"
GS020	S · A	produtos na escala 1:25000	scale: "1:25.000"
GS021	S · A	produtos na escala de 50 mil	scale: "1:50.000"
GS022	S · A	produtos na escala de 100 mil	scale: "1:100.000"
GS023	S · A	produtos em 250k	scale: "1:250.000"
GS024	S · A	cartas topográficas disponíveis	productType: "SCN Carta Topográfica Matricial"
GS025	S · A	cartas vetoriais disponíveis	productType: "SCN Carta Topográfica Vetorial"
GS026	S · A	ortoimagens disponíveis	productType: "SCN Carta Ortoimagem"
GS027	S · A	ortoimagens banda P HH disponíveis	productType: "SCN Carta Ortoimagem Banda P Pol HH"
GS028	S · A	ortoimagens banda X HH disponíveis	productType: "SCN Carta Ortoimagem Banda X Pol HH"
GS029	S · A	MDTs disponíveis	productType: "MDT — RAM"
GS030	S · A	MDSs disponíveis	productType: "MDS — RAM"
GS031	S · A	cartas CIRC disponíveis	productType: "CIRC"
GS032	S · A	mapas temáticos disponíveis	productType: "Cartas Temáticas Não SCN"
GS033	S · A	produtos do primeiro cgeo	supplyArea: "1° Centro de Geoinformação"
GS034	S · A	produtos do 2o cgeo	supplyArea: "2° Centro de Geoinformação"
GS035	S · A	produtos do 3o cgeo	supplyArea: "3° Centro de Geoinformação"
GS036	S · A	produtos do quarto cgeo	supplyArea: "4° Centro de Geoinformação"
GS037	S · A	produtos do 5º CGEO	supplyArea: "5° Centro de Geoinformação"
GS038	S	cartas de Brasília	city: "Brasília"
GS039	S	cartas de Manaus	city: "Manaus"
GS040	S	cartas de Porto Velho	city: "Porto Velho"
GS041	S	cartas de Cuiabá	city: "Cuiabá"
GS042	S	cartas de Fortaleza	city: "Fortaleza"
GS043	S	cartas de Salvador	city: "Salvador"
GS044	S	cartas de Belo Horizonte	city: "Belo Horizonte"
GS045	S	cartas de Curitiba	city: "Curitiba"
GS046	S	cartas de Porto Alegre	city: "Porto Alegre"
GS047	S	cartas de Recife	city: "Recife"
GS048	S · M	carta MI 2965-2-NE	keyword: "2965-2-NE"
GS049	S · M	carta MI 2866-3	keyword: "2866-3"

ID	Cat.	Consulta	Leituras aceitas
GS050	S · M	carta MI 2901	keyword: "2901"
GS051	S · M	carta MI 530	keyword: "530"
GS052	S · M	carta MI 3010	keyword: "3010"
GS053	S · M	carta MI 2965	keyword: "2965"
GS054	S · M	carta MI 2866-2-SO	keyword: "2866-2-SO"
GS055	S · M	carta MI 2901-4-NE	keyword: "2901-4-NE"
GS056	S · M	folha INOM SF-22-Y-D	keyword: "SF-22-Y-D"
GS057	S · M	folha INOM SF-22-Y-D-II	keyword: "SF-22-Y-D-II"
GS058	S · M	folha INOM SF-22-Y-D-II-4	keyword: "SF-22-Y-D-II-4"
GS059	S · M	folha INOM SF-22-Y-D-II-4-SE	keyword: "SF-22-Y-D-II-4-SE"
GS060	S · M	folha INOM SG-22-X-A	keyword: "SG-22-X-A"
GS061	S · M	folha INOM SF-23-V-C	keyword: "SF-23-V-C"

A.6 Consultas compostas geradas por *templates* (G-C)

Total desta família: **55 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GC001	C · A	cartas do amapá na escala 1:2.000	state: "Amapá"; scale: "1:2.000"
GC002	C · A	cartas de Mato Grosso do Sul em 25k	state: "Mato Grosso do Sul"; scale: "1:25.000"
GC003	C · A	cartas de santa catarina na escala 1:2.000	state: "Santa Catarina"; scale: "1:2.000"
GC004	C · A	cartas do Amapá na escala de 25 mil	state: "Amapá"; scale: "1:25.000"
GC005	C · A	cartas do RJ na escala 1:2000	state: "Rio de Janeiro"; scale: "1:2.000"
GC006	C · A	cartas de RO em 25k	state: "Rondônia"; scale: "1:25.000"
GC007	C · A	cartas do ES na escala 1:100000	state: "Espírito Santo"; scale: "1:100.000"
GC008	C · A	cartas da Paraíba em 50k	state: "Paraíba"; scale: "1:50.000"
GC009	C · A	cartas de mato grosso do sul na escala de 10 mil	state: "Mato Grosso do Sul"; scale: "1:10.000"
GC010	C · A	cartas de Goiás na escala 1:2000	state: "Goiás"; scale: "1:2.000"
GC011	C · A	cartas do Piauí na escala 1:10000	state: "Piauí"; scale: "1:10.000"
GC012	C · A	cartas da paraíba na escala 1:25000	state: "Paraíba"; scale: "1:25.000"
GC013	C · A	cartas de rondônia na escala 1:1000	state: "Rondônia"; scale: "1:1.000"
GC014	C · A	cartas do TO na escala 1:50000	state: "Tocantins"; scale: "1:50.000"
GC015	C · A	cartas de GO na escala de 10 mil	state: "Goiás"; scale: "1:10.000"
GC016	C · A	cartas de roraima na escala de 10 mil	state: "Roraima"; scale: "1:10.000"
GC017	C · A	cartas do paraná na escala 1:250000	state: "Paraná"; scale: "1:250.000"
GC018	C · A	cartas do Piauí em escala detalhada	state: "Piauí"; scale: "1:25.000"
GC019	C · A	cartas do rio de janeiro em 50k	state: "Rio de Janeiro"; scale: "1:50.000"; aceita-se também city no lugar de state

ID	Cat.	Consulta	Leituras aceitas
GC020	C · A	cartas da Bahia em 250k	state: "Bahia"; scale: "1:250.000"
GC021	C · A	cartas de SP na escala 1:2000	state: "São Paulo"; scale: "1:2.000"
GC022	C · A	cartas do RS na escala de 100 mil	state: "Rio Grande do Sul"; scale: "1:100.000"
GC023	C · A	cartas do amapá na escala de 100 mil	state: "Amapá"; scale: "1:100.000"
GC024	C · A	cartas da Paraíba em 25k	state: "Paraíba"; scale: "1:25.000"
GC025	C	cartas de Rondônia na escala 1:2.000	state: "Rondônia"; scale: "1:2.000"
GC026	C · A	cartas de São Paulo em 50k	state: "São Paulo"; scale: "1:50.000"; aceita-se também city no lugar de state
GC027	C · A	cartas do maranhão na escala 1:5.000	state: "Maranhão"; scale: "1:5.000"
GC028	C · A	cartas do Acre na escala 1:25000	state: "Acre"; scale: "1:25.000"
GC029	C · A	cartas de PE na escala 1:25000	state: "Pernambuco"; scale: "1:25.000"
GC030	C · A	cartas da bahia na escala 1:5000	state: "Bahia"; scale: "1:5.000"
GC031	C · A	mapas de Recife na escala 1:50000	city: "Recife"; scale: "1:50.000"
GC032	C	mapas de Brasília na escala 1:2.000	city: "Brasília"; scale: "1:2.000"
GC033	C · A	mapas de Fortaleza na escala 1:10000	city: "Fortaleza"; scale: "1:10.000"
GC034	C · A	mapas de Cuiabá na escala 1:2000	city: "Cuiabá"; scale: "1:2.000"
GC035	C · A	mapas de Belém em 250k	city: "Belém"; scale: "1:250.000"
GC036	C · A	mapas de Porto Alegre na escala 1:5000	city: "Porto Alegre"; scale: "1:5.000"
GC037	C · A	mapas de Goiânia na escala 1:250000	city: "Goiânia"; scale: "1:250.000"
GC038	C · A	mapas de Fortaleza em 100k	city: "Fortaleza"; scale: "1:100.000"
GC039	C · A	cartas temáticas de São Paulo	productType: "Cartas Temáticas Não SCN"; state: "São Paulo"; aceita-se também city no lugar de state
GC040	C · A	ortoimagens banda X HH de minas	productType: "SCN Carta Ortoimagem Banda X Pol HH"; state: "Minas Gerais"
GC041	C · A	cartas temáticas da PB	productType: "Cartas Temáticas Não SCN"; state: "Paraíba"
GC042	C · A	ortoimagens banda P HH do amapá	productType: "SCN Carta Ortoimagem Banda P Pol HH"; state: "Amapá"
GC043	C · A	cartas temáticas do Espírito Santo	productType: "Cartas Temáticas Não SCN"; state: "Espírito Santo"
GC044	C · A	cartas topográficas do Amapá	productType: "SCN Carta Topográfica Matricial"; state: "Amapá"
GC045	C · A	produtos do projeto copa das confederacoes no AP	project: "Copa das Confederações"; state: "Amapá"
GC046	C · A	produtos do projeto olimpíadas em PE	project: "Olimpíadas Rio 2016"; state: "Pernambuco"

ID	Cat.	Consulta	Leituras aceitas
GC047	C	produtos do projeto NGA-BECA no Distrito Federal	project: "NGA-BECA"; state: "Distrito Federal"
GC048	C · A	produtos do projeto beca no espírito santo	project: "NGA-BECA"; state: "Espírito Santo"
GC049	C · A	produtos do projeto copa das confederacoes no AM	project: "Copa das Confederações"; state: "Amazonas"
GC050	C · A	produtos do projeto BCD de Rondônia no Pará	project: "Base Cartográfica Digital de Rondônia"; state: "Pará"
GC051	C · A	cartas do 1º CGEO na escala 1:2.000	supplyArea: "1º Centro de Geoinformação"; scale: "1:2.000"
GC052	C · A	cartas do 3º cgeo na escala 1:2000	supplyArea: "3º Centro de Geoinformação"; scale: "1:2.000"
GC053	C · A	cartas do 2º cgeo em pequena escala	supplyArea: "2º Centro de Geoinformação"; scale: "1:250.000"
GC054	C · A	cartas do 2º CGEO na escala 1:25000	supplyArea: "2º Centro de Geoinformação"; scale: "1:25.000"
GC055	C · A	cartas do 1º CGEO em 250k	supplyArea: "1º Centro de Geoinformação"; scale: "1:250.000"

A.7 Consultas com código MI/INOM geradas por *templates* (G-M)

Total desta família: **28 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GM001	C · M	folha MI 2965-2-NE do Rio Grande do Sul	keyword: "2965-2-NE"; state: "Rio Grande do Sul"
GM002	C · M · A	folha MI 2866-3 do TO	keyword: "2866-3"; state: "Tocantins"
GM003	C · M · A	folha MI 2901 do AP	keyword: "2901"; state: "Amapá"
GM004	C · M · A	folha MI 530 do ceará	keyword: "530"; state: "Ceará"
GM005	C · M · A	folha MI 3010 do paraná	keyword: "3010"; state: "Paraná"
GM006	C · M · A	folha MI 2965 do distrito federal	keyword: "2965"; state: "Distrito Federal"
GM007	C · M · A	folha MI 2866-2-SO de AL	keyword: "2866-2-SO"; state: "Alagoas"
GM008	C · M · A	folha MI 2901-4-NE de MG	keyword: "2901-4-NE"; state: "Minas Gerais"

ID	Cat.	Consulta	Leituras aceitas
GM009	C · M · A	folha MI 531-1-NE de minas	keyword: "531-1-NE"; state: "Minas Gerais"
GM010	C · M	folha MI 2965-3 de Sergipe	keyword: "2965-3"; state: "Sergipe"
GM011	C · M · A	folha MI 2866-3-NO do maranhão	keyword: "2866-3-NO"; state: "Maranhão"
GM012	C · M	folha MI 3010-2-SE de Roraima	keyword: "3010-2-SE"; state: "Roraima"
GM013	C · M · A	INOM SF-22-Y-D do tipo temática	keyword: "SF-22-Y-D"; productType: "Cartas Temáticas Não SCN"
GM014	C · M · A	INOM SF-22-Y-D-II do tipo ortoimagem	keyword: "SF-22-Y-D-II"; productType: "SCN Carta Ortoimagem"
GM015	C · M · A	INOM SF-22-Y-D-II-4 do tipo banda X HH	keyword: "SF-22-Y-D-II-4"; productType: "SCN Carta Ortoimagem Banda X Pol HH"
GM016	C · M · A	INOM SF-22-Y-D-II-4-SE do tipo topográfica	keyword: "SF-22-Y-D-II-4-SE"; productType: "SCN Carta Topográfica Matricial"
GM017	C · M · A	INOM SG-22-X-A do tipo MDT	keyword: "SG-22-X-A"; productType: "MDT — RAM"
GM018	C · M · A	INOM SF-23-V-C do tipo topo	keyword: "SF-23-V-C"; productType: "SCN Carta Topográfica Matricial"
GM019	C · M · A	INOM SG-21-Z-B do tipo temática	keyword: "SG-21-Z-B"; productType: "Cartas Temáticas Não SCN"
GM020	C · M · A	INOM SH-22-W-A-III-1-NE do tipo topográfica	keyword: "SH-22-W-A-III-1-NE"; productType: "SCN Carta Topográfica Matricial"
GM021	C · M · A	INOM SG-22-Z-D-II-3 do tipo ortoimagem	keyword: "SG-22-Z-D-II-3"; productType: "SCN Carta Ortoimagem"
GM022	C · M · A	INOM SF-24-Y-A do tipo vetorial	keyword: "SF-24-Y-A"; productType: "SCN Carta Topográfica Vetorial"
GM023	C · M · A	MI 2866-2-SO produzido pelo 1 CGEO	keyword: "2866-2-SO"; supplyArea: "1° Centro de Geoinformação"
GM024	C · M · A	MI 2866-3 produzido pelo quinto cgeo	keyword: "2866-3"; supplyArea: "5° Centro de Geoinformação"
GM025	C · M · A	MI 2965-3 produzido pelo 3o cgeo	keyword: "2965-3"; supplyArea: "3° Centro de Geoinformação"
GM026	C · M · A	MI 530 produzido pelo 3o cgeo	keyword: "530"; supplyArea: "3° Centro de Geoinformação"
GM027	C · M · A	MI 2965-2-NE produzido pelo 3 CGEO	keyword: "2965-2-NE"; supplyArea: "3° Centro de Geoinformação"
GM028	C · M · A	MI 3010 produzido pelo segundo cgeo	keyword: "3010"; supplyArea: "2° Centro de Geoinformação"

A.8 Consultas com tempo relativo geradas por *templates* (G-T)

Total desta família: **41 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GT001	T	cartas publicadas esse ano	publicationPeriod: ano corrente
GT002	T	cartas publicadas no ano passado	publicationPeriod: ano anterior, inteiro
GT003	T	cartas publicadas no mês passado	publicationPeriod: mês anterior, inteiro
GT004	T	cartas publicadas na semana passada	publicationPeriod: semana passada
GT005	T	cartas publicadas nos últimos 3 meses	publicationPeriod: últimos 3 meses
GT006	T	cartas publicadas nos últimos 6 meses	publicationPeriod: últimos 6 meses
GT007	T	cartas publicadas nos últimos 5 anos	publicationPeriod: últimos 5 anos
GT008	T	cartas publicadas desde 2020	publicationPeriod: desde 2020
GT009	T	cartas publicadas depois de 2022	publicationPeriod: depois de 2022
GT010	T	cartas publicadas antes de 2020	publicationPeriod: antes de 2020
GT011	T	cartas publicadas entre 2022 e 2023	publicationPeriod: 2022-01-01 a 2023-12-31
GT012	T	cartas publicadas no primeiro trimestre deste ano	publicationPeriod: 1º trimestre do ano corrente
GT013	T	cartas publicadas no segundo semestre do ano passado	publicationPeriod: 2º semestre do ano anterior
GT014	T	cartas publicadas nesta semana	publicationPeriod: semana corrente
GT015	T	cartas publicadas hoje	publicationPeriod: dia da execução
GT016	C · T · A	cartas de pernambuco publicadas desde 2020	state: "Pernambuco"; publicationPeriod: desde 2020
GT017	C · T · A	cartas da bahia publicadas nos últimos 6 meses	state: "Bahia"; publicationPeriod: últimos 6 meses
GT018	C · T · A	cartas do AP produzidas no segundo semestre do ano passado	state: "Amapá"; creationPeriod: 2º semestre do ano anterior
GT019	C · T · A	cartas do MA publicadas no ano passado	state: "Maranhão"; publicationPeriod: ano anterior, inteiro
GT020	C · T	cartas da Paraíba publicadas esse ano	state: "Paraíba"; publicationPeriod: ano corrente
GT021	C · T · A	cartas de roraima elaboradas hoje	state: "Roraima"; creationPeriod: dia da execução
GT022	C · T · A	cartas de sergipe publicadas no primeiro trimestre deste ano	state: "Sergipe"; publicationPeriod: 1º trimestre do ano corrente
GT023	C · T	cartas de Rondônia publicadas depois de 2022	state: "Rondônia"; publicationPeriod: depois de 2022
GT024	C · T	cartas do Maranhão criadas nos últimos 3 meses	state: "Maranhão"; creationPeriod: últimos 3 meses

ID	Cat.	Consulta	Leituras aceitas
GT025	C · T · A	cartas da bahia publicadas antes de 2020	state: "Bahia"; publicationPeriod: antes de 2020
GT026	C · T · A	produtos do 3o cgeo publicados no ano passado	supplyArea: "3° Centro de Geoinformação"; publicationPeriod: ano anterior, inteiro
GT027	C · T · A	produtos do 3 CGEO publicados hoje	supplyArea: "3° Centro de Geoinformação"; publicationPeriod: dia da execução
GT028	C · T · A	produtos do 3º CGEO criados no primeiro trimestre deste ano	supplyArea: "3° Centro de Geoinformação"; creationPeriod: 1º trimestre do ano corrente
GT029	C · T · A	produtos do quarto cgeo publicados depois de 2022	supplyArea: "4° Centro de Geoinformação"; publicationPeriod: depois de 2022
GT030	C · T · A	produtos do 1º CGEO publicados no mês passado	supplyArea: "1° Centro de Geoinformação"; publicationPeriod: mês anterior, inteiro
GT031	C · T · A	produtos do terceiro cgeo criados nos últimos 3 meses	supplyArea: "3° Centro de Geoinformação"; creationPeriod: últimos 3 meses
GT032	C · T · A	produtos do 2 CGEO publicados antes de 2020	supplyArea: "2° Centro de Geoinformação"; publicationPeriod: antes de 2020
GT033	C · T · A	produtos do 5 CGEO publicados na semana passada	supplyArea: "5° Centro de Geoinformação"; publicationPeriod: semana passada
GT034	C · T	cartas na escala 1:5.000 publicadas depois de 2022	scale: "1:5.000"; publicationPeriod: depois de 2022
GT035	C · T · A	cartas na escala 1:5000 publicadas esse ano	scale: "1:5.000"; publicationPeriod: ano corrente
GT036	C · T · A	cartas na escala de 25 mil criadas no ano passado	scale: "1:25.000"; creationPeriod: ano anterior, inteiro
GT037	C · T · A	cartas em 50k publicadas no segundo semestre do ano passado	scale: "1:50.000"; publicationPeriod: 2º semestre do ano anterior
GT038	C · T · A	cartas na escala de 10 mil publicadas no mês passado	scale: "1:10.000"; publicationPeriod: mês anterior, inteiro
GT039	C · T	cartas na escala 1:2.000 elaboradas hoje	scale: "1:2.000"; creationPeriod: dia da execução
GT040	C · T · A	cartas na escala de 100 mil publicadas nesta semana	scale: "1:100.000"; publicationPeriod: semana corrente
GT041	C · T · A	cartas na escala 1:5000 publicadas nos últimos 6 meses	scale: "1:5.000"; publicationPeriod: últimos 6 meses

A.9 Consultas com ordenação geradas por *templates* (G-O)

Total desta família: **19 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GO001	C · O · A	a carta mais recente do CE	state: "Ceará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO002	C · O · A	a carta mais antiga do PA	state: "Pará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO003	C · O	as cartas mais recentes do Ceará	state: "Ceará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO004	C · O · A	as cartas mais antigas de mato grosso	state: "Mato Grosso"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"
GO005	C · O · A	a primeira carta produzida de sergipe	state: "Sergipe"; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO006	C · O · A	a última carta publicada de GO	state: "Goiás"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO007	C · O · A	as 3 cartas mais antigas de sergipe	state: "Sergipe"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 3
GO008	C · O · A	as 5 publicações mais recentes de MG	state: "Minas Gerais"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 5
GO009	C · O · A	as 10 cartas criadas mais recentemente do PR	state: "Paraná"; sortField: "creationDate"; sortDirection: "DESC"; limit: 10
GO010	C · O · A	cartas do distrito federal em ordem cronológica de publicação	state: "Distrito Federal"; sortField: "publicationDate"; sortDirection: "ASC"
GO011	C · O	as cartas temáticas mais recentes	productType: "Cartas Temáticas Não SCN"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO012	C · O	as 3 cartas CIRC mais antigas	productType: "CIRC"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 3
GO013	C · O	a última ortoimagem publicada	productType: "SCN Carta Ortoimagem"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO014	C · O	a primeira carta topográfica vetorial elaborada	productType: "SCN Carta Topográfica Vetorial"; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO015	C · O	cartas topográficas da mais nova para a mais antiga	productType: "SCN Carta Topográfica Matricial"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"

ID	Cat.	Consulta	Leituras aceitas
GO016	C · T · O	as cartas mais recentes publicadas entre 2022 e 2023	publicationPeriod: 2022-01-01 a 2023-12-31; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO017	C · T · O	a primeira carta criada na semana passada	creationPeriod: semana passada; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO018	C · T · O	as 5 cartas mais antigas publicadas nos últimos 5 anos	publicationPeriod: últimos 5 anos; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 5
GO019	C · T · O	a última carta publicada nos últimos 6 meses	publicationPeriod: últimos 6 meses; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)

A.10 Consultas ambíguas/informais adicionais (G-A)

Total desta família: **24 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GA001	S · A	mapas de pe	state: "Pernambuco"
GA002	S · A	mapas do pr	state: "Paraná"
GA003	S · A	mapas de sc	state: "Santa Catarina"
GA004	S · A	mapas de go	state: "Goiás"
GA005	S · A	mapas do ce	state: "Ceará"
GA006	S · A	mapas do pa	state: "Pará"
GA007	S · A	mapas de goias	state: "Goiás"
GA008	S · A	mapas de rondonia	state: "Rondônia"
GA009	S · A	mapas do piaui	state: "Piauí"
GA010	S · A	mapas do amapa	state: "Amapá"
GA011	S · A	mapas do maranhao	state: "Maranhão"
GA012	S · A	mapas da paraiba	state: "Paraíba"
GA013	S · A	mapas de cuiaba	city: "Cuiabá"
GA014	S · A	mapas de goiania	city: "Goiânia"
GA015	S · A	mapas de belem	city: "Belém"
GA016	S · A	mapas de brasilia	city: "Brasília"
GA017	C · A	cartas do rio em 50k	state: "Rio de Janeiro"; scale: "1:50.000"; aceita-se também city no lugar de state
GA018	C · A	mapas detalhados do Amazonas	scale: "1:25.000"; state: "Amazonas"
GA019	S · A	cartas em pequena escala do sul	scale: "1:250.000"
GA020	S · A	produtos em média escala	scale: "1:50.000"ou "1:100.000"
GA021	C · A	cartas de Goiás em 25k	state: "Goiás"; scale: "1:25.000"
GA022	C · A	cartas do Tocantins em 50k	state: "Tocantins"; scale: "1:50.000"
GA023	C · A	cartas de Mato Grosso do Sul em 100k	state: "Mato Grosso do Sul"; scale: "1:100.000"

ID	Cat.	Consulta	Leituras aceitas
GA024	C · A	cartas de Pernambuco em 250k	state: "Pernambuco"; scale: "1:250.000"

A.11 Consultas amplas, com muitos produtos (G-P)

Total desta família: **13 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GP001	S	todas as cartas de Rondônia	state: "Rondônia"
GP002	S	todas as cartas do Tocantins	state: "Tocantins"
GP003	S	todas as cartas do Piauí	state: "Piauí"
GP004	S	todas as cartas de Mato Grosso	state: "Mato Grosso"
GP005	S	todas as cartas do Acre	state: "Acre"
GP006	S	todos os produtos do 1° Centro de Geoinformação	supplyArea: "1° Centro de Geoinformação"
GP007	S	todos os produtos do 2° Centro de Geoinformação	supplyArea: "2° Centro de Geoinformação"
GP008	S	todos os produtos do 3° Centro de Geoinformação	supplyArea: "3° Centro de Geoinformação"
GP009	S	todos os produtos do 4° Centro de Geoinformação	supplyArea: "4° Centro de Geoinformação"
GP010	S	todos os produtos do 5° Centro de Geoinformação	supplyArea: "5° Centro de Geoinformação"
GP011	C	cartas na escala 1:2.000 do Rio Grande do Norte	scale: "1:2.000"; state: "Rio Grande do Norte"
GP012	C	cartas na escala 1:5.000 de Alagoas	scale: "1:5.000"; state: "Alagoas"
GP013	C	cartas na escala 1:250.000 do Rio Grande do Sul	scale: "1:250.000"; state: "Rio Grande do Sul"

A.12 Consultas de fronteira do domínio (G-F)

Total desta família: **7 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GF001	F	me ajuda	não chamar a ferramenta (sem intenção de busca)
GF002	F	cartas de Marte	não chamar a ferramenta (fora da Terra)
GF003	F	cartas da Argentina	não chamar a ferramenta (fora do território brasileiro)
GF004	F	qual a previsão do tempo para amanhã em Manaus?	não chamar a ferramenta (outro assunto)

ID	Cat.	Consulta	Leituras aceitas
GF005	F	quanto custa uma carta topográfica?	não chamar a ferramenta (pergunta de preço, não de busca)
GF006	F	mapa do tesouro pirata	não chamar a ferramenta (ficção)
GF007	S	cartas do estado 42	não chamar a ferramenta (observacional, P6: valor impossível de representar)

A.13 Requisitos exercitados pelas consultas

O Quadro 6 relaciona os grupos de consultas aos requisitos funcionais da Seção 4.2. Todas as consultas exercitam a tradução (RF2), a seleção do modelo por configuração (RF4) e o registro dos traços de execução (RF5), pois cada rodada do *pipeline* de avaliação escolhe o modelo por parâmetro e grava um registro por consulta. O recebimento pela rota REST (RF1) e a execução da busca no banco (RF3) são exercitados pela API e pela demonstração ponta a ponta descritas no Capítulo 4, e não entram nas métricas, que avaliam a camada de tradução.

Quadro 6 – Grupos de consultas do *dataset* e aspectos da tradução que exercitam

Grupo de consultas	O que exige do modelo	Requisitos
Todas (310)	Preencher os campos da ferramenta sustentados pela consulta, na forma canônica, sem acrescentar campos	RF2, RF4, RF5
Com código MI/INOM (M)	Preservar o código em keyword , normalizando grafias compactadas do INOM	RF2
Com tempo relativo (T)	Resolver a expressão em datas ISO a partir da data de referência informada e escolher entre período de publicação e de criação	RF2
Com ordenação (O)	Preencher sortField , sortDirection e, quando pedido, limit , sem ordenar quando a consulta não pede	RF2
Ambíguas/informais (A)	Normalizar siglas, grafias sem acento, apelidos e expressões qualitativas sem dicionário de normalização	RF2
Fora do domínio (F)	Reconhecer que o catálogo não atende ao pedido e não chamar a ferramenta	RF2
Observacionais (4)	Comportamento diante de valores impossíveis ou de pedidos de resposta discutível (reportado, não pontuado)	RF2, RF5

Fonte: Elaborado pelo autor. As contagens por categoria estão na Tabela 5.

A.14 Auditoria do *dataset*

A auditoria segue o procedimento da Seção 5 do manual e é reproduzível a partir do repositório. Seus artefatos ficam em `data/auditoria/`:

- `relatorio_auditoria.md` e `auditoria.json`: resultado das checagens automáticas (estrutura, rastreabilidade, anotador por regras e consistência), das medidas de concordância e da adjudicação;
- `auditoria_casos.csv`: uma linha por consulta, com o gabarito legível, as evidências localizadas pelo anotador por regras, os apontamentos e as divergências com a anotação independente;
- `anotacao_independente.json`: a anotação às cegas das 310 consultas, com a justificativa de cada uma;
- `adjudicacao.json`: as decisões de adjudicação, com o gabarito anterior a cada decisão;
- `revisao_humana_amostra.csv`: amostra estratificada por origem (40 consultas, somente 42) para conferência manual do gabarito.

O comando que refaz a auditoria e regenera as tabelas deste trabalho é:

```
pfc-auditar --anotacao-independente data/auditoria/anotacao_independente.json \
--paper <diretorio-do-texto>
```

A Tabela 31 detalha, por campo, a concordância entre a leitura preferencial do gabarito e a da anotação independente, e o Quadro 7 lista todas as decisões de adjudicação. As divergências de leitura preferencial nas consultas P13, P21 e P22 decorrem da regra que mantém, na camada P, a anotação original da equipe do 1º CGEO como preferencial; em todas elas a leitura do anotador está entre as leituras aceitas.

Quadro 7 – Decisões de adjudicação da auditoria do *dataset*

Consultas	Divergência	Decisão	Justificativa
N36, N37, GF001, GF002, GF003, GF004, GF005, GF006 <i>anotação independente</i>	o gabarito classificava a consulta como observacional; o anotador a tratou como fora do domínio, com gabarito determinado 'não chamar'	corrigido	P4 e §3 do manual: consulta sem intenção de busca no acervo brasileiro tem comportamento correto determinado (não chamar); P6 reserva 'observacional' às consultas cujo comportamento correto não é determinável. A consulta passa às métricas principais, na categoria F (fora do domínio).
N33 <i>anotação independente</i>	o gabarito exigia project = Mapeamento Sistemático; o anotador anotou só state = São Paulo e listou project como leitura alternativa, pois 'cartografia sistemática' não é nome nem apelido informado na ferramenta	ampliado	As duas leituras são razoáveis (P3): a expressão designa o programa pelo termo distintivo do nome, mas não consta da descrição da ferramenta. project passa a campo opcional; a regra de project do manual passa a registrar o caso.
GF007 <i>anotação independente</i>	ambos classificam a consulta como observacional; o gabarito indica 'não chamar' e o anotador, 'chamar sem parâmetros'	mantido	Consulta observacional (P6, valor impossível de representar): o comportamento é reportado, mas não pontuado, de modo que a divergência não afeta nenhuma métrica. Mantém-se a indicação 'não chamar', coerente com N38 (escala 1:10.000.000).
P14 <i>consistência</i>	'detalhado' é anotado como 1:25.000 em todo o dataset, mas P14 aceita 1:25.000 ou 1:50.000	mantido	As escalas explícitas '(25k ou 50k)' prevalecem sobre o qualificativo 'detalhado' (regra de scale: duas escalas explícitas, aceita-se qualquer uma).
P13 <i>anotação independente (medida estrita)</i>	leitura preferencial diferente dentro do conjunto aceito: scale 1:25.000 no gabarito, 1:50.000 no anotador (ambas aceitas por 'maior que 100k')	mantido	Na camada P, a anotação original da equipe do 1º CGEO é mantida como leitura preferencial (manual, §4); a escolha da preferencial não afeta a pontuação.
P21, P22 <i>anotação independente (medida estrita)</i>	leitura preferencial diferente dentro do conjunto aceito: sortField creationDate no gabarito, publicationDate no anotador (ambos aceitos, sem pista ligada à ordenação)	mantido	Na camada P, a anotação original da equipe do 1º CGEO é mantida como leitura preferencial (manual, §4); a escolha da preferencial não afeta a pontuação.
GO016 <i>anotação independente (medida estrita)</i>	o gabarito aceita sortField publicationDate ou creationDate; o anotador aceitou só publicationDate	mantido	Em 'as cartas mais recentes publicadas entre 2022 e 2023', o verbo qualifica o filtro de período, não a ordenação; sem pista ligada à ordenação, aceitam-se os dois campos (regra de ordenação). É o mesmo raciocínio do anotador em P22.

Fonte: Elaborado pelo autor a partir de `data/auditoria/adjudicacao.json`.

Tabela 31 – Concordância por campo entre o gabarito e a anotação independente (leitura preferencial, após a adjudicação)

Campo	Gabarito	Anotador	Ambos	Kappa	Valor igual
keyword	62	62	62	1,000	62/62
scale	84	84	84	1,000	83/84
productType	44	44	44	1,000	44/44
state	130	130	130	1,000	130/130
city	27	27	27	1,000	27/27
supplyArea	40	40	40	1,000	40/40
project	11	10	10	0,951	10/10
publicationPeriod	51	51	51	1,000	51/51
creationPeriod	10	10	10	1,000	10/10
sortField	29	29	29	1,000	27/29
sortDirection	29	29	29	1,000	29/29
limit	21	21	21	1,000	21/21

Fonte: Elaborada pelo autor. Gabarito, Anotador e Ambos: consultas em que o campo está na leitura preferencial de cada um. Kappa de Cohen sobre a presença do campo. Valor igual: entre as consultas em que ambos o anotam; períodos comparados pelos limites, textos livres após normalização.

APÊNDICE B – ESPECIFICAÇÃO DOS AMBIENTES DE EXECUÇÃO

Este apêndice documenta o *hardware* e o *software* dos ambientes de execução descritos na Seção 3.5.2. Em todos eles, cada rodada grava um manifesto (`results/<modelo>/manifesto.json`) com as versões de *software*, a configuração de inferência, a data de referência e os *hashes* do *dataset* e da definição da ferramenta.

B.1 Estação de referência

Os dados da estação foram coletados executando-se, no próprio equipamento, os comandos PowerShell da Seção B.1.1; a saída é transcrita na Seção B.1.2.

B.1.1 Comandos PowerShell de coleta

```
# CPU
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
    NumberOfLogicalProcessors, MaxClockSpeed

# Memoria RAM (total em GB e detalhes por pente)
(Get-CimInstance Win32_PhysicalMemory |
    Measure-Object -Property Capacity -Sum).Sum / 1GB
Get-CimInstance Win32_PhysicalMemory | Select-Object Manufacturer,
    Capacity, Speed, ConfiguredClockSpeed, MemoryType, DeviceLocator

# GPU (geral)
Get-CimInstance Win32_VideoController | Select-Object Name,
    AdapterRAM, DriverVersion, VideoProcessor

# VRAM real (se GPU NVIDIA)
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv

# Sistema operacional
Get-CimInstance Win32_OperatingSystem | Select-Object Caption,
    Version, BuildNumber, OSArchitecture

# Disco principal
Get-PhysicalDisk | Select-Object FriendlyName, MediaType, Size, BusType

# Versoes de software
python --version
ollama --version
```

```
psql --version
```

B.1.2 Saída coletada

```
# CPU
Name                                NumberOfCores  NumberOfLogicalProcessors  MaxClockSpeed
12th Gen Intel(R) Core(TM) i7-12700H          14                20                2300

# RAM
RAM total (GB): 32
Pente 1: Kingston, 16 GB DDR4, 3200 MHz (Controller0-ChannelA-DIMM0)
Pente 2: Kingston, 16 GB DDR4, 3200 MHz (Controller1-ChannelA-DIMM0)

# GPU
Name: NVIDIA GeForce RTX 3050 Laptop GPU
AdapterRAM 4 293 918 720; DriverVersion 32.0.15.6607
VideoProcessor NVIDIA GeForce RTX 3050 Laptop GPU
Name: Intel(R) Iris(R) Xe Graphics
AdapterRAM 2 147 479 552; DriverVersion 32.0.101.6790
VideoProcessor Intel(R) Iris(R) Xe Graphics Family

# nvidia-smi
NVIDIA GeForce RTX 3050 Laptop GPU, 4096 MiB, 566.07

# Sistema operacional
Microsoft Windows 11 Pro, versao 10.0.26200, Build 26200, 64 bits

# Disco principal
INTEL SSDPEKNU512GZ, SSD NVMe, 512 GB

# Software
Python 3.14.4
Ollama 0.34.0 (cliente e servidor)
PostgreSQL 16.4 + PostGIS 3.4 em container Docker 29.6.1
```

B.2 Rodadas completas em GPU de nuvem

As rodadas completas dos quatro modelos locais foram executadas numa máquina virtual do Google Colab com GPU NVIDIA T4 de 16 GB de VRAM (Tabela 8), pelo caderno `notebooks/avaliacao_colab.ipynb` do repositório. O caderno instala o Ollama pelo instalador oficial, baixa os modelos pelas mesmas *tags* usadas na estação de referência (`qwen3:4b-instruct-2507-q4_K_M`, `gemma4:e4b-it-qat`, `gemma4:e2b-it-qat` e `mistral-nemo:12b`), instala o pacote do projeto a partir do repositório e executa `pfc-avaliar` com três repetições por modelo, data de referência fixa (14 de setembro de 2026) e sem execução de SQL, pois a busca no banco não entra nas métricas. Nenhum serviço de LLM em nuvem é usado nesse ambiente: o Ollama executa dentro da máquina virtual, com os pesos baixados nela; somente a GPU é provida pela nuvem.

B.3 Referência em nuvem (Groq)

A referência em nuvem usa a API do Groq, compatível com a da OpenAI, por meio da classe `ChatOpenAI` do pacote `langchain-openai` (versão 1.6.6), com `langchain-core` 1.6.4, a partir da estação de referência. O *prompt* de sistema, a definição da ferramenta, a temperatura (zero) e o limite de 1024 *tokens* de saída são os mesmos das rodadas locais. O esforço de raciocínio foi fixado em `low` para o `openai/gpt-oss-20b` e o `openai/gpt-oss-120b`, o menor valor aceito por esses modelos. Cada modelo executou as 310 consultas uma vez, com data de referência de 24 de setembro de 2026 e intervalo mínimo de 11 s entre chamadas; quando o provedor sinaliza o esgotamento do limite de taxa do plano gratuito, a execução aguarda o tempo indicado e retoma a mesma consulta, sem contá-la como erro. Uma chamada de ferramenta rejeitada pelo provedor por argumentos inválidos é registrada como erro do modelo.

APÊNDICE C – REPOSITÓRIO DE CÓDIGO E REPRODUÇÃO DOS EXPERIMENTOS

O código-fonte, o *dataset*, o manual de anotação, os registros de auditoria, os registros de execução de todas as rodadas e as fontes deste texto estão no repositório do projeto, <<https://github.com/vhenequim/bdgex-busca-tool-calling>>, sob a licença MIT; as partes derivadas do protótipo do 1º CGEO mantêm a licença MIT original, como registrado no arquivo `THIRD_PARTY_NOTICES.md`. A estrutura principal é a seguinte:

```
bdgex-busca-tool-calling/
  src/pfc_busca/
    schema.py          definicao da ferramenta buscar_catalogo (12 campos)
    prompts.py         prompt de sistema (com a data de referencia)
    agent.py           traducao via ChatOllama.bind_tools() (modelos locais)
    agent_groq.py       traducao via ChatOpenAI (referencia em nuvem)
    tools.py, db.py     busca no PostgreSQL/PostGIS (psycopg)
    pipeline.py         ciclo completo: traducao, busca e resposta
    api.py             rotas POST /api/search e GET /api/health (FastAPI)
  evaluation/
    manual_cases.py    camadas P e N, com rastreabilidade e notas
    gabarito.py        leituras multiplas (valor alternativo, opcional, trocas)
    relative_time.py   regras de tempo relativo (leituras aceitas)
    dataset_builder.py pfc-dataset: constroi e valida data/dataset.json
    audit.py           pfc-auditar: auditoria do dataset
    run_evaluation.py  pfc-avaliar: execucao, manifesto, repontuacao
    metrics.py         acuracia, P/R/F1 por campo, latencia
    report.py          pfc-relatorio: tabelas, figuras, Wilson, McNemar
    conferencia.py     pfc-conferir: conferencia das rodadas
  scripts/generate_dataset_paper.py  gerador da camada G (semente 42)
  docs/manual_de_anotacao.md         manual de anotacao
  data/dataset.json                  dataset (310 consultas)
  data/auditoria/                    artefatos da auditoria
  results/                           registros e manifestos das rodadas
  notebooks/avaliacao_colab.ipynb    rodadas completas em GPU de nuvem
  tests/                             testes automatizados (pytest)
  texto/                             fontes LaTeX e PDF deste trabalho
```

Os passos para reproduzir os resultados, a partir de um ambiente com Python 3.12 ou superior e o Ollama instalado, são:

```
pip install -e .          # instala o pacote e os comandos pfc-*
pfc-dataset               # reconstrói data/dataset.json (mesmo hash)
```

```
pfc-auditar --anotacao-independente data/auditoria/anotacao_independente.json
ollama pull qwen3:4b-instruct-2507-q4_K_M
pfc-avaliar --modelo qwen3:4b-instruct-2507-q4_K_M --repeticoes 3 \
    --hoje 2026-09-14 --sem-sql
pfc-conferir                # confere as rodadas antes de consolidar
pfc-relatorio --modelos qwen3:4b-instruct-2507-q4_K_M,gemma4:e4b-it-qat,...
pytest                     # testes do calculo das metricas e do dataset
```

A pontuação é sempre recalculada contra o gabarito vigente, lido do arquivo `data/dataset.json`: o comando `pfc-relatorio` relê os registros de execução e reaplica o gabarito, de modo que as tabelas do Capítulo 5 podem ser regeneradas sem nova execução dos modelos.